



Universitatea
Politehnica
din București



Facultatea de
Automatică și
Calculatoare



Departamentul
Calculatoare

BAZE DE DATE



Capitolul 1

Concepte de baze de date



Definitii

- **Baza de date** reprezinta o modalitate de stocare a datelor pe un suport extern de memorie, intr-o anumita structura, in vederea interogarilor concurente si prelucrarilor ulterioare pentru extragerea informatiilor.
- **Sistem de gestiune a bazei de date(SGBD)** este un ansamblu de programe software care permit utilizatorilor definirea obiectelor, administrarea bazei de date si accesul concurent la baza de date. Cele mai cunoscute si utilizate sisteme de gestiune sunt Oracle, Microsoft SQL Server, IBM DB2, IBM Informix, MySQL, PostgreSQL, Sybase, etc.



Definitii

- **Dictionarul bazei de date** este acea componenta a unui SGBD care contine informatii despre obiectele din baza de date si parametrii de sistem, cum ar fi:
 - Structurile tabelurilor, indecsii si constrangerile de integritate definite pe ele;
 - Spatiul fizic si organizarea logica a fisierelor;
 - Userii creati pe baza de date si drepturi de acces.
- **Organizarea datelor** reprezinta procesul de definire, structurare si relationare a datelor in colectii de date.
- **Colectie de date** este un ansamblu de date organizate pe anumite criterii de functionalitate, denumita tabela (pentru bazele de date relaționale) și obiect (pentru bazele de date orientate obiect).



Definitii

- **Structura bazei de date** reprezinta o colectie de descrieri statice ale tipurilor de entitati impreuna cu relatiile logice stabilite intre ele.
- **Entitate** reprezinta un obiect al bazei de date care are o reprezentare unica(de ex. tabelele *Studenti*, *Grupe*, *Catalog*, etc.).
- **Atribut** este o proprietate ce descrie o anumita caracteristica a unei entitati(de ex. pentru tabela *Studenti* attribute pot fi *nr_matricol*, *nume*, *grupa*, *specializare*, etc.).



Definitii

- **Relatiile logice** reprezinta asocierile dintre mai multe entitati care respecta anumite restrictii de functionalitate (un ex. de asociere intre tabelele *Studenti* si *Grupe* ar fi *nr_matricol*, *nume*, *cod_grupa*)
- **Fișier de date** reprezinta o colectie de date aflate in asociere, definit printr-o structura logica si una fizica.



Modele de baze de date

Regulile și conceptele care permit descrierea structurii unei baze de date formeaza **modelul datelor** :

- **Modelul ierarhic** - datele sunt organizate sub forma unui arbore, nodurile constand din inregistrari(date) iar arcele fiind referinte(pointeri) catre alte noduri(datele sunt organizate intr-o structura arborescenta de tip tata/fiu).
- Datele sunt grupate in inregistrari ce descriu o ierarhie de tip 1-1 sau 1-N (un parinte poate avea unul sau mai multi fii, in timp ce un fiu poate avea doar un parinte).



Modele de baze de date

- Datele sunt organizate in entitati si fiecare entitate poate avea mai multe attribute.
- Echivalentul unei entitati intr-o baza de date este tabela, iar a unui atribut este coloana.
- Modelul ierarhic este rar folosit in bazele de date moderne, cel mai cunoscut fiind sistemul de fisiere din Windows.



Modele de baze de date

- **Modelul retea** - datele sunt organizate sub forma unui graf orientat.
- Nodurile si arcele au aceeasi semnificatie ca la modelul ierarhic.
- Modelul retea completeaza modelul ierarhic iar datele sunt grupate in inregistrari ce descriu o ierarhie de tip 1-1, 1-N, K-N (un parinte poate avea unul sau mai multi fii, iar un fiu poate avea mai multi parinti).



Modele de baze de date

- Modelul retea a fost conceput de Charles Bachman si datele sunt structurate sub forma unui graf orientat, fiecare nod putand avea mai multe inregistrari parinte si mai multi fii.
- Este conceput ca o metoda flexibila de reprezentare a obiectelor si relatiilor dintre ele.
- In modelul retea fiecare inregistrare poate avea parinti si fii multipli, formand o latice care permite o mai buna modelare a relatiilor dintre entitati.



Modele de baze de date

- **Modelul entitate-asociere** - este folosit in proiectarea conceptuala de nivel inalt a bazelor de date.
- Modelul consta intr-o abordare grafica a proiectarii bazelor de date.
- Datorita simplitatii si expresivitatii sale a fost adoptat de comunitatea stiintifica precum si de producatorii de software si va fi tratat in detaliu intr-un capitol separat.



Modele de baze de date

- **Modelul relational** - datele sunt organizate sub forma de tabele ce pot fi relationate intr-o ierarhie de tip 1-1, 1-N, K-N.
- Modelul este sustinut si de un limbaj standard de manipulare a datelor.
- Este cel mai utilizat model de baze de date si de aceea va fi tratat mai in detaliu intr-un capitol ulterior.



Modele de baze de date

- **Modelul obiect** - datele sunt reprezentate sub forma de obiecte si sunt folosite in programarea orientata pe obiecte.
- Un obiect este o unitate de program care este folosita in constructia blocurilor de program.
- Fiecare obiect poate sa receptioneze parametri, sa prelucreze date si sa transmita rezultate altor obiecte.



Modele de baze de date

- **Modelul obiect-relational** - se bazeaza pe tehnici de programare *Object-Relational Mapping* (O/RM, ORM, si O/R mapping) pentru conversia datelor din diferite sisteme de baze de date si limbaje de programare orientate pe obiecte.
- Ele au ca efect crearea unei “baze de date virtuale” care poate fi accesata in limbaje de programare specifice.
- Pentru aceasta sunt disponibile pe piata diferite pachete de programe dar se pot crea si propriile tools-uri ORM.



Modele de baze de date

- Cel mai raspandit model de baze de date este cel relațional, in care datele sunt stocate in tabele. Popularitatea acestui model se datoreaza simplitatii sale (din punct de vedere al utilizatorului) si a posibilitatii de definire a unor limbaje neprocedurale de descriere si manipulare a datelor.
- Termenul de relatie (care da denumirea modelului) provine din matematica, iar reprezentarea intuitiva a unei relatii este o tabela de asociere intre doua coloane a doua tabele diferite.
- In cazul modelului relational descrierea structurii unei baze de date consta in principal din descrierea tabelelor componente: denumire, lista de coloane si tipul datelor stocate in acestea.



Sistemul de gestiune a bazei de date (SGBD)

- Sistemele de gestiune a bazelor de date structurate au avut, aproximativ, urmatoarea evolutie de-a lungul timpului:
 - Sisteme bazate pe fisiere de date (1950)
 - SGBD bazate pe modelul de date ierarhic(1960)
 - SGBD bazate pe modelul de date retea (1970)
 - SGBD entitate-asociere (1975)
 - SGBD relationale (1980, varianta comerciala)
 - SGBD obiect-relationale(1990)
 - SGBD orientate spre aplicatii (web, baze de date spatiale, temporale, multimedia, etc.) (2000)
 - SGBD de depozitare a volumelor mari de date(data warehousing, data mining, etc.) (2010)
 - SGBD de depozitare a datelor Big Data(de ordinul petabytes de date) (2015)
 - SGBD native in mediul cloud(2020)



Sistemul de gestiune a bazei de date (SGBD)

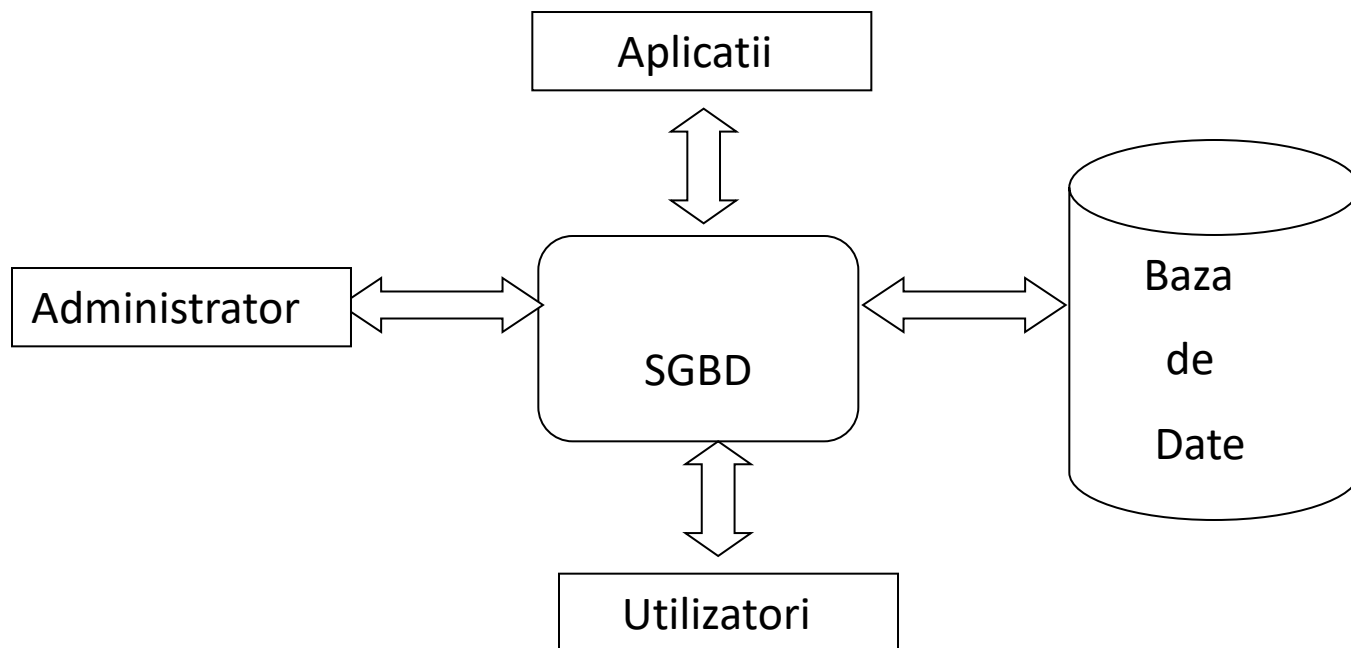


Figura 1. Schema bloc de functionare a unui SGBD



Sistemul de gestiune a bazei de date (SGBD)

- Functiile principale ale unui SGBD sunt:
 - Definirea structurilor tabelare;
 - Definirea relatiilor dintre tabele;
 - Definirea constrangerilor de integritate a datelor;
 - Manipularea datelor cu ajutorul unui limbaj SQL;
 - Controlul drepturilor de acces la baza de date;
 - Controlul accesului concurent al userilor la date;
 - Administrarea automata a catalogului bazei de date;
 - Administrarea spatiului fizic si logic de stocare;
 - Controlul backup-urilor (copii de siguranta).



Definirea structurilor tabelare

- Un SGBD trebuie sa includa posibilitatea definirii stucturii obiectelor care formeaza baza de date.
- In cazul bazelor de date relationale aceasta consta in principal in posibilitatea crearii si modificarii structurii tabelelor si constrangerilor de integritate asociate acestora.
- Limbajul prin care se realizeaza aceste operatii se numeste DDL (*Data Definition Language*)
- In sistemele relationale bazate pe SQL aceste operatii au fost incluse in limbaj prin intermediul comenzilor de tip CREATE (pentru creare), ALTER (modificare) sau DROP(stergere din dictionar).



Definirea structurilor tabelare

- Structura unei tabele este data de urmatoarele specificatii de definire:
 - Definirea coloanelor si tipurile acestora;
 - Definirea constrangerilor de integritate;
 - Definirea tablespace-lui unde se creeaza tabela;
 - Definirea parametrilor logici si fizici.
- Definirea unei tabele trebuie sa respecte anumite reguli:
 - Sa nu existe duplicare de randuri si coloane ;
 - Nu trebuie respectata o anumita ordine a coloanelor;
 - Tipurile de coloane trebuie definite in concordanta cu tipurile de date ;
 - Relatiile intre tabele se fac pe coloane de acelasi tip .



Integritatea datelor

- Constrangerile de integritate reprezinta anumite reguli pe care datele trebuie sa le respecte la nivel de tabela sau in relatiile cu alte tabele.
- Aceste reguli sunt verificate automat in cazul operatiilor de inserare, stergere si modificare, iar in cazul in care nu se valideaza se genereaza o eroare si tranzactia nu se efectueaza.
- In felul acesta este asigurata o mai mare siguranta in ceea ce priveste corectitudinea datelor si reducerea erorii umane.



Integritatea datelor

- Constrangerile de integritate pot fi :
 - **Valori nenule** – inregistrarile nu pot contine valori nule;
 - **Cheie unica** – defineste o cheie unica pe una sau mai multe coloane (nu pot fi mai multe inregistrari cu aceleasi valori pe coloanele respective, dar se accepta valori nule);
 - **Cheie primara** – defineste o cheie primara la nivel de coloana sau tabela (nu pot fi mai multe inregistari cu aceeasi cheie primara, care nu accepta valori nule);
 - **Cheie externa** – defineste o cheie externa (tabela se relationeaza cu alta tabela pe o cheie unica sau cheie primara);
 - **Verificare** – se forteaza o verificare de indeplinire a unor conditii pe o coloana.



Manipularea datelor

- Manipularea datelor se refera la operatiile de lucru cu datele inregistrate intr-o baza de date si se realizeaza cu ajutorul unui limbaj DML (*Data Manipulation Language*) care este inclus in limbajul SQL;
- Operatii principale de manipulare a datelor sunt urmatoarele:
 - **Inserarea de date**(adaugarea de linii noi in tabele);
 - **Stergerea de date** (stergerea de linii din tabele);
 - **Modificarea datelor** (modificarea continutului unor linii existente in tabele);
 - **Interogarea datelor** (selectarea liniilor dupa anumite criterii de interogare).



Categorii de utilizatori

- **Utilizatori privilegiati** - Acestia sunt utilizatori care au dreptul de a afecta toate tipurile de operatii puse la dispozitie de catre sistem (**administratorul bazei de date**);
- **Utilizatori neprivilegiati** - Acestia sunt utilizatorii obisnuiti ai SGBD-ului si dispun de drepturile de acces care le-au fost alocate de catre administratorul bazei de date;
- **Dezvoltatori de aplicatii** - In aceasta categorie intra toti cei implicati in crearea unei aplicatii si in activitatea de mentenanta;
- **Utilizatori de aplicatii** - Sunt cei care au acces la baza de date prin intermediul interfetelor pentru care sunt autorizati.



Niveluri de reprezentare a datelor

- Exista mai multe niveluri de reprezentare a datelor, fiecare avand caracteristici specifice in functie de perspectiva sub care este vazuta, asa cum se vede in Figura 2:
 - **Nivelul intern** – descrie organizarea interna a datelor;
 - **Nivelul conceptual** – descriere modelul de date(de exemplu modelul relational);
 - **Nivelul extern** - nivelul utilizatorilor care acceseaza baza de date.



Niveluri de reprezentare a datelor

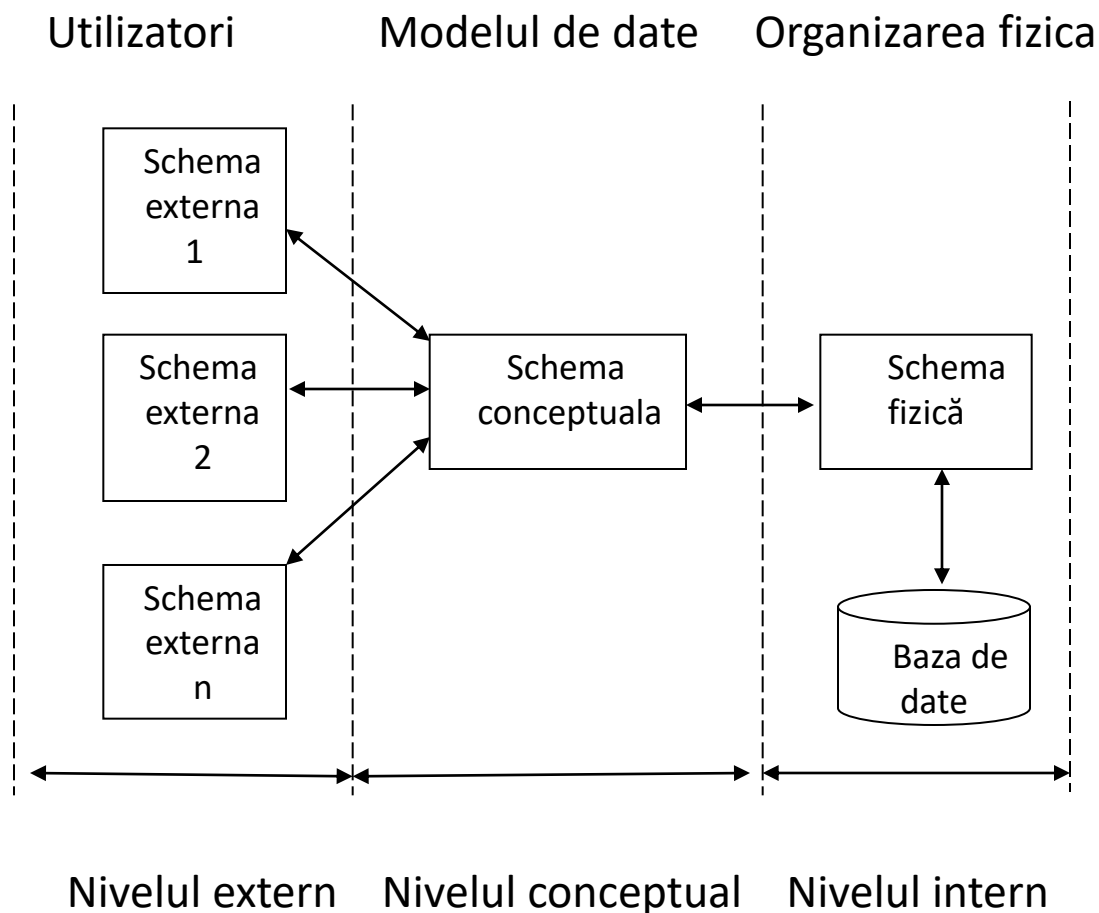


Figura 2. Niveluri de reprezentare a datelor



Nivelul intern

- Descrierea bazei de date la **nivel intern** poarta numele de **schema fizica** si sistemul de gestiune a bazelor de date pune la dispozitie facilitati pentru inregistrarea si modificarea acesteia.
- La acest nivel baza de date este descrisa din perspectiva stocarii sale pe dispozitivele fizice: identificarea discurilor si a cailor unde este stocata, numele fisierelor care formeaza baza de date, structura fizica a acestora, etc.



Nivelul conceptual

- Descrierea bazei de date la **nivel conceptual** poarta numele de **schema conceptuala** (numita uneori si **schema logica**) a bazei de date.
- In modelul relational schema consta in:
 - Tabelele care formeaza baza de date;
 - Structura (coloanele) fiecărei tabele;
 - Tipul de date asociat fiecărei coloane;
 - Elementele pe baza carora se realizeaza interconectarea tabelelor (coloane comune);
 - Constrangeri de integritate;
 - Operatii declansate automat la modificarea unor elemente ale bazei de date;



Nivelul extern

- Diferite categorii de utilizatori ai unei baze de date au nevoie in activitatea lor numai de anumite parti specifice ale schemei conceptuale.
- **Nivelul extern** se ocupa de descrierea acestor parti si poarta numele de **scheme externe** (echivalentul userilor creati pe baza de date).
- O baza de date are asociata o singura schema fizica si o singura schema conceptuala, dar mai multe scheme externe.
- Pentru administratorului bazei de date schema externa coincide cu schema conceptuala, deoarece are toate drepturile de acces.
- Celelalte categorii de utilizatori acceseaza baza de date doar prin intermediul schemelor externe specifice acestora, pe baza drepturilor de acces alocate.



Independenta datelor

- Existenta celor trei niveluri de descriere permite definirea conceptului de independenta intre datele stocate in baza de date si aplicatiile care utilizeaza aceste date.
- Conceptul de independenta a datelor a aparut odata cu dezvoltarea sistemelor complexe de aplicatii, pentru care cablarea informatiilor structurate in cadrul programului constituie o bariera in calea dezvoltarii si modificarii acestora.



Independenta datelor

- ***Independenta logica*** reprezinta posibilitatea de schimbare a schemei conceptuale a bazei de date fara modificarea schemelor externe. Conditia este ca modificarea sa nu elimine niciunul dintre elementele necesare translatiei de la schema externa la schema conceptuala.
- Unele operatii implica insa si modificarea definirii schemelor externe.
- Exemple de modificare a schemei conceptuale ar fi:
 - Adaugarea de noi tabele in baza de date;
 - Adaugarea de noi coloane in tabelele existente;
 - Adaugarea de noi constrangeri de integritate;
 - Modificarea numelor tabelor si coloanelor existente;
 - Modificarea in anumite limite a tipurilor de date;
 - Restructurarea bazei de date.



Independenta datelor

Exemple:

- Sa consideram o baza de date continand la nivel conceptual o tabela cu date despre studenti cu urmatoarea structura:

Studenti (*nr_matricol, nume, cod_facultate, medie*)

si *n* scheme externe continand tabelele virtuale

Studenti-1, Studenti-2, ..., Studenti-n definite astfel:

Studenti-n (*nr_matricol, nume, cod_facultate, medie*)

care contine liniile din tabela **Studenti** unde *cod_facultate* = *n*.

- In cazul modificarii bazei de date prin adaugarea unei noi tabele cuprinzand lista specializarilor din cadrul facultatii si a unei noi coloane in tabela **Studenti**, pentru a specifica la ce specializare este inscris fiecare student, tabelele cu datele studentilor din schemele externe vor ramane aceleasi, daca schimbam structura astfel:



Independenta datelor

- Baza de date conceptuala:

Studenti (*nr_matricol, nume, cod_facultate, medie, cod_specializare*)

Specializari (*cod_specializare, den_specializare*)

- Schemele externe:

Studenti-n (*nr_matricol, nume, cod_facultate, medie*) va contine in continuare valorile de pe coloanele *nr_matricol, nume, cod_facultate* si *medie* din liniile din tabela **Studenti** unde *cod_facultate* = *n*.

- Independenta logica implica folosirea de catre SGBD a informatiilor de definire a schemelor externe, stocate in catalogul sistemului, pentru conversia oricarei operatii din structura schemei externe a aplicatiei sau utilizatorului care a lansat-o, in structura schemei conceptuale a bazei de date.



Independenta datelor

- ***Independenta fizica*** reprezinta posibilitatea de schimbare a schemei fizice a bazei de date fara modificarea schemei conceptuale si implicit a schemelor externe. Aceasta da posibilitatea reorganizarii fizice a bazei de date fara afectarea aplicatiilor care o folosesc.
- Alte operatii pot fi suportate numai prin modificarea catalogului sau a fisierelor de configurare pe care SGBD-ul le foloseste pentru a face translatia de la schema conceptuala la schema fizica, cum ar fi:
 - Schimbarea dispozitivelor fizice pe care este stocata baza de date;
 - Schimbarea numelor fisierelor fizice in care este stocata baza de date sau a directoarelor unde acestea sunt plasate;



Independenta datelor

- Adaugarea de noi structuri de cautare rapida (indecsi) pentru cresterea vitezei de executie a anumitor operatii;
 - Schimbarea in anumite conditii a structurii fizice a fisierelor bazei de date;
 - Schimbarea unor parametri ai sistemului de gestiune care afecteaza modul in care datele sunt stocate la nivel fizic, de exemplu dimensiunea blocului de date.
- Deoarece nu toate sistemele de gestiune a bazelor de date implementeaza total cele trei niveluri de descriere, posibilitatea de a asigura cele doua tipuri de independenta a datelor este conditionata de facilitatile oferite de catre sistemul de gestiune.



Capitolul 2

Modelul Relational



Modelul relational

- Modul de organizarea a datelor intr-o baza de date trebuie sa respecte un anumit model, iar operatia in sine se numeste **modelarea datelor**.
- Asa cum am aratat intr-un capitol anterior, au existat mai multe modele de baze de date in evolutia sistemelor de gestiune a bazelor de date, fiecare aducand ceva nou pentru modelarea datelor si o crestere a performantelor.
- **Modelul Relational (MR)** a fost introdus in anul 1970 de Edgar Frank Codd si este modelul cel mai utilizat de catre sistemele de gestiune aparute dupa anul 1980.
- Obiectele bazei de date se numesc **entitati** si sunt relationate intre ele prin anumite **constrangeri** care formeaza asa numita **diagrama de relatii**.



Modelul relational

- In **Modelul Relational** datele sunt stocate sub forma tabelara si aduce o serie de avantaje fata de modelele anterioare:
 - datele sunt stocate sub forma de linii intr-o tabela;
 - s-a renuntat la folosirea pointerilor si navigarea prin structuri arborescente;
 - pentru accesarea datelor exista limbaje specializate de nivel inalt, de exemplu limbajul SQL;
 - permite introducerea de constrangeri de integritate, ceea ce duce la reducerea anomaliilor si a redundantei datelor;
 - asigura un control riguros al drepturilor de acces;
 - ofera facilitati sporite de securitate a datelor.



Elementele modelului relational

■ Domeniul

Domeniul reprezinta o multime de valori si este identificat printr-un nume.

- Un domeniu se poate defini fie prin enumerarea elementelor sale, fie prin specificarea unor caracteristici definitorii ale acestora.

Exemplu:

- Culori = {rosu, galben, albastru, violet, verde}
- Note = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10} sau
Note = $\{n \in \mathbb{N}^* \mid n \geq 1 \text{ si } n \leq 10\}$
- Char40 = {Multimea sirurilor de maxim 40 caractere}
- Intreg5 = {Multimea numerelor intregi pozitive din intervalul [0, 99999]}



Elementele modelului relational

- **Produsul cartezian pe domenii** este definit similar cu cel din teoria multimilor:

Fiind date ***n*** domenii $D_1, D_2, \dots, D_n$ produsul cartezian al domeniilor este multimea sirurilor de domenii:

$$D_1 \times D_2 \times \dots \times D_n = \{ (v_1, v_2, \dots, v_n) \mid v_i \in D_i, i = 1, \dots, n \}$$

- Trebuie mentionat ca in sirul de domenii care participa la un produs cartezian unele se pot gasi in mod repetat, de exemplu:

$$PC = \text{Intreg} \times \text{Char40} \times \text{Intreg} \times \text{Char40}$$



Elementele modelului relational

▪ Relatia

Relatia reprezinta o submultime a unui produs cartezian avand asociat un nume.

- Un exemplu de relatie apartinand produsului cartezian PC = Studenti x Specializari este:

Specializari _stud

```
{ (1011, 'Ionescu Silvia', 1, 'CTI'),  
  (1022, 'Popescu Daniel', 2, 'AI'),  
  (1033, 'Popa Cornel', 1, 'CTI') }
```

- **Elementele unei relatii** sunt denumite in literatura de specialitate **tupluri** (tuple). Relatia de mai sus contine 3 dintre elementele produsului cartezian din care provine (3 tupluri).



Elementele modelului relational

- O reprezentare intuitiva pentru relatia de mai sus, in care fiecare element al relatiei devine o linie a unei tabele si fiecare coloana corespunde unui domeniu din produsul cartezian de baza, este urmatoarea :

Specializari_stud

1011	Ionescu Silvia	1	CTI
1022	Popescu Daniel	2	AII
1033	Popa Cornel	1	CTI

- Deci o relatie se poate reprezenta ca o tabela de date, fiecare coloana avand asociat un anumit tip de date, impus de domeniul din care provine.



Elementele modelului relational

▪ Atributul

Atributul reprezinta o coloana a unei relatii avand asociat un nume.

- Deoarece o relatie are o reprezentare tabelara putem vorbi de “coloana a unei relatii” si putem sa facem urmatoarea similitudine: “atributele unei relatii” reprezinta “coloanele unei tabele”.
- Pentru relatia **Specializari_stud** putem defini urmatoarele attribute:
 - Nr_matricol – matricolul studentului
 - Nume_student – numele studentului
 - Cod_spec – codul specializarii
 - Den_specializare – denumirea specializarii



Elementele modelului relational

■ Schema

Schema unei relatii reprezinta structura relatiei compusa din **nume, attribute, domeniul atributelor** si **constrangerile de integritate** asociate.

- Datele dintr-o relatie se pot schimba in timp deoarece se pot adauga, modifica sau sterge linii, dar structura relatiei ramane constanta.
- Exista mai multe modalitati prin care se poate specifica schema unei relatii. In exemplele urmatoare prezentam cateva dintre acestea cu referire la relatia Medii:



Elementele modelului relational

Medii = Nr_matricol, Nume_student, Cod_grupa, Media

Medii (Nr_matricol, Nume_student, Cod_grupa, Media)

Medii (Nr_matricol: Intreg, Nume_student: Char40,
Cod_grupa: Char5, Media: Numar)

Medii (Nr_matricol intreg, Nume_student char(40),
Cod_grupa char(5), Media numar)

- In teoria bazelor de date relationale se foloseste si notatia urmatoare pentru o relatie:

R = ABCDE ,

care specifica ca schema relatiei **R** contine 5 attribute notate cu A, B, C, D si E.



Elementele modelului relational

▪ Cheia

Cheia unei relatii este o multime minimala de attribute ale caror valori identifica in mod unic un tuplu al relatiei respective. Deoarece o relatie nu accepta linii duplicat inseamna ca ele pot fi identificate in mod unic prin una sau mai multe coloane. Cheia unei relatii este o caracteristica a schemei acesteia si se defineste ca o constrangere de integritate.

- In tabela **Medii** tuplurile pot fi identificate in mod unic prin attributele (Nr_matricol, Nume_student) sau (Nr_matricol, Nume_student, Cod_grupa) dar ele nu indeplinesc si conditia de minimalitate.



Elementele modelului relational

- Daca consideram ca numarul matricol este unic pentru fiecare student atunci putem defini atributul Nr_matricol ca fiind cheia relatiei Medii, fiind indeplinita si conditia de minimalitate.
- Niciunul dintre celelalte attribute, luate separat, nu poate fi cheie, de exemplu Nume_student nu este cheie deoarece pot exista doi studenti cu acelasi nume si in acest caz nu pot fi identificate unic tuplurile .
- Daca facem referire la tabele, rezulta ca nu pot exista doua linii avand aceeasi combinatie de valori pe coloanele care formeaza cheia tablei respective (proprietate denumita in literatura de specialitate si unicitatea cheii).



Elementele modelului relational

- Trebuie specificat ca intr-o relatie se pot identifica uneori mai multe chei. Sa luam ca exemplu relatia: **Studenti**(Nr_matricol, Nume_student, Adresa, CNP, Data_nastere)
- Dupa cum se stie CNP-ul unei persoane este unic asa ca relatia Studenti poate avea doua chei Nr_matricol si CNP, ambele indeplinind conditia de unicitate si minimalitate.
- Observatie: Deoarece intr-o relatie nu pot exista doua tupluri identice, rezulta ca multimea tuturor atributelor relatiei formeaza o cheie sau contine cel putin o cheie, deci orice relatie are cel putin o cheie.



Elementele modelului relational

- In literatura de specialitate si in sistemele de gestiune a bazelor de date exista alte trei concepte legate de cheie si care vor fi prezentate in acest capitol si capitolele urmatoare:
- **Cheie primara** (*Primary Key*)
- **Cheie straina** (*Foreign Key*)
- **Supercheie** (*Superkey*)



Elementele modelului relational

■ Valori nule

Valoare nula (*null value*) este o valoare care modeleaza o informatie nespecificata si este o informatie inaplicabila pentru prelucrari de date(nu se pot aplica operatori aritmetici pe ea).

- Uneori, unele attribute ale unei relatii nu au nicio valoare specificata si in acest caz se spune ca atributul respectiv are o valoare nula.

Exemplu:

Studenti

Nr_matricol	Nume_student	CNP	Cod_Grupa
1011	Ionescu Silvia	2901028400772	331CC
1022	Popescu Daniel	<null>	332CC
1033	Popa Cornel	1911115451664	<null>



Elementele modelului relational

- In exemplul de mai sus studentul Popescu Daniel nu are specificat CNP-ul si spunem ca atributul CNP are valoare nula. Inregistrarea poate fi totusi identificata unic dupa atributul Nr_matricol care este cheie. Daca se doreste insa sa se identifice studentul dupa CNP atunci cererea de interogare nu va returna nicio linie.
- Valorile nule pot fi modificate ulterior prin comenzi DML, de exemplu in SQL se foloseste comanda UPDATE.
- Observatie: Intr-o tabela o cheie care identifica unic un tuplu nu trebuie sa contina attribute cu valoare nula.



Elementele modelului relational

■ **Constrangeri de integritate**

Constrangerile de integritate reprezinta un mecanism prin care un SGDB poate controla corectitudinea datelor in cazul operatiilor DML de manipulare a datelor. In cazul violarii acestor constrangeri de integritate SGDB-ul va rejecta orice tranzactie pe tabela respectiva.

✓ **Constrangerea de valori nenule(NOT NULL)**

Este o constrangere care se aplica numai la nivel de atribut(coloana) si verifica daca inregistrarile au valori nule pe coloanele respective, fortand un cod de eroare care anuleaza tranzactia. Orice incercare de a adauga o linie care contine valori nule pe acea coloana sau de a modifica o valoare nenula intr-una nula va genera o eroare sistem.



Elementele modelului relational

- Cand se creeaza constrangeri pe o cheie primara se creeaza automat si o constrangere NOT NULL pe attributele respective (o cheie primara nu trebuie sa contina valori nule pe coloanele care o definesc).

Exemplu:

Daca definim atributul CNP de tip NOT NULL atunci cand se insereaza o linie sau se modifica valoarea CNP-ul in valoare nula (se sterge valoarea) SGDB-ul va genera o eroare si operatia se anuleaza.

✓ **Constrangerea de unicitate (UNIQUE)**

Se foloseste cand vrem ca un atribut(coloana) sau perechi de attribute sa nu contina valori duplicate.



Elementele modelului relational

- Verificarea se face numai pentru inregistrari cu valori nenule deoarece constrangerea de unicitate permite inserarea de valori nule in coloanele respective.
- Daca coloanele definite unice sunt si not null, atunci putem considera unicitatea ca o cheie unica.
- In mod automat se creeaza si un index pe coloanele definite unice, ceea ce duce la cresterea vitezei de interogare pe tabela.

Exemplu: Daca definim atributul CNP de tip UNIQUE, la orice operatie de inserare sau modificare de date, SGDB-ul face o verificare automata de unicitate si in caz de duplicare se genereaza o eroare care anuleaza operatia.



Elementele modelului relational

✓ Constrangerea de cheie primara (PRIMARY KEY)

Se foloseste pentru definirea unei chei primare la nivel de coloana (cheia contine o singura coloana) sau la nivel de tabela (cheia contine una sau mai multe coloane).

- O tabela poate avea o singura cheie primara.
- Nu se accepta valori nule pentru niciuna dintre coloanele care definesc o cheie primara.
- Cand se creeaza o cheie primara timpul de raspuns in cazul unei interogari se reduce foarte mult.



Elementele modelului relational

Exemplu:

- Daca definim atributul Nr_matricol cheie de tip PRIMARY KEY atunci cand se face o inserare sau o modificare de linie se face o verificare de unicitate dar si de valori nenule(nu se accepta valori nule ale atributelor care compun cheia primara) pe coloana de matricole.
- In caz de violare a constrangerii se anuleaza operatia.
- Daca avem o cheie primara compusa(pe mai multe coloane) verificarea se face pe toate coloanele care compun cheia primara, nu pe coloane individuale.



Elementele modelului relational

✓ Constrangerea de cheie straina (FOREIGN KEY)

Acest tip de constrangere se foloseste pentru relationarea unei tabele cu una sau mai multe tabele, verificand daca valorile atributelor definite ca FOREIGN KEY (cheie straina sau cheie externa) sunt cuprinse in valorile coloanelor altei tabele care trebuie sa fie definite UNIQUE sau PRIMARY KEY.

Mai exact, o relationare cu cheie externa pe o tabela se poate face numai daca coloanele de referinta formeaza o cheie primara sau au unicitate.



Elementele modelului relational

- Cand se face relationarea intre doua tabele trebuie avute in vedere urmatoarele reguli de functionare :
 - Inserarea unei linii intr-o tabela relationata (pe care am definit FOREIGN KEY) se poate face daca exista numai o singura linie in tabela de referinta (in care am definit PRIMARY KEY sau UNIQUE) corespunzator coloanelor de relationare;
 - Coloanele definite FOREIGN KEY accepta valori nule;
 - Stergerea unei linii din tabela de referinta nu se poate face atata timp cat exista linii relationate in tabela relationata, pe linia respectiva;
 - Regulile de mai sus sunt valabile si in cazul relationarii pe coloane.



Elementele modelului relational

Exemplu: Consideram relatiile:

Specializari (Cod_spec, Den_specializare) si

Studenti (Nr_matricol, Nume_student, Cod_grupa, Cod_spec).

- Tabela **Specializari** va contine toate specializarile din facultate, fiecare avand un cod unic.
- Daca definim in tabela **Studenti** atributul Cod_spec ca fiind FOREIGN KEY cu referinta pe atributul Cod_spec din tabela **Specializari**, in momentul in care se face o operatie de inserare sau modificare de linie in tabela **Studenti** SGDB-ul face o verificare daca valoarea Cod_spec se afla printre valorile de pe coloana Cod_spec din tabela **Specializari**, iar daca rezultatul este negativ operatia se anuleaza. Coloana Cod_spec din tabela **Specializari** trebuie sa contina valori unice, adica trebuie obligatoriu sa fie o cheie primara/unica.



Elementele modelului relational

✓ Constrangerea de conditie (CHECK)

Acest tip de constrangere se foloseste pentru a forta valorile unei coloane sa verifice o conditie prestabilita.

Conditia poate sa contina si functii, cu unele exceptii (sysdate, user, unele functii de tip data calendaristica etc.) .

Exemplu: Consideram relatia

Note(Nr_matricol, Nume_student, Cod_grupa, Disciplina, Data_examen, Nota).

- Pentru evitarea erorilor de operare se poate defini coloana Nota de tip CHECK cu conditia (Nota >=1 and Nota<=10) sau conditia max(Nota)<=10.



Capitolul 3

Modelul Entitate-Asociere



Modelul Entitate-Asociere

- **Modelul Entitate-Asociere** (*Entity Relationship*) este un model de date folosit in proiectarea conceptuala de nivel inalt a bazelor de date si poate fi transformat in model relational prin aplicarea unor reguli specifice.
- Modelul EA se constituie intr-o abordare grafica a proiectarii bazelor de date si a fost adoptat de comunitatea stiintifica, precum si de producatorii de software din domeniu, datorita simplitatii si expresivitatii sale.
- Acest model a fost introdus de P. P. Chen in 1976 si a fost utilizat de multe sisteme CASE pentru proiectarea bazelor de date DB2, Oracle, SQL Server, Sybase, s.a.



Modelul Entitate-Asociere

- Pana la aparitia unor astfel de metodologii se pornea de la o colectie de tabele si de la dependentele functionale sau multivalorice asociate si se ajungea, prin aplicarea unor algoritmi sau metode de normalizare, la schema dorita a bazei de date.
- Aceasta abordare de tip *bottom-up* creeaza insa dificultati in cazul bazelor de date complexe in care numarul de tabele si de dependente este mare.
- Probabilitatea ca unele interdependente intre date sa nu fie sesizate in procesul de proiectare este in aceste cazuri ridicata si pot duce la anomalii in exploatarea aplicatiilor.



Modelul Entitate-Asociere

- Impactul conceptelor de abstractizare si generalizare precum si elaborarea unui model de descriere informala a datelor, prin modelul Entitate-Asociere (EA), au dus la gasirea unor cai moderne de proiectare a bazelor de date. Modelul EA este unul dintre cele mai utilizate modele datorita intuitivitatii si simplitatii elementelor sale. Imbunatatirile aduse ulterior prin folosirea abstractizarilor si generalizarilor au dus la crearea de variante ale modelului, doua dintre acestea fiind descrise in acest capitol.
- Extensiile modelului EA au aparut si pentru alte necesitati:
 - modelarea cerintelor de secretizare a datelor;
 - elaborarea documentatiei pentru aplicatiile cu baze de date si usurarea comunicarii intre dezvoltatorul de sistem si utilizator;
 - proiectarea bazelor de date complexe pe portiuni si integrarea ulterioara a acestora (asa numita integrare a vederilor).



Modelul Entitate-Asociere

- Prin aplicarea modelului se obtine *diagrama entitate-asociere* care poate fi apoi translatata in modelul de date folosit de sistemul de gestiune a bazei de date.
- Cateva caracteristici ale modelului sunt urmatoarele:
 - Nu este legat direct de niciunul dintre modelele folosite de sistemele de gestiune a bazelor de date (relational sau orientat obiect) dar exista algoritmi bine pusi la punct de transformare din model EA in celelalte modele de date;
 - Este intuitiv, rezultatul modelarii fiind o diagrama care defineste atat datele stocate in baza de date cat si interdependentele dintre acestea;
 - Proiectarea se poate face pe module, diagramele partiale rezultate putand fi apoi integrate pe baza unor algoritmi si metode bine puse la punct;
 - Este usor de inteles si interpretat.



Elementele modelului Entitate-Asociere

- Modelul entitate-asociere permite reprezentarea informatiilor despre structura bazelor de date folosind trei elemente de constructie: **entitati**, **attribute** ale entitatilor si **asocieri** intre entitati.

- **Entitati**

Entitatile modeleaza clase de obiecte concrete sau abstracte despre care se colecteaza informatii, au existenta independenta si pot fi identificate in mod unic. Ele definesc de obicei obiecte sau evenimente cu importanta informationala. Membrii unei clase care formeaza o astfel de entitate poarta numele de **instante** ale acelei entitati.

Exemple de entitati: Studenti, Produse, Facturi, etc. 6



Elementele modelului Entitate-Asociere

Entitatea este un obiect generic care reprezinta multimea tuturor instantelor sale si sunt de doua categorii:

- **Entitati independente** (sau tari) sunt cele care au existenta independenta de alte entitati.
- **Entitati dependente** (sau slabe) sunt formate din instante care isi justifica incadrarea in clasa respectiva doar atata timp cat intr-o alta entitate (tata) exista o anumita instanta de care sunt dependente.

Exemplu:

Daca consideram o baza de date universitara putem considera ca entitatea STUDENTI este entitate independenta iar entitatea NOTE entitate dependenta, deoarece notele unui student sunt dependente de existenta studentului in facultatea respectiva.



Elementele modelului Entitate-Asociere

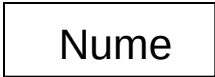
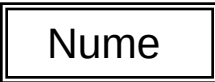
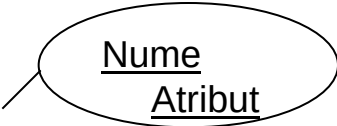
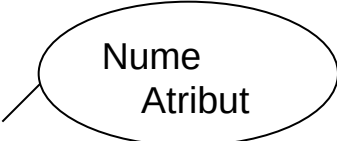
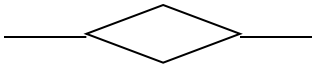
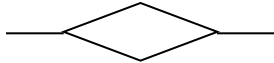
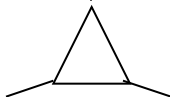
Element	Tip	Reprezentare	Exemplu
Entitate	Tare		
	Slaba		
Atribut	Identificare		
	Descriere		
Asociere	Asociere n=2 entitati	Nume asociere 	Inscris_la 
	Asociere n > 2 entitati	n=3 n=6 	Alocare 

Figura 1. Conventia de reprezentare a elementelor modelului EA



Elementele modelului Entitate-Asociere

■ **Attribute**

Attributele modeleaza proprietati atomice distincte ale entitatilor. In procesul de modelare vor fi luate in considerare doar acele proprietati ale entitatilor care sunt semnificative pentru aplicatia respectiva.

Exemplu:

Entitatea STUDENTI poate avea ca attribute Matricol, Nume, CNP, Grupa, etc. Din acest motiv, la entitatea STUDENTI nu vom lua in considerare caracteristici cum ar fi Talia, nefiind necesara pentru baza de date a universitatii (astfel de attribute ar putea exista insa intr-o baza de date privind personalul militar).



Elementele modelului Entitate-Asociere

Atributele unei entitati sunt de doua feluri:

- **Atribute de identificare** (formand impreuna identificatorul entitatii) - reprezinta acea multime de atribute care permite identificarea unica a instantelor unei entitati;
- **Atribute de descriere** (sau **descriptori**) - sunt folosite pentru specificarea informatiilor suplimentare ale instantelor.
- In cazul entitatii STUDENTI atributul Matricol este atribut de identificare (deoarece nu pot exista doi studenti cu acelasi numar matricol intr-o facultate) pe cand celelalte atribute sunt atribute de descriere.



Elementele modelului Entitate-Asociere

■ Asocieri

Asocierile modeleaza interdependentele dintre clasele de obiecte reprezentate prin entitati.

De exemplu, intre entitatile STUDENTI si FACULTATI se poate figura o asociere Inscris_la care descrie apartenenta studentilor pe facultati.

- In crearea diagramei nu vor fi luate in considerare decat interdependentele care sunt necesare aplicatiei respective, in lumea reala putand exista intre entitatile diagramei si alte asocieri care nu sunt semnificative in contextul dat.



Elementele modelului Entitate-Asociere

- In constructia unei diagrame EA trebuie avute in vedere urmatoarele:
 - Entitatile se reprezinta prin dreptunghiuri in care este inscris numele entitatii. In cazul entitatilor dependente (slabe), conturul va fi cu linie dubla.
 - Atributele se reprezinta prin cercuri (sau ovale) in interiorul carora apare numele atributului. Ele sunt conectate cu un segment de dreapta la entitatea de care apartin. Pentru a distinge atributele de identificare de cele de descriere, numele primelor va fi subliniat.
 - Asocierile se reprezinta prin romburi (daca conecteaza una sau doua entitati) sau poligoane regulate (daca conecteaza mai mult de doua entitati) conectate prin segmente de dreapta la entitatile asociate, avand inscris in interior (sau alaturi) numele asocierii.



Extensii ale modelului Entitate-Asociere

- Modelul entitate-asociere clasic are unele lipsuri in ceea ce priveste posibilitatea modelarii caracteristicilor asociate unor subclase de obiecte modelate prin simple entitati.
- Pentru aceasta, la modelul original au fost adaugate doua noi concepte: **ierarhia de generalizare** si **ierarhia de incluziune**. Prima defineste partitionarea instantelor unei entitati in n subclase diferite, iar a doua permite clasarea unora dintre instantele unei entitati in k subclase care nu reprezinta o partitie in sens matematic.
- Din punct de vedere formal, cele doua concepte se pot defini astfel:



Extensii ale modelului Entitate-Asociere

■ Ierarhia de incluziune

Definitie: O entitate E_k este o submultime a entitatii E (sau este inclusa in entitatea E) daca fiecare instanta a lui E_k este de asemenea o instanta a lui E .

- Un exemplu de incluziune este definirea in cadrul entitatii ANGAJATI a unor subclase modelate prin entitatile INGINERI, ECONOMISTI si COLABORATORI.
- In cazul ierarhiei de incluziune entitatile *fiu* pot sa nu fie disjuncte, de exemplu, pot exista angajati ingineri dar care sunt incadrati cu contract de colaborare. De asemenea, reuniunea lor poate sa nu acopere in intregime entitatea tata, de exemplu, exista angajati care nu sunt ingineri, economisti sau colaboratori.



Extensii ale modelului Entitate-Asociere

■ Ierarhia de generalizare

Definitie: O entitate E este generalizarea entitatilor $E_1, E_2, \dots, E_n$ daca orice instanta a lui E este de asemenea instanta in una, si numai una, dintre entitatile $E_1, E_2, \dots, E_n$.

- Un exemplu de generalizare este clasarea instantelor entitatii ANGAJATI in subclasele BARBATI si FEMEI.
- O caracteristica a ierarhiei de generalizare este ca din punct de vedere matematic entitatile *fiu* reprezinta o partitie a entitatii *tata*:
 - $E_1 \cup E_2 \cup \dots \cup E_n = E$ si
 - $E_i \cap E_j = \emptyset$ pentru orice $i \neq j$ din intervalul $1..n$
- Ierarhiile de incluziune si generalizare se folosesc doar in cazul in care, pentru subclasele unor clase modelate prin entitati, este nevoie de stocarea unor informatii suplimentare specifice.



Extensii ale modelului Entitate-Asociere

Element	Reprezentare	Exemplu
Ierarhie de incluziune		
Ierarhie de generalizare		

Figura 2. Conventia grafica de reprezentare grafica a ierarhiilor



Extensii ale modelului Entitate-Asociere

Exemplu:

- In cazul unei baze de date de personal este nevoie sa fie specificat numarul de copii ai fiecarui angajat. Acest fapt se poate modela in doua feluri: fie prin adaugarea la entitatea ANGAJATI a unui atribut suplimentar Numar_copii (care va avea valoarea 0 pentru angajatii fara copii), sau prin crearea unei entitati suplimentare COPII aflata intr-o relatie de incluziune cu entitatea ANGAJATI, care va avea ca attribute de identificare pe cele ale angajatului iar ca atribut descriptiv numarul de copii, acesta fiind atributul specific subclasei.
- Vom alege a doua varianta in cazul in care avem nevoie de informatii suplimentare despre copii(ume, CNP, etc.) sau numarul angajatilor care au date specifice acelu atribut este mult mai mic decat numarul total al angajatilor(de exemplu angajati absolventi a doua facultati), fapt care va duce la economie de spatiu pe disc. In acest caz o noua entitate(o noua tabela a bazei de date) va ocupa mai putin spatiu decat un atribut suplimentar (o coloana suplimentara) a tablei ANGAJATI.



Caracteristici ale elementelor modelului

Asa cum entitatile au attribute care specifica diverse proprietati ale clasei de obiecte modelate, si asocierile au caracteristici care aduc informatii suplimentare.

Acestea sunt urmatoarele:

- **Gradul asocierii**

Este o valoare numerica intreaga si este dat de numarul de entitati care participa la acea asociere.

Asocierile de grad 1, 2 si 3 se mai numesc si asocieri **unare**, **binare** si respectiv **ternare**.

Exemplu:

Vom considera o baza de date continand informatii despre studenti, proiectele realizate de acestia, calculatoarele pe care au alocate ore de lucru si facultatile la care sunt inscrisi. De asemenea vom considera ca unii dintre studenti au un **tutor** care ii indruma, acesta fiind un student dintr-un an mai mare.

Caracteristici ale elementelor modelului

- Diagrama EA a bazei de date este prezentata in Figura 3. Pentru simplificarea figurii nu s-au reprezentat decat entitatile si asocierile dintre ele, nu si attributele fiecarei entitati in parte.

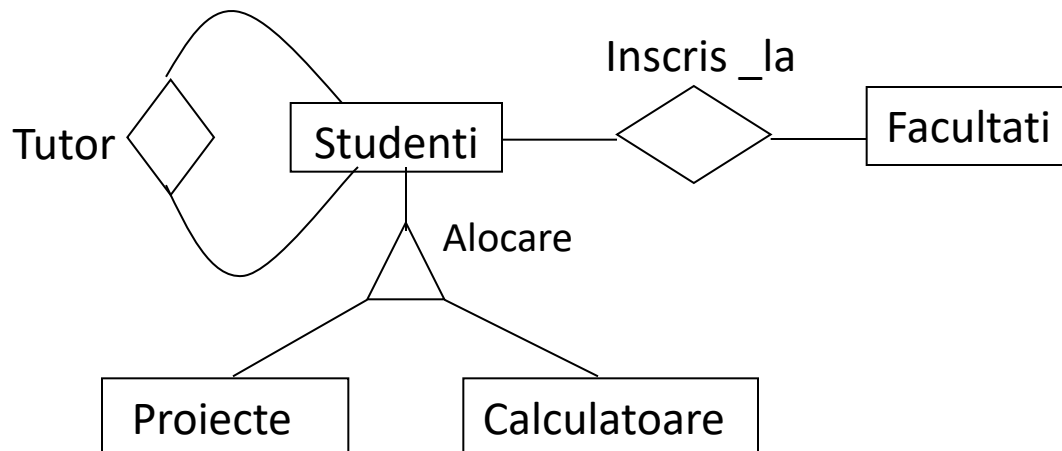


Figura 3. Exemple de asocieri de grad 1, 2 si 3



Caracteristici ale elementelor modelului

- Asocierea **Tutor** este o asociere unara deoarece la ea participa doar entitatea STUDENTI. Aceasta asociere arata cine este tutorul fiecarui student (daca exista).
- Asocierea **Inscris_la** este o asociere binara intre entitatile STUDENTI si FACULTATI si arata la ce facultate/facultati este inscris fiecare student.
- Asocierea **Alocare** este o asociere ternara intre entitatile STUDENTI, PROIECTE si CALCULATOARE. Ea modeleaza pe ce calculatoare are alocate ore de lucru fiecare student pentru fiecare proiect.



Caracteristici ale elementelor modelului

Un exemplu de asociere de grad mai mare ca trei este orarul unui an de studiu al unei facultati. Acesta este o asociere intre urmatoarele entitati:

- GRUPE - Fiecare grupa are un cod unic.
- SALI - Salile sunt etichetate printr-un indicativ alfanumeric.
- ORE - Un interval orar este un triplet (Zi, De la ora, La ora).
- ACTIVITATE - Este o activitate prezenta in orar (curs, laborator, seminar, proiect).
- PROFESOR - Este cadrul didactic titular pentru o activitate.



Caracteristici ale elementelor modelului

- Modelarea acestor clase de obiecte ca entitati presupune faptul ca in baza de date sunt stocate si alte informatii despre ele. Diagrama EA este prezentata in Figura 4. Ea modeleaza programarea activitatilor didactice efectuate de profesori pe intervale orare, sali si grupe.

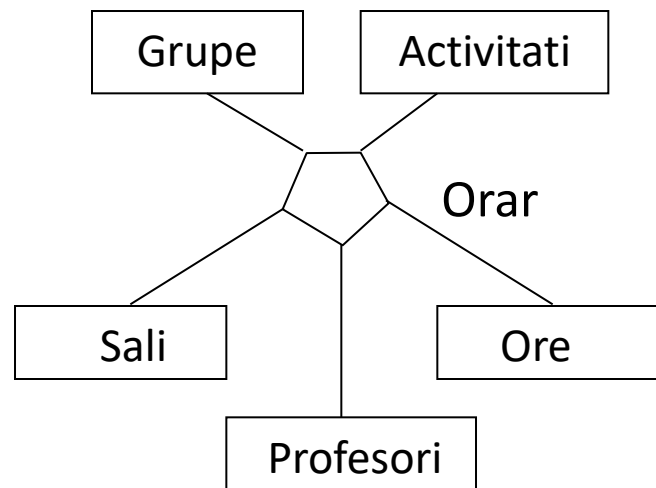


Figura 4. Asociere de grad 5



Caracteristici ale elementelor modelului

■ Conectivitatea asocierii

Conectivitatea este specifica fiecărei ramuri a unei asocieri și poate avea una dintre următoarele două valori: **unu** sau **multi**. Determinarea ei se face astfel:

- Se alege o entitate **E**;
- Se fixează o entitate **F** care participă la asociere;
- Se determină cu câte instanțe din entitatea **F** se poate conecta o instanță din **E** ;
- Dacă poate fi cel mult una, conectivitatea ramurii este **unu**, altfel conectivitatea este **multi**.
- Pentru stabilirea conectivității verificarea se face în ambele sensuri între **E** și **F** .



Caracteristici ale elementelor modelului

Conventii de reprezentare a conectivitatii:

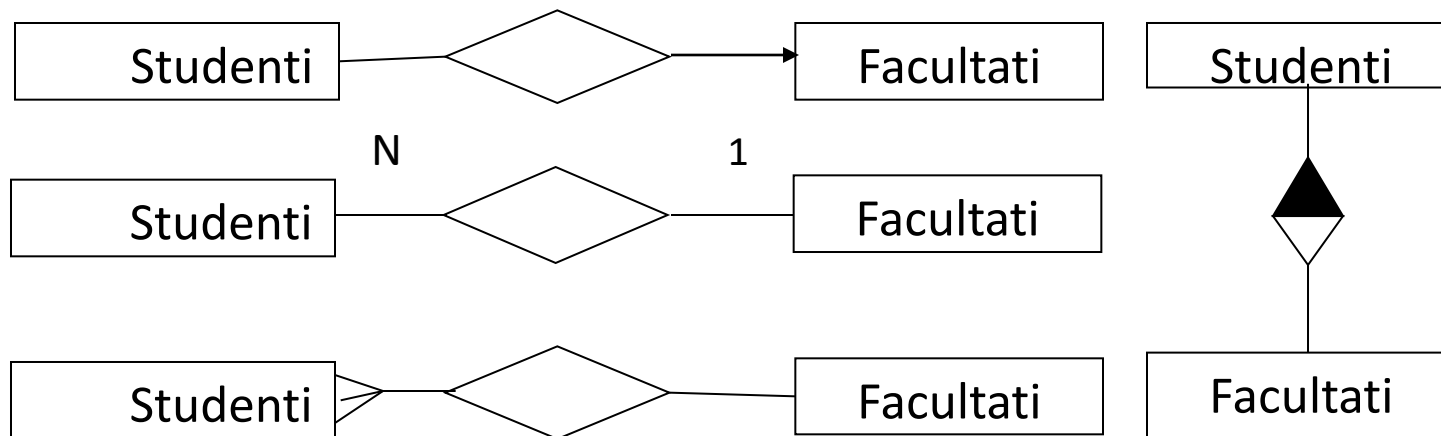


Figura 5. Exemple de reprezentare a conectivitatii



Caracteristici ale elementelor modelului

Exemplu:

Pentru determinarea conectivitatii trebuie stabilite mai intai cerintele de functionare si modelare.

Revenim la exemplul din Figura 3:

- Asocierea **Tutor** este **unu-unu** sau **multi-unu** dupa cum un student poate fi tutor pentru un singur student sau pentru mai multi studenti de an inferior.
- Asocierea **Inscris_la** este **multi-unu** (multi spre STUDENTI) sau **multi-multi** dupa cum un student poate fi inscris la una sau mai multe facultati.
- Asocierea ternara **Alocare** (aplicam definitia):



Caracteristici ale elementelor modelului

- Ramura spre STUDENTII: fiind dat un proiect si un calculator, cati studenti au ore alocate pe acel calculator pentru respectivul proiect? Considerand ca mai multi studenti lucreaza pentru acelasi proiect, pe acelasi calculator, ramura va fi **multi**;
- Ramura spre PROIECTE: fiind dat un student si un calculator, la cate proiecte are acesta alocate ore pe acel calculator? Considerand ca pentru fiecare proiect exista un calculator dedicat, ramura va fi **unu**;
- Ramura spre CALCULATOARE: fiind dat un student si un proiect, pe cate calculatoare are alocate acesta ore pentru realizarea proiectului? Considerand ca la un proiect se lucreaza pe un singur calculator, ramura va fi **unu**.
- Deci asocierea **Alocare** este **multi-unu-unu**.



Caracteristici ale elementelor modelului

- Observam ca raspunsul la fiecare dintre cele trei intrebari se da in functie de realitatea modelata. Aceeasi asociere poate avea conectivitati diferite in cazuri diferite: daca exista chiar si un singur proiect la care un student are ore alocate pe mai mult de un calculator, ramura spre CALCULATOR va fi **multi** iar asocierea va fi **multi-unu-multi**.
- Conventia de reprezentare grafica a conectivitatii folosita in aceasta prezentare este urmatoarea: ramurile **unu** vor fi reprezentate cu sageata, iar ramurile **multi** fara segeata.
- In Figura 6 sunt prezentate cele trei asocieri avand figurata si conectivitatea. S-a presupus ca asocierea **Tutor** este **unu-unu** (un student are un singur tutor) iar **Inscris_la** este **multi-unu** (mai multi studenti sunt inscrisi la o facultate).



Caracteristici ale elementelor modelului

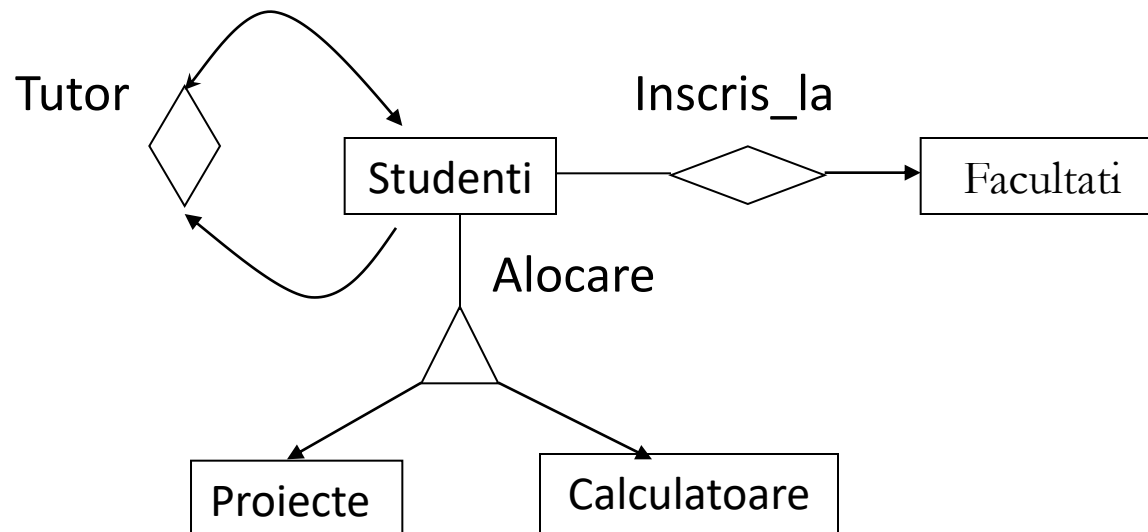


Figura 6. Reprezentarea conectivitatii



Caracteristici ale elementelor modelului

■ **Obligativitatea asocierii**

Ca si conectivitatea, **obligativitatea** se determina pentru fiecare ramura si poate avea doar una dintre urmatoarele valori: **obligatorie** sau **optionala**.

Determinarea ei se face astfel:

- Se alege ramura spre o entitate **E**;
- Se fixeaza o entitate **F** care participa la asociere;
- Este obligatoriu sa existe o instanta a lui **E** asociata cu o instanta din **F**?
- Daca raspunsul este DA ramura este obligatorie, altfel este optionala.



Caracteristici ale elementelor modelului

Exemplu:

In exemplul anterior ramurile asocierilor **Tutor** si **Alocare** sunt optionale iar cele ale asocierii **Inscria_la** sunt obligatorii deoarece:

- Pentru asocierea **Tutor**:
 - exista studenti care nu au un tutor si nici nu sunt tutori pentru alti studenti;
- Pentru asocierea **Alocare**:
 - un student poate sa nu aiba alocate ore pe niciun calculator la un proiect (de exemplu in cazul unui proiect la o materie umanista);
 - un student si un calculator, respectiv un calculator si un proiect, pot sa nu fie asociati prin alocare de ore de lucru (de exemplu pentru calculatoarele din birourile cadrelor didactice).
- Pentru asocierea **Inscris_la**:
 - nu exista studenti care nu sunt inscrisi la nicio facultate si nici facultati fara studenti inscrisi.



Caracteristici ale elementelor modelului

- Conventia de reprezentare grafica a clasei de apartenenta folosita in continuare este urmatoarea: ramurile **obligatorii** vor fi reprezentate prin linie continua iar cele **optionale** prin linie intrerupta. In Figura 7 este reprezentata obligativitatea.

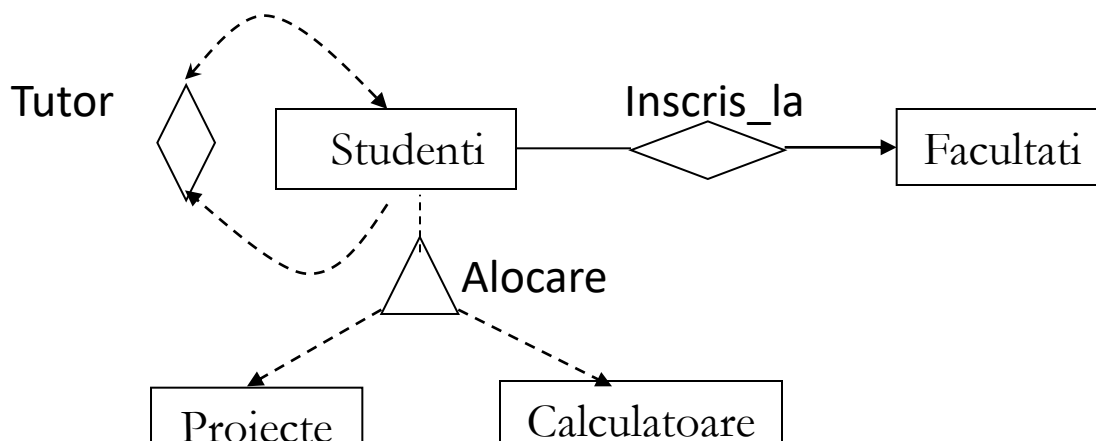


Figura 7. Reprezentarea obligativitatii



Caracteristici ale elementelor modelului

- Daca gradul si conectivitatea unei asocieri sunt folosite in proiectarea conceptuala a schemei bazei de date, obligativitatea se modeleaza pentru definirea unui criteriu de integritate specificand posibilitatea de aparitie a valorilor nule.
- La transformarea diagramei EA in model relational atributele tabelelor care modeleaza informatia reprezentata de asocieri pot avea sau nu valori nule, dupa cum ramurile acestora sunt optionale sau obligatorii.



Caracteristici ale elementelor modelului

■ Atributele asocierilor

In unele cazuri o anumita informatie descriptiva este asociata cu un ansamblu de clase diferite, nu cu o singura clasa de obiecte , modelate fiecare prin entitati. In acest caz aceasta va fi modelata ca un **atribut al asocierii** dintre entitatile respective.

Exemplu:

Sa luam cazul unei asocieri **A_absolvit** de tip **multi-multi** intre entitatile STUDENTI si FACULTATI care contine informatii privind facultatile absolvite anterior de unii studenti, asa cum este prezentata in Figura 8.



Caracteristici ale elementelor modelului

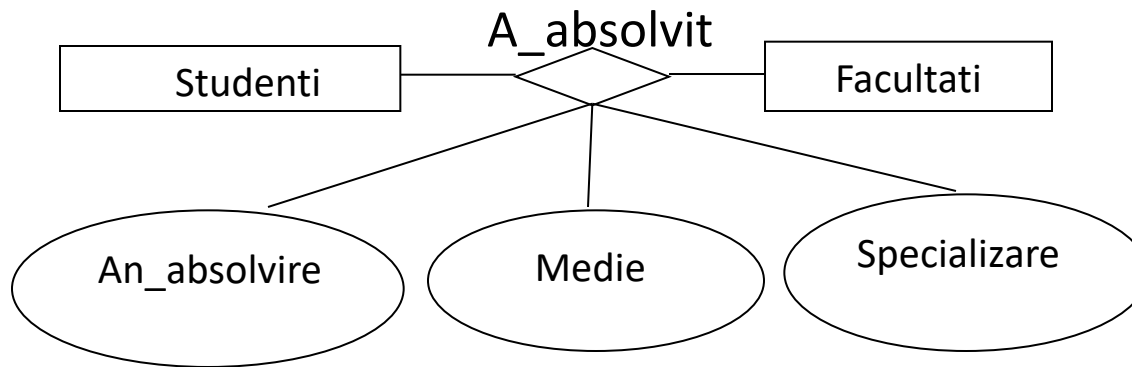


Figura 8. Reprezentarea atributelor asocierilor

- In acest caz informatii ca anul absolvirii, media, specializarea nu pot fi conectate nici la STUDENTI (pentru ca un student poate fi absolventul mai multor facultati in ani diferiti, cu medii diferite, etc.) si din motive similare nici la FACULTATI. Ele descriu asocierea unui student cu o facultate si de aceea vor fi atasate asocierii **A_absolvit**. Toate attributele unei asocieri sunt attribute descriptive, neexistand in acest caz un identificator al asocierii printre ele.



Caracteristici ale elementelor modelului

■ Rolul

În cazul în care de la o asociere pornesc mai multe ramuri către aceeași entitate, fiecareia dintre acestea i se poate asocia un **rol**. Acesta arată semnificațiile diferite pe care le are aceeași entitate în cadrul asocierii respective, așa cum se vede în Figura 9.

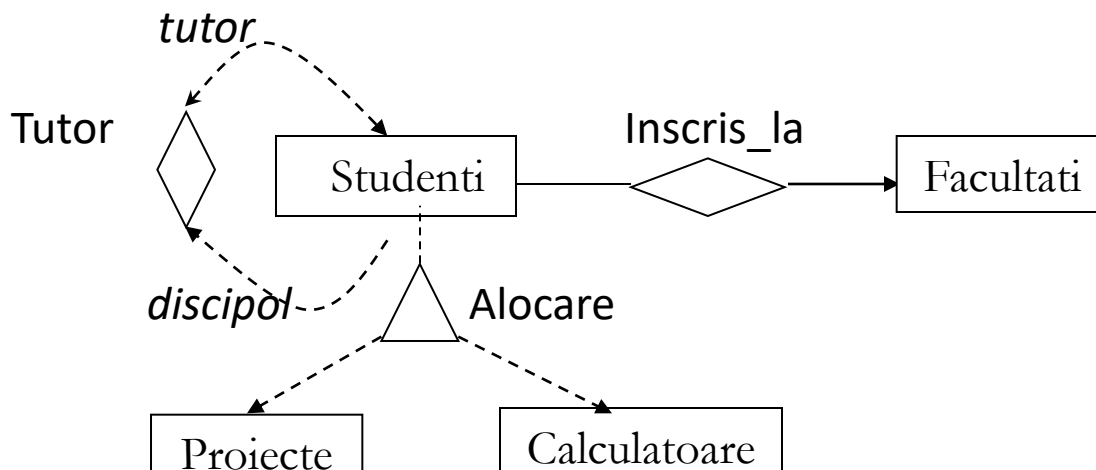


Figura 9. Reprezentarea obligativității și a rolurilor

- În cazul asocierii **Tutor** cele două ramuri pot fi etichetate, de exemplu, cu **tutor** și **discipol**, arătând că instante diferite ale aceleiași entități au roluri diferite.



Criterii de modelare

■ Clasificarea in entitati si attribute

Desi definitia notiunilor de entitate, atribut, asociere este destul de simpla, in practica modelarii apar dificultati in clasificarea diverselor informatii intr-una din aceste categorii. De exemplu, in cazul sediilor unei banci localizate in diverse orase: obiectul ORASE este entitate distincta sau atribut descriptiv al entitatii SEDII ?

- Pentru a putea clasifica corect informatiile, exista cateva reguli care trebuie respectate si pe care le prezentam in continuare.
- Prima regula da un criteriu general de impartire in entitati si attribute, urmatoarele doua semnaleaza exceptii iar ultimele doua reguli au un caracter mai degraba orientativ si mai putin normativ.



Criterii de modelare

- ***Regula 1.*** Entitatile au informatii descriptive, pe cand atributele nu poseda astfel de informatii.

Daca exista informatii descriptive despre o anumita clasa de obiecte, aceasta va fi modelata ca o entitate.

In cazul in care pentru o clasa de obiecte nu este nevoie decat de un identificator (codul, denumirea, etc), ea va fi modelata ca un atribut.

De exemplu, daca despre un oras este necesara stocarea in baza de date unor informatii ca judet, codul strazilor, adrese, etc. atunci ORASE va fi o entitate. Daca singura informatie necesara este numele atunci Nume_oras va fi un atribut al altei entitati.



Criterii de modelare

- **Regula 2. Atributele multivalorice** vor fi reclasificate ca entitati.

Daca la o valoare a unui **identificator** corespund mai multe valori ale unui **descriptor**, atunci descriptorul va fi clasat ca entitate. De exemplu, in cazul unei baze de date privind localizarea in teritoriu a unor banci, daca se stocheaza date doar despre banci care au un singur sediu, **Localitate** este atribut al entitatii BANCI, asa cum este prezentat in Figura 10.

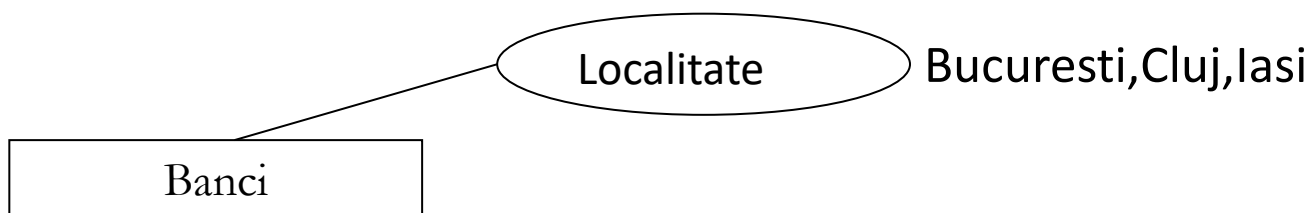


Figura 10. Reprezentarea atributelor multivalorice



Criterii de modelare

- Daca insa se stocheaza date despre banci care au sucursale si filiale in diverse localitati, deci pentru o singura banca (o valoare a identificatorului entitatii BANCI) avem mai multe localitati in care aceasta are sedii (mai multe valori ale descriptorului Localitate), atunci **LOCALITATI** va fi entitate distincta desi are numai un singur atribut. Pentru a modela localizarea sediilor in diverse localitati intre cele doua entitati va exista o asociere binara unu-multi (unu spre BANCI) numita de exemplu **Are_sediu_in**, asa cum se vede in Figura 11.

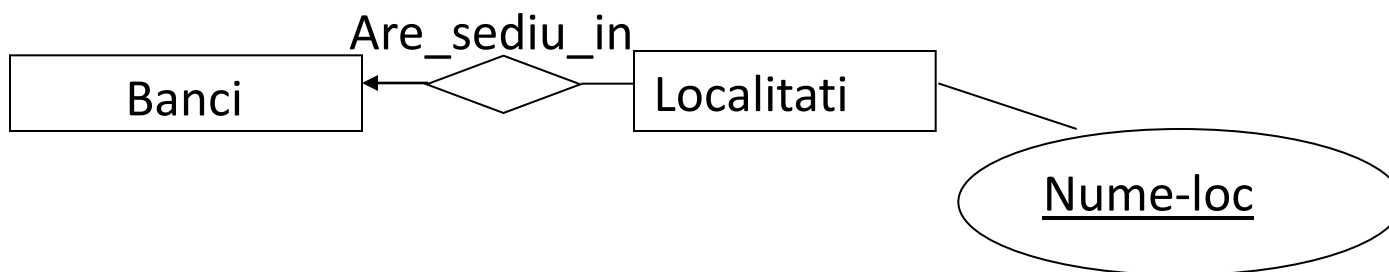


Figura 11. Reprezentarea atributelor multivalorice reclasificate ca entitati



Criterii de modelare

- **Regula 3.** Atributele unei entitati care au o asociere multi-unu cu o alta entitate vor fi reclassificate ca entitati.

Asa cum am vazut, asocierile pot lega doar entitati. Daca un descriptor al unei entitati este intr-o relatie multi-unu cu o alta entitate, acel descriptor va fi trecut in categoria entitatilor.

De exemplu, daca avem entitatile BANCI avand ca atribut descriptiv monovaloric Localitate si Judet, daca se doreste modelarea apartenentei la judete a localitatilor va exista o asociere multi-unu intre atributul Localitate si entitatea JUDETE, reprezentata in Figura 12.

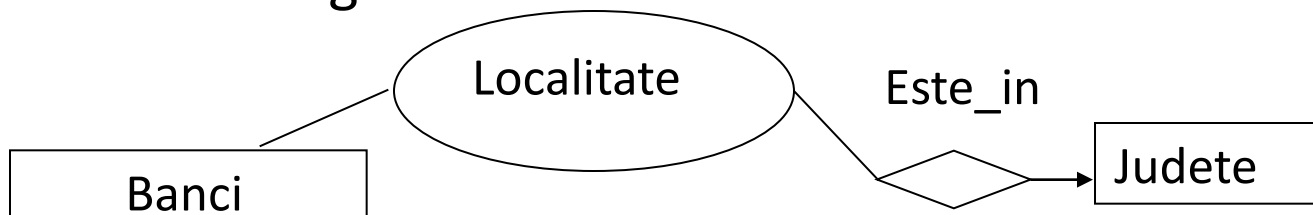


Figura 12. Reprezentarea atributelor multivalorice multi-unu



Criterii de modelare

- In acest caz atributul Localitate va fi reclasificat ca entitatea LOCALITATI desi nu sunt necesare alte informatii in afara numelui localitatii, ca in Figura 13.

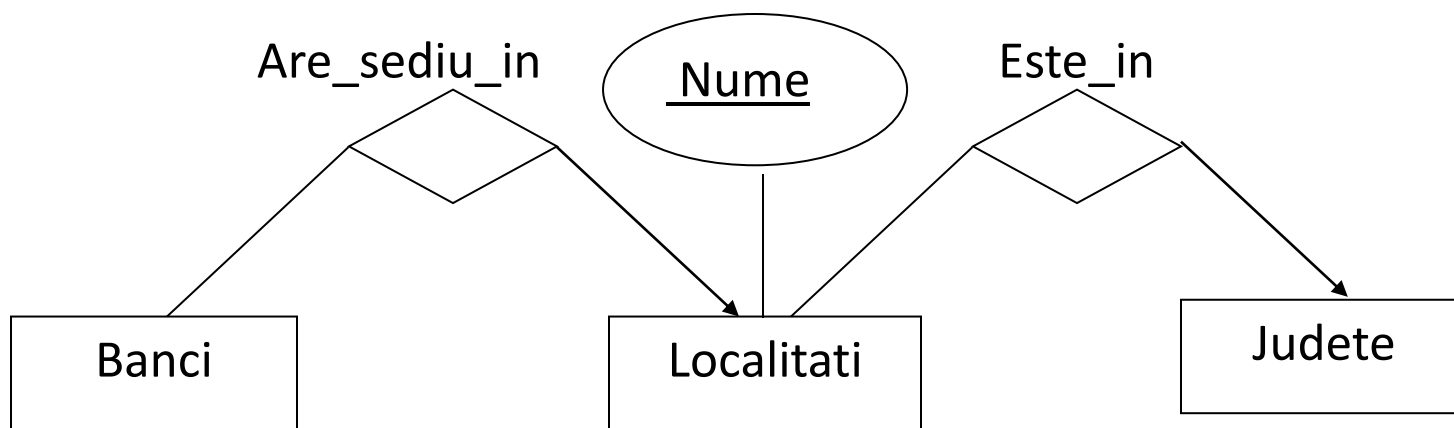


Figura 13. Reprezentarea atributelor multivalorice multi-unu reclasificate



Criterii de modelare

- ***Regula 4.*** Atributele vor fi atasate la entitatile pe care le descriu in mod nemijlocit. De exemplu, Universitate va fi atasat ca atribut al entitatii FACULTATI si nu al entitatilor STUDENTI sau PROFESORI.
- ***Regula 5.*** Folosirea identificatorilor compusi va fi evitata, pe cat posibil. Identificatorul unei entitati este acea submultime de attribute ale acesteia care identifica in mod unic fiecare instanta a sa. In modelul relational pentru attributele de acest fel se construiesc, de regula, structuri de cautare rapida (indecsi) care functioneaza cu atat mai lent cu cat complexitatea index-ului creste. Aplicarea acestei reguli se poate face in diverse moduri:



Criterii de modelare

- Daca identificatorul unei entitati este compus din mai multe attribute care sunt toate identificatori in alte entitati, acea entitate se elimina. Informatia continuta de aceasta va fi modelata sub forma unei asocieri intre acele entitati.
- Daca identificatorul unei entitati este compus din mai multe attribute care nu sunt toate identificatori in alte entitati, exista doua solutii:
 - Entitatea respectiva se elimina si este inlocuita prin alte entitati si asocieri astfel incat per ansamblu informatia modelata in varianta initiala sa fie pastrata.
 - Entitatea respectiva ramane in forma initiala, cu dezavantaje insa in privinta vitezei tranzactiilor pe baza de date dar cu evitarea erorilor de operare.



Criterii de modelare

- Se observa ca procedura clasificarii obiectelor in entitati si attribute este iterativa:
 - ✓ Se face o prima impartire conform primei reguli;
 - ✓ O parte dintre attributele astfel obtinute se reclasifica in entitati conform regulilor 2 si 3;
 - ✓ Se face o rafinare finala conform regulilor 4 si 5.



Criterii de modelare

▪ **Identificarea ierarhiilor de generalizare si incluziune**

In cazul in care despre anumite subclase ale unei clase de obiecte exista informatii specifice, clasa si subclasele (care la pasul anterior au fost catalogate ca entitati) sunt interconectate intr-o ierarhie de incluziune sau generalizare, dupa cum este cazul.

- La acest pas se face si o reatasare a atributelor pentru evitarea redundantei, astfel:

- La entitatea *tata* vor fi atasate attributele care formeaza identificatorul si descriptorii care modeleaza informatii specifice intregii clase.
- La entitatile *fiu* vor fi atasate attributele de identificare (aceleasi ca ale tatalui) plus attributele care modeleaza informatii specifice doar acelei subclase de obiecte.



Criterii de modelare

Exemplu:

Sa consideram o ierarhie care imparte studentii unei facultati in doua subclase:

- caministi – daca locuiesc in campusul universitar;
- necaministi – daca nu locuiesc in campus.

Diagrama entitate-asociere si ierarhiile de asociere sunt prezentate in Figura 14.



Criterii de modelare

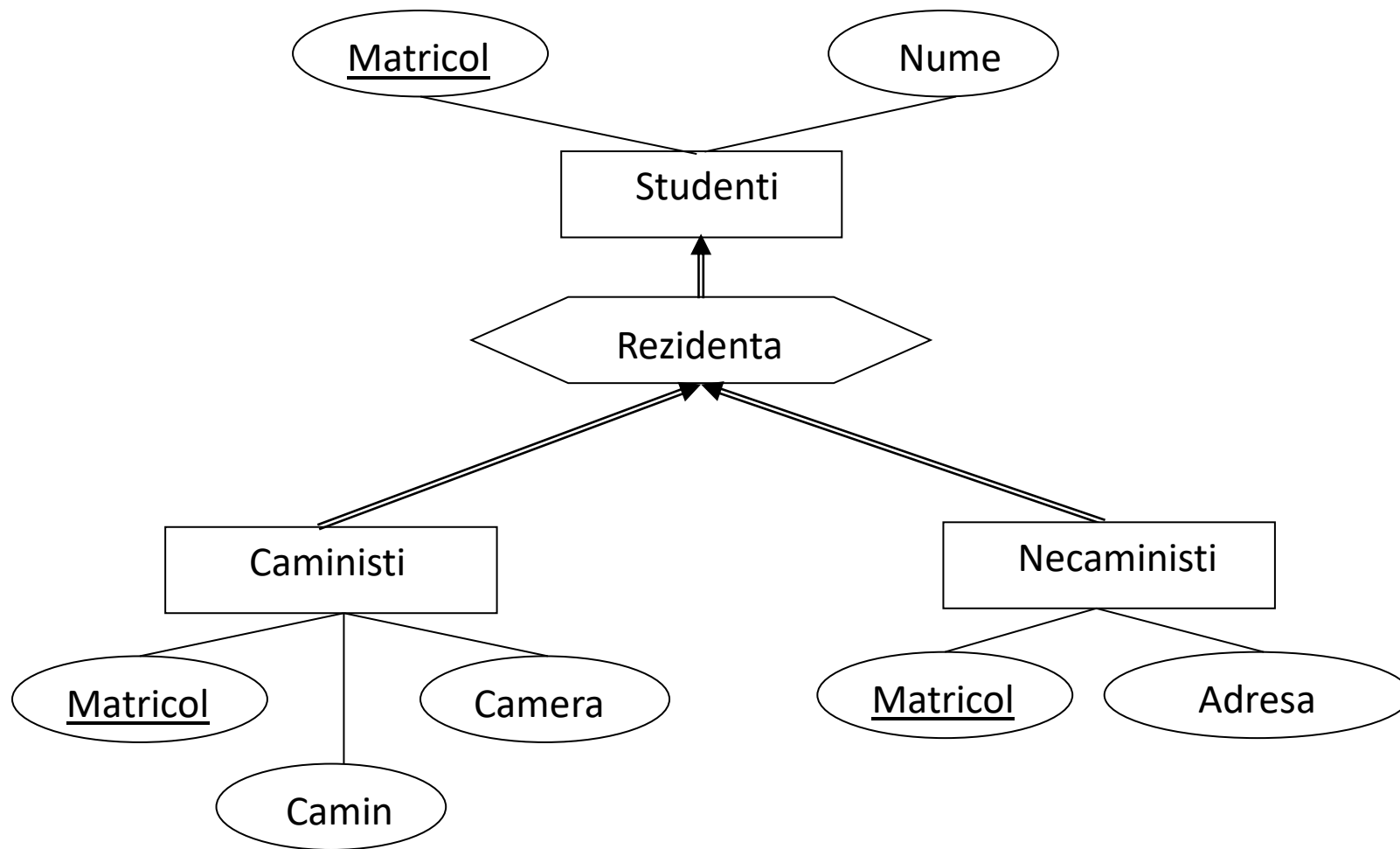


Figura 14. Atributele entitatilor unei ierarhii



Criterii de modelare

- In acest caz atributul Nume trebuie eliminat de la entitatile fiu CAMINISTI si NECAMINISTI deoarece el este prezent deja la entitatea tata STUDENTI.
- Rezulta urmatoarele reguli:
 - Identificatorul tata din cadrul unei ierarhii se regaseste in identificatorul tuturor fiilor ierarhiei;
 - Descriptorii care apar si la tata si la fii, se elimina de la fii;
 - Descriptorii care apar la toti fiii unei ierarhii de generalizare, dar nu apar la tata, se muta la tata.



Criterii de modelare

■ Identificarea asocierilor

In aceasta etapa se trateaza informatiile care nu au fost clasificate ca entitati sau attribute ci reprezinta interdependente intre clase de obiecte. Ele sunt modelate ca **asocieri intre entitati**. Pentru fiecare asociere se specifica gradul, conectivitatea, obligativitatea si daca este cazul si attributele asocierii precum si rolurile ramurilor sale.

- Ca si in cazul clasificarii in entitati si attribute, exista cateva reguli de urmat in operatia de definire a asocierilor:



Criterii de modelare

■ Eliminarea asocierilor redundante

In cazul in care o asociere poate fi dedusa din alte asocieri deja catalogate, aceasta se elimina. De retinut ca intre doua entitati pot sa existe oricate asocieri si ele nu sunt considerate redundante atata timp cat au semnificatii diferite.

- Un caz des intalnit de redundanta este cel al compunerii (tranzitivitatii) asocierilor. Prezentam in Figura 15 un exemplu:

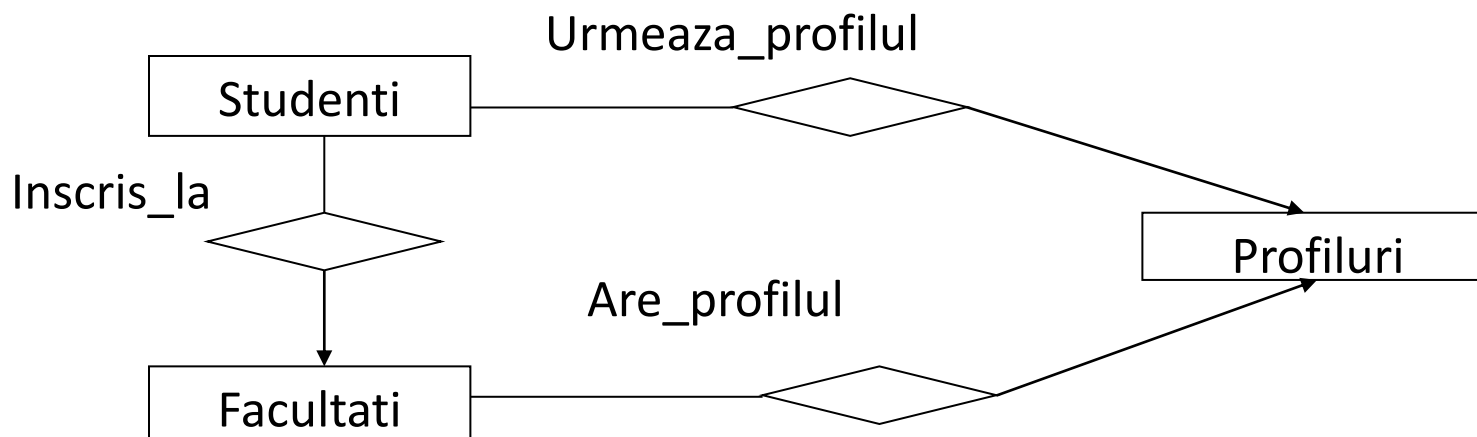


Figura 15. Asocieri redundante



Criterii de modelare

- In acest exemplu, asocierea **Inscris_la** modeleaza apartenenta fiecarui student la o facultate a unei universitati. Fiecare facultate are un profil unic descris de asocierea **Are_profilul**(de ex. Electric, Mecanic, Chimic, etc.). Ambele asocieri sunt multi-unu in sensul STUDENTI→FACULTATI→PROFILURI.
- Deoarece asocierile multi-unu (ca si cele unu-unu) sunt din punct de vedere matematic functii, din compunerea asocierilor putem afla profilul la care este inscris fiecare student.
- Rezulta ca asocierea **Urmeaza_profilul** care are chiar aceasta semnificatie este redundanta si trebuie eliminata.



Criterii de modelare

■ Asocieri de grad mai mare decat doi

Asocierile ternare (sau de grad mai mare ca trei) se folosesc doar atunci cand sunt strict necesare.

- Este de multe ori posibil ca o aceeași informație să fie modelată ca o asociere ternară sau ca un ansamblu de asocieri binare și unare. În cazul acesta, este de preferat ca să se opteze pentru a doua variantă, așa cum se vede în Figura 16 și Figura 17.
- Doar când asocierile binare nu pot modela întreaga semnificație dorită se va opta pentru asocieri de grad mai mare decât doi. Această cerință derivă din faptul că la transformarea în modelul relational asocierile de grad superior devin scheme de relații de sine statatoare, mărin numărul de tabele din bază de date pe când cele de grad unu și doi (cu excepția celor multi-multi) nu au acest efect.



Criterii de modelare

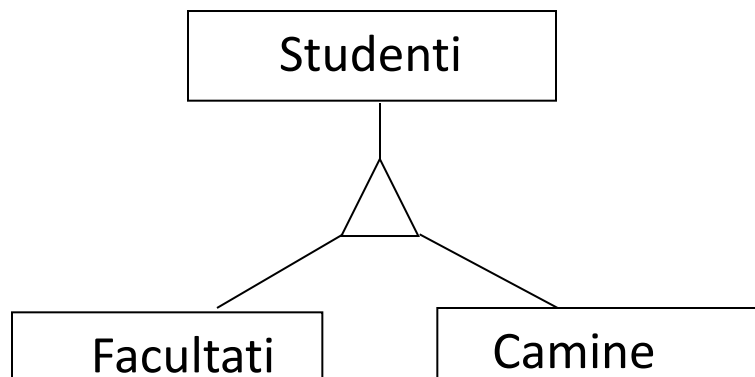


Figura 16. Exemplu de asociere ternara

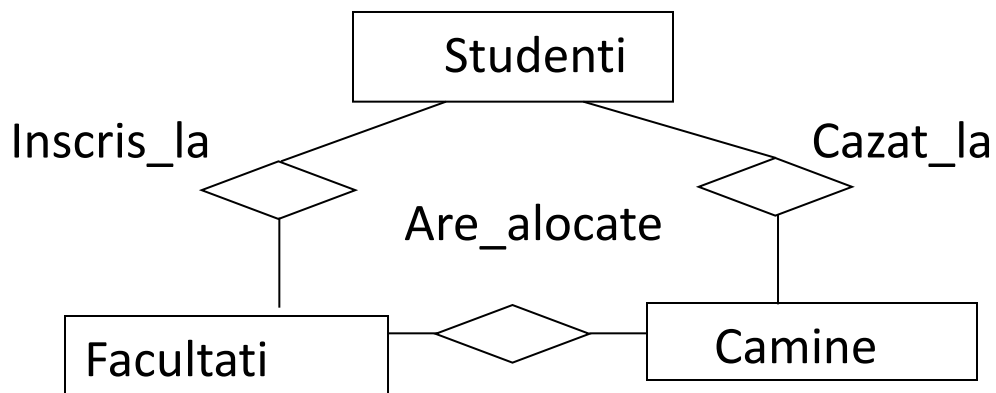


Figura 17. Exemplu de asociere ternara remodelata



Criterii de modelare

■ Integrarea vederilor

În cazul proiectării bazelor de date complexe, activitatea se desfășoară uneori de către mai multe colective simultan, fiecare modelând o porțiune distinctă a bazei de date.

Deoarece în final trebuie să se obțină o singură diagramă a bazei de date, după terminarea modelării pe porțiuni diagramele rezultate sunt integrate, eliminându-se redundanțele și inconsistențele.



Transformarea diagramelor EA in MR

- In **procesul de transformare** vom pleca de la o diagrama EA si vom obtine trei tipuri de scheme de relatie in MR:
 - **Relatii provenite din entitati.** Ele contin aceleasi informatii ca si entitatile din care au rezultat.
 - **Relatii provenite din entitati si care contin chei straine.** Ele contin pe langa informatiile provenite din entitatile din care au rezultat si attribute care in alte entitati sunt identificatori. Este cazul acelor entitati care au asocieri multi-unu si partial din cele care au asocieri unu-unu cu alte entitati.
 - **Relatii provenite din asocieri.** Este cazul celor care apar din transformarea asocierilor binare multi-multi si a asocierilor de grad mai mare ca doi. Ele contin ca attribute reuniunea identificatorilor entitatilor asociate si attributele proprii ale asocierilor.

Procesul de transformare are un algoritm foarte precis si este din acest motiv etapa care se preteaza cel mai bine pentru crearea de instrumente software CASE care sa-l asiste.



Transformarea diagramelor EA in MR

■ Transformarea entitatilor

Fiecare entitate a diagramei se transforma intr-o schema de relatie avand:

- **Numele relatiei** = Numele entitatii
- **Atributele relatiei** = Atributele entitatii
- **Cheia relatiei** = Identificatorul entitatii

Exemplu:

Fie entitatea ANGAJATI din Figura 18:

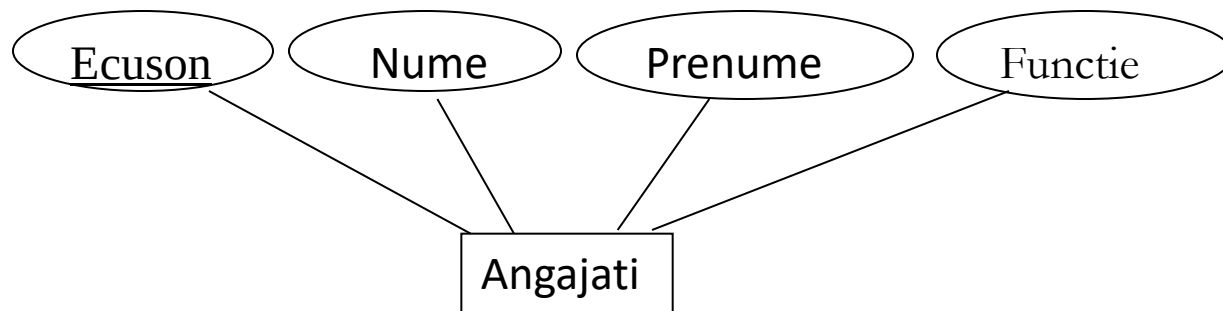


Figura 18. Exemplu de schema de relatie

Schema rezultata este ANGAJATI (Ecuson, Nume, Prenume, Functie).



Transformarea diagramelor EA in MR

- **Transformarea asocierilor unare si binare unu-unu si multi-unu**
Fiecare asociere din aceasta categorie va avea ca rezultat imbogatirea multimii de attribute descriptive ale uneia dintre cele doua scheme rezultate din entitatile asociate, cu cheia celeilalte scheme. Aceste attribute care se adauga sunt denumite in prezentarea de fata cheie straina (sau cheie externa) deoarece ele sunt cheie dar in alta schema de relatie.
- In cazul asocierilor **multi-unu**, se adauga identificatorul entitatii unu in schema rezultata din entitatea multi.
- In cazul asocierilor **unu-unu**, se adauga identificatorul unei entitati in schema rezultata din transformarea celeilalte. Alegerea schemei in care se face adaugarea se poate face dupa doua criterii:
 - fie in acea schema care defineste relatia cu cele mai putine tupluri dintre cele doua;
 - fie pastrandu-se, daca exista, filiatia naturala intre cele doua entitati: identificatorul tatalui se adauga la fiu.



Transformarea diagramelor EA in MR

- In cazul acestui tip de asociere, la schema de relatie care primeste cheia straina se ataseaza una sau doua dependente functionale primare, conform Tabelului 1.

Un exemplu este prezentat in Figura 19.

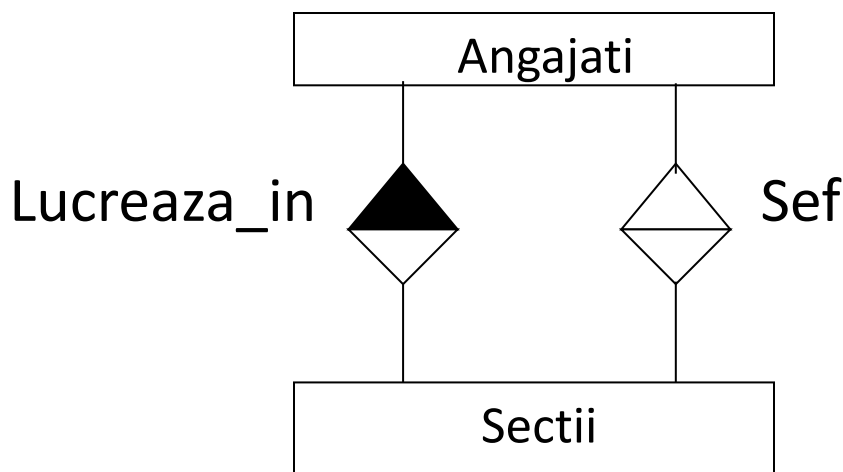
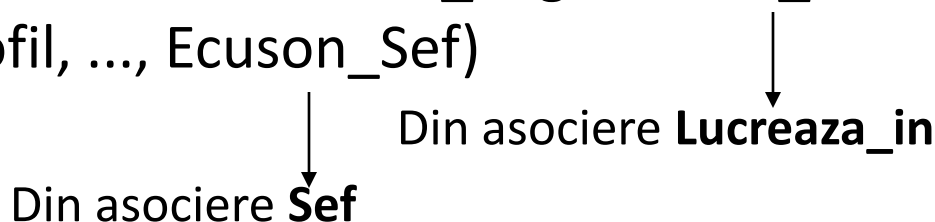


Figura 19. Exemplu de asociere cu doua dependente



Transformarea diagramelor EA in MR

- Intre entitatile ANGAJATI si SECTII exista doua asocieri:
 - **Sef** (unu-unu), care modeleaza faptul ca o sectie este condusa de un angajat (seful de sectie);
 - **Lucreaza_in** (multi-unu) care modeleaza faptul ca o sectie are mai multi angajati.
- Rezultatul transformarii este cel de mai jos. Atributele aflate dupa punctele se suspensie sunt cele adaugate (chei straine):
ANGAJATI (Ecuson, Nume, Functie, Data_ang, ..., Cod_Sectie)
SECTII (Cod_Sectie, Profil, ..., Ecuson_Sef)



In atributul Cod_Sectie din relatia ANGAJATI se va inregistra pentru fiecare angajat codul sectiei in care acesta lucreaza iar in atributul Ecuson_Sef din relatia SECTII se va inregistra, pentru fiecare sectie, ecusonul sefului de sectie. Pentru asocierea Sef s-a aplicat primul criteriu (relatia SECTII va avea mult mai putine inregistrari decat ANGAJATI), dar si al doilea criteriu este indeplinit.



Transformarea diagramelor EA in MR

▪ Transformarea asocierilor unare si binare multi-multi si a celor de grad mai mare decat doi

Fiecare asociere binara multi-multi si fiecare asociere cu grad mai mare decat doi se transforma intr-o schema de relatie astfel:

- **Nume relatie** = Nume asociere;
- **Atributele relatie** = Reuniunea identificatorilor entitatilor asociate la care se adauga atributele proprii ale asocierii ;
- **Cheia relatiei** = Conform Tabelului 2.



Transformarea diagramelor EA in MR

Grad	Conectivitate	Dependente Functionale Primare (DFP)
Unare	• unu (E) - unu (E) • cheie(E) se introduce ca si cheie straina in tabela E, cu redenumire	$Cheie(E) \Rightarrow Cheie\ straina(E)$ $Cheie\ straina(E) \Rightarrow Cheie(E)$
	unu (E) - multi (E)	$Cheie(E) \Rightarrow Cheie\ straina(E)$
	multi (E) - multi (E)	$Cheie(E) \Rightarrow AP$
Binare	• unu (E1) - unu (E2) • cheie(E2) se introduce in E1	$Cheie(E1) \Rightarrow Cheie\ straina(E2)$ $Cheie(E2) \Rightarrow Cheie\ straina(E1)$
	unu (E1) - multi (E2)	$Cheie(E1) \Rightarrow Cheie\ straina(E2)$
	multi (E1) - multi (E2)	$Cheie(E1) + Cheie(E2) \Rightarrow AP$
Ternare	unu (E1) - unu (E2) - unu (E3)	$Cheie(E1) + Cheie(E2) \Rightarrow Cheie(E3)$ sau $Cheie(E1) + Cheie(E3) \Rightarrow Cheie(E2)$ sau $Cheie(E2) + Cheie(E3) \Rightarrow Cheie(E1)$
	unu (E1) - unu (E2) - multi (E3)	$Cheie(E1) + Cheie(E3) \Rightarrow Cheie(E2)$ sau $Cheie(E2) + Cheie(E3) \Rightarrow Cheie(E1)$
	unu (E1) - multi (E2) - multi (E3)	$Cheie(E2) + Cheie(E3) \Rightarrow Cheie(E1)$
	multi (E1) - multi (E2) - multi (E3)	$Cheie(E1) + Cheie(E2) + Cheie(E3) \Rightarrow AP$

Tabel 1. Dependente functionale primare rezultate din transformare

Legenda: $X \Rightarrow Y$: Multimea de coloane X determina functional multimea de coloane Y

AP : Atribute proprii transformarii sau multimea vida (daca nu exista)

$X+Y$: Coloanele X impreuna cu coloanele Y



Transformarea diagramelor EA in MR

Grad	Conectivitate	Cheia relatiei obtinuta din asociere
Unare	multi (E) - multi (E)	Cheie(E) + Cheie(E)
Binare	multi (E1) - multi (E2)	Cheie(E1)+Cheie(E2)
Ternare	unu (E1) - unu (E2) - unu (E3)	Cheie(E1)+Cheie(E2) sau Cheie(E1)+Cheie(E3) sau Cheie(E2)+Cheie(E3)
	unu (E1) - unu (E2) - multi (E3)	Cheie(E1)+Cheie(E3) sau Cheie(E2)+Cheie(E3)
	unu (E1)- multi (E2) - multi (E3)	Cheie(E2)+Cheie(E3)
	multi (E1)-multi(E2) - multi (E3)	Cheie(E1)+Cheie(E2)+Cheie(E3)

Tabel 2. Cheile schemelor de relatie rezultate din asocieri

Legenda:

X + Y: Multimea de attribute X impreuna cu multimea de attribute Y



Transformarea diagramelor EA in MR

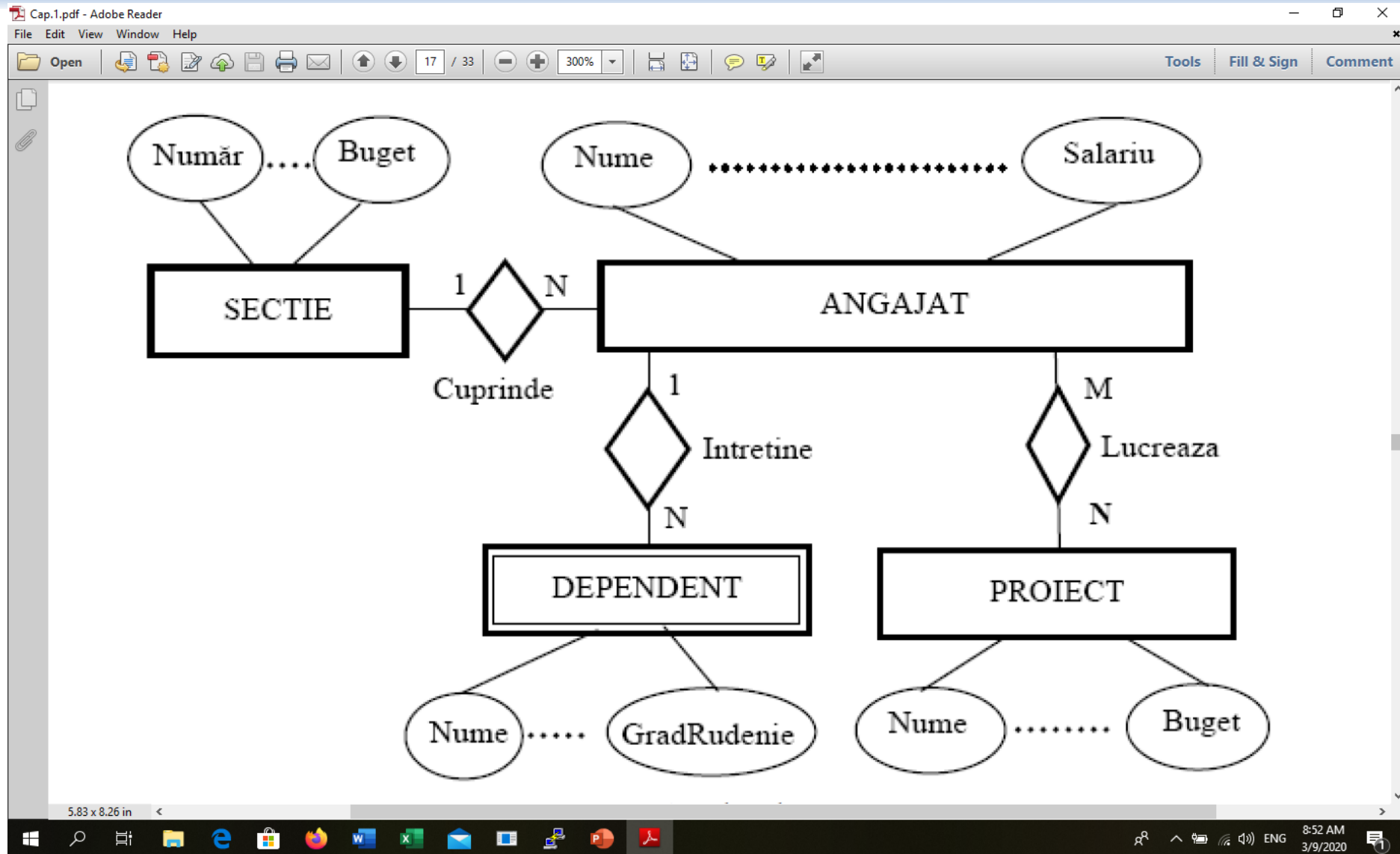


Figura 20. Exemplu de diagrama de asociere in modelul EA



Transformarea diagramelor EA in MR



Figura 20. Exemplu de diagrama de relatii in modelul MR



Capitolul 4

Proiectarea bazelor de date



Proiectarea bazelor de date

Proiectarea corecta a bazei de date este primul pas spre o implementare de succes a unei aplicatii software si de aceea trebuie respectate metodologiile care impun normalizarea schemelor de relatie, pastrarea integritatii datelor si eliminarea anomaliilor.

Proiectarea trebuie vazuta sub doua aspecte:

- proiectare conceptuala;
 - proiectare logica.
- Deoarece o schema de relatie poate avea multiple forme, se pune problema alegerii corecte a structurii relatiei printr-o proiectare conceptuala adecvata, dar si respectarea cerintelor de functionare rezultate din analiza si regasite in proiectarea logica.
 - Schema conceptuala descrie in mod abstract forma in care dorim sa stocam datele reale iar procesul de realizare se numeste **modelare conceptuala**.



Dependente functionale

In paragrafele urmatoare vom folosi o conventie de notare intalnita in literatura de specialitate:

- $R, S, T, \dots$: scheme de relatii;
- $r, s, \dots$: instante ale relatiilor R , respectiv S ;
- $A, B, C, D, \dots$ (litere mari de la inceputul alfabetului): attribute ale unei relatii;
- $X, Y, Z, W, U, \dots$ (litere mari de la sfarsitul alfabetului): multimi de attribute dintr-o schema de relatie;
- $X \subseteq R$: Multimea de attribute X este inclusa in multimea atributelor relatiei R ;
- $Y \subseteq X$: Multimea de attribute Y este inclusa in multimea de attribute X ;
- $A \in X$: Atributul A apartine multimii de attribute X ;
- $t, t_1, t_2, \dots$ tupluri ale unei relatii;
- $t[X]$: valorile atributelor din multimea X aflate in tuplul t ;
- $F, G, \dots$: multimi de dependente functionale atasate unei scheme de relatie.



Dependente functionale

- Termenul **relatie** semnifica atat **schema relatiei** (descrierea structurii acesteia) cat si o **instanta** a acesteia (datele propriu-zise).
- **Dependenta datelor** este o restrictie asupra relatiilor (instantelor) care pot constitui valoarea curenta a unei scheme de relatie **R**.
- Dependenta trebuie vazuta ca o legatura intre doua attribute, in sensul ca valoarea unui atribut determina valoarea celuilalt, de exemplu atributul Matricol determina unic atributul Nume al unui student.



Dependente functionale

- **Definitie:** Fie:

R o schema de relatie si

X, Y $\subseteq$ **R** doua multimi de attribute ale acesteia.

Spunem ca **X determina functional pe Y** (sau **Y** este determinata functional de **X**), prin notatia

simbolica **X** $\rightarrow$ **Y** daca si numai daca oricare ar fi doua tupluri t_1 si t_2 din orice instanta a lui **R** atunci:

$$t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y]$$

Cu alte cuvinte, daca doua tupluri au aceleasi valori pe attributele **X** atunci ele au aceleasi valori si pe attributele **Y**.



Dependente functionale

Exemple:

- Consideram relatia FURNIZORI cu urmatoarea structura:
FURNIZORI (ID_furniz, Den_furniz, Adresa) unde:
 - ID_furniz = cheia de identificare unica a unui furnizor;
 - Den_furniz = denumirea furnizorului;
 - Adresa = adresa furnizorului.
- Relatia COMPONENTE este un catalog de componente pentru calculatoare, cu urmatoarea structura:
COMPONENTE(ID_com, Den_com, ID_furniz, Pret, Um) unde:
 - ID_com = cheia de identificare unica a unei componente;
 - Den_com = denumirea componentei;
 - ID_furniz = cheia de identificare unica a unui furnizor pentru o componenta;
 - Pret = pretul unitar al componentei ;
 - Um = unitatea de masura (bucati, metri, set, etc.).



Dependente functionale

- Putem identifica urmatoarele dependente functionale:
- In relatia FURNIZORI :
 - $ID_furniz \rightarrow Den_furniz$
 - $ID_furniz \rightarrow Adresa$
- In relatia COMPONENTE :
 - $ID_com \rightarrow Den_com$
 - $ID_com \rightarrow ID_furniz$
 - $ID_com \rightarrow Pret, Um$
 - $ID_com \rightarrow ID_furniz, Pret$
- Observatie: Singura cale de a gasi dependentele functionale valabile pentru o schema R este analiza atenta a intelesului (semnificatiei) fiecarui atribut al acesteia si a modului in care sunt asignate valori atributelor.



Dependente functionale

- Dependenta functionala $ID_com \rightarrow ID_furniz, Pret$ specifica ca ID_com identifica unic un singur produs, achizitionat de la un singur furnizor si la un anumit pret. In acest caz, daca un produs cu acelasi nume este achizitionat de la un alt furnizor, eventual la acelasi pret, trebuie sa i se dea un alt ID_com .
- Dependenta functionala $ID_furniz \rightarrow Den_furniz$ specifica ca un ID_furniz identifica unic un furnizor de componente si daca doua componente au acelasi ID_furniz inseamna ca ele sunt achizitionate de la acelasi furnizor.
- Observatie: Dependentele functionale nu se determina prin inspectarea valorilor relatiei ci din semnificatia atributelor acesteia.



Axiome si reguli

- Pornind de la o multime de dependente functionale atasate unei scheme de relatie se pot deduce alte dependente functionale valide, folosind reguli de inferenta.
- Exista mai multe reguli de inferenta. Pentru a se putea face o prezentare formala a acestora, trei dintre ele au fost alese ca axiome iar restul se pot deduce pornind de la ele.
- Cele trei axiome (numite Axiomele lui Armstrong) sunt urmatoarele:



Axiome si reguli

- **A1. Reflexivitate:** Fie R o schema de relatie si $X \subseteq R$.

Daca $Y \subseteq X$ atunci $X \rightarrow Y$.

Toate dependentele functionale care rezulta din aceasta axioma sunt numite si **dependente triviale**.

Ele nu spun nimic in plus fata de setul de dependente initial dar sunt dependente functionale valide.

- **A2. Augmentare:** Fie R o schema de relatie si

$X, Y, Z \subseteq R$. Daca $X \rightarrow Y$ atunci si $XZ \rightarrow YZ$.

Aceasta axioma arata ca se poate reuni o aceeaasi multime Z in stanga si in dreapta unei dependente functionale valide, obtinand de asemenea o dependenta functionala valida.



Axiome si reguli

- **A3. Tranzitivitate:** Fie R o schema de relatie si $X, Y, Z \subseteq R$. Daca $X \rightarrow Y$ si $Y \rightarrow Z$ atunci si $X \rightarrow Z$.
- Pe baza acestor axiome se pot demonstra o serie de **reguli de inferenta** pentru dependente functionale dintre care cele mai importante sunt urmatoarele:
- **R1. Descompunere:** Fie R o schema de relatie si $X, Y, Z \subseteq R$. Daca $X \rightarrow Y$ si $Z \subseteq Y$ atunci si $X \rightarrow Z$.
Demonstratie:
 $X \rightarrow Y$ este data. Prin ipoteza $Z \subseteq Y$, aplicand A1 rezulta $Y \rightarrow Z$.
Cu A3, din $X \rightarrow Y$ si $Y \rightarrow Z$ obtinem $X \rightarrow Z$.



Axiome si reguli

- Regula descompunerii ne permite sa rescriem un set de dependente functionale astfel incat sa obtinem dependente care au in partea dreapta doar un singur atribut. Sa presupunem ca avem o dependenta functionala de forma:

$$X \rightarrow A_1 A_2 A_3 \dots A_n$$

- Atunci ea poate fi inlocuita cu urmatoarele **n** dependente functionale:

$$X \rightarrow A_1$$

$$X \rightarrow A_2$$

$$X \rightarrow A_3$$

...

$$X \rightarrow A_n$$



Axiome si reguli

- **R2. Reuniune:** Fie R o schema de relatie si $X, Y, Z \subseteq R$.
Daca $X \rightarrow Y$ si $X \rightarrow Z$ atunci si $X \rightarrow YZ$.

Rezulta si faptul ca din cele n dependente obtinute prin descompunere se poate obtine dependentata initiala, deci inlocuirea acesteia nu duce la pierderea vreunei corelatii existente.

Demonstratie:

$X \rightarrow Y$ este data. Amplificam cu X si prin inferenta obtinem $XX \rightarrow XY$, sau $X \rightarrow XY$. De asemenea, $X \rightarrow Z$ este data; amplificam cu Y si $XY \rightarrow ZY$ sau $XY \rightarrow YZ$.

Aplicand A3 rezulta $X \rightarrow YZ$.



Axiome si reguli

- **R3. Pseudotranzitivitate:** Fie R o schema de relatie si $X, Y, Z, W \subseteq R$.

Daca $X \rightarrow Y$ si $YZ \rightarrow W$ atunci si $XZ \rightarrow W$.

Demonstratie:

$X \rightarrow Y$ este data. Amplificam cu Z si obtinem $XZ \rightarrow YZ$.

Dar $YZ \rightarrow W$ si aplicand A3 rezulta $XZ \rightarrow W$.



Inchiderea unei multimi de DF

- Pornind de la un set de dependente functionale **F** si utilizand axiome si reguli obtinem o multitudine de alte dependente, triviale sau nu. Multimea tuturor dependentelor functionale care se pot deduce din **F** se numeste **inchiderea multimii de dependente functionale F**, notata cu **F⁺**. Definitia formală a acestei inchideri este urmatoarea:

$$F^+ = \{X \rightarrow Y \mid F \Rightarrow X \rightarrow Y\}$$

Unde prin $\Rightarrow$ am notat faptul ca dependenta respectiva se poate deduce din F folosind axiome si reguli.



Inchiderea unei multimi de DF

- Multimea F^+ contine foarte multe dependente, inclusiv dependente triviale ca:

$ABC \rightarrow A$, $ABC \rightarrow B$, $ABC \rightarrow C$, $ABC \rightarrow AB$, $ABC \rightarrow AC$,
 $ABC \rightarrow BC$ sau $ABC \rightarrow ABC$

- F^+ nu se calculeaza in totalitate, algoritmii care au nevoie de ea ocolesc, intr-un fel sau altul, calculul acesteia.
- Introducerea acestei notiuni s-a facut pentru a explica, in cazul descompunerii unei scheme de relatie, care sunt dependentele mostenite de elementele descompunerii de la relatia initiala si pentru a putea defini formal urmatoarele notiuni:



Acoperirea unei multimi de DF

✓ **Acoperirea unei multimi de DF** - Fie R o schema de relatie si F, G doua multimi de dependente pentru R . Se spune ca F **acopera** pe G daca si numai daca $G \subseteq F^+$.

✓ **Echivalenta a doua multimi de dependente** - Fie R o schema de relatie si F, G doua multimi de dependente pentru R .

Se spune ca F este **echivalenta** cu G daca si numai daca F acopera pe G si G acopera pe F , adica, daca $G \subseteq F^+$ si $F \subseteq G^+$ rezulta $F^+ = G^+$.



Forma canonica a unei multimi de DF

- ✓ **Forma canonica a unei multimi de DF** - Din definitiile de mai sus rezulta ca o multime de dependente poate fi inlocuita cu alta echivalenta continand alte dependente. In cazul aceasta se poate spune ca:
 - **Definitie:** O multime de dependente este in **forma canonica** daca:
 - Orice dependenta are in partea dreapta un singur atribut. Acest lucru se poate obtine aplicand regula descompunerii prezentata anterior.
 - Multimea de dependente este minimala, niciuna dintre dependente neputand sa fie dedusa din celelalte (altfel spus, nu exista dependente redundante).



Forma canonica a unei multimi de DF

Exemple:

1) Fie $R = ABCDE$ o schema de relatie si F multimea de dependente functionale asociata, cu

$$F = \{ AB \rightarrow CDE, C \rightarrow DE \}:$$

Aplicam regula de descompunere si obtinem:

$$F = \{ AB \rightarrow C, AB \rightarrow D, AB \rightarrow E, C \rightarrow D, C \rightarrow E \}$$

Multimea nu este insa minimala deoarece $AB \rightarrow D$ si $AB \rightarrow E$ se pot deduce prin tranzitivitate din $AB \rightarrow C$ impreuna cu $C \rightarrow D, C \rightarrow E$.

Rezulta ca forma canonica a lui F este:

$$F = \{ AB \rightarrow C, C \rightarrow D, C \rightarrow E \}$$



Forma canonica a unei multimi de DF

2) Pentru relatia

COMPONENTE = (ID_com, Den_com, Pret, ID_furniz, Den_furniz).

- Multimea de dependente functionale F este:

$$F = \{ ID_com \rightarrow Den_com, Pret, ID_furniz, Den_furniz, ID_furniz \rightarrow Den_furniz \}.$$

- Forma canonica a lui F este:

$$F = \{ ID_com \rightarrow Den_com, ID_com \rightarrow Pret, ID_com \rightarrow ID_furniz, ID_furniz \rightarrow Den_furniz \}$$

A fost eliminata dependenta redundantă $ID_com \rightarrow Den_furniz$.



Implicatii logice ale dependentelor

➤ Implicatii logice ale dependentelor

- Fie F o multime de dependente functionale pentru R si fie $X \rightarrow Y$ o dependenta functionala, valabila tot pentru schema R . Atunci,
 F implica logic $X \rightarrow Y$, daca orice relatie r pentru R , care satisface dependentele din F , satisface si $X \rightarrow Y$.

Exemplu:

- Fie R o schema de relatie, iar A, B, C attribute in R . Avem, de exemplu, dependentele $A \rightarrow B$ si $B \rightarrow C$ valabile in R . Se poate arata ca, de asemenea, ca $A \rightarrow C$ este valabila in R (tranzitivitate).



Inchiderea unei multimi de DF

- **Inchiderea** multimii de dependente F , notata cu F^+ , se mai defineste ca **multimea dependentelor functionale implicate logic** de catre F .

Daca $F^+=F$, F este o familie completa de dependente.

Exemplu:

Fie $R=ABC$ si F multimea de dependente. Atunci, F^+ contine toate dependentele $X \rightarrow Y$, astfel incat:

- X contine pe A , de exemplu $ABC \rightarrow AB$, $AB \rightarrow BC$ sau $A \rightarrow C$.
- X contine B dar nu A , iar Y nu contine A , de exemplu, $BC \rightarrow B$, $B \rightarrow C$, $B \rightarrow \emptyset$.
- $X \rightarrow Y$ este una dintre cele doua dependente $C \rightarrow C$ sau $C \rightarrow \emptyset$.



Cheia si supercheia unei relatii

In acest moment putem sa dam o definitie echivalenta a cheii unei relatii pe baza dependentelor functionale:

- **Definitie:** Fie **R** o schema de relatie, **F** multimea de dependente functionale asociata si $X \subseteq R$. Atunci **X** este **cheie** pentru **R** daca si numai daca:
 - $F \Rightarrow X \rightarrow R$ (deci $X \rightarrow R$ se poate deduce din **F**);
 - **X** este minimala: oricare ar fi $Y \subset X$, $Y \neq X$ atunci $\neg(F \Rightarrow Y \rightarrow R)$ (deci orice submultime stricta a lui **X** nu mai indeplineste conditia anterioara).



Cheia si supercheia unei relatii

- Deci o cheie determina functional toate attributele relatiei, este minimala si nicio submultime stricta a sa nu determina functional pe R. Se observa faptul ca aceasta definitie este echivalenta cu cea din capitolul anterior: cunoscandu-se valorile pe attributele X, pot fi determinate unic valorile pentru toate attributele relatiei, deci este determinat unic un tuplu din relatie.
- In cazul in care doar prima conditie este indeplinita (fara minimalitate) multimea X se numeste **supercheie**.
- Observatie: Faptul ca o supercheie nu este constransa de minimalitate nu inseamna insa ca ea nu poate fi minimala. Rezulta ca orice cheie este in acelasi timp si supercheie, reciproca nefiind insa adevarata.



Cheia si supercheia unei relatii

Exemplu:

Fie $R = ABCDE$ si $F = \{ AB \rightarrow C, C \rightarrow D, C \rightarrow E \}$. Atunci AB este cheie pentru R:

- Din $AB \rightarrow C, C \rightarrow D$ si $C \rightarrow E$ obtinem prin tranzitivitate $AB \rightarrow D$ si $AB \rightarrow E$;
- Din $AB \rightarrow C, AB \rightarrow D$ si $AB \rightarrow E$ obtinem prin reuniune $AB \rightarrow CDE$;
- Din $AB \rightarrow CDE$ obtinem prin augmentare cu AB $AB \rightarrow ABCDE$, deci $AB \rightarrow R$;
- Rezulta ca AB este supercheie pentru R. Vom arata intr-un alt capitol cum se poate demonstra ca AB este si minimala, deci este chiar cheie pentru R, nu numai supercheie.



Proiectia unei multimi de DF

- Asa cum s-a mentionat anterior, inchiderea unei multimi de dependente functionale F^+ a fost introdusa si pentru a putea defini setul de dependente functionale mostenite de o schema de relatie obtinuta prin descompunerea unei scheme incorect proiectata.
- Sa analizam o noua structura a relatiei:

COMPONENTE_FURNIZORI =

(ID_com, Den_com, Pret, ID_furniz, Den_furniz, Adresa).

Multimea de dependente functionale F este:

$$F = \{ \text{ID_com} \rightarrow \text{Den_com}, \text{ID_com} \rightarrow \text{Pret}, \\ \text{ID_com} \rightarrow \text{ID_furniz}, \text{ID_furniz} \rightarrow \text{Den_furniz}, \\ \text{ID_furniz} \rightarrow \text{Adresa} \}$$



Proiectia unei multimi de DF

- Prin descompunerea acestei relatii in doua relatii, obtinem relatiile:

COMPONENTE = (ID_com, Den_com, Pret, ID_furniz)

FURNIZORI = (ID_furniz, Den_furniz, Adresa)

- Atributele relatiei initiale se regasesc fie doar intr-una dintre schemele rezultate, fie in amandoua.

Se pune problema identificarii dependentelor mostenite de cele doua relatii de la relatia initiala.

Pentru aceasta trebuie definita proiectia unei multimi de dependente pe o multime de attribute.



Proiectia unei multimi de DF

- **Definitie.** Fie o relatie **R**, o multime asociata de dependente functionale **F** si o submultime de attribute **S** $\subseteq$ **R** .

Proiectia multimii de dependente F pe S, notata cu $\pi_S(F)$ este multimea dependentelor din F^+ care au attributele din partea stanga si cea dreapta incluse in **S**.

Formal, putem scrie:

$$\pi_S(F) = \{X \rightarrow Y \in F^+ \mid X, Y \subseteq S\}$$



Proiectia unei multimi de DF

Exemplu:

Fie relatia COMPONENTE_FURNIZORI =

(ID_com, Den_com, Pret, ID_furniz, Den_furniz, Adresa)
avand multimea de dependente functionale:

$F = \{ ID_com \rightarrow Den_com, ID_com \rightarrow Pret, ID_com \rightarrow ID_furniz, ID_furniz \rightarrow Den_furniz, ID_furniz \rightarrow Adresa \}.$

- Aplicand definitia se obtin proiectiile urmatoare:

$F_{COMPONENTE} = \pi_{COMPONENTE}(F) = \{ ID_com \rightarrow Den_com, ID_com \rightarrow Pret, ID_com \rightarrow ID_furniz \}$

$F_{FURNIZORI} = \pi_{FURNIZORI}(F) = \{ ID_furniz \rightarrow Den_furniz, ID_furniz \rightarrow Adresa \}$



Proiectia unei multimi de DF

- Observatie: Atunci cand descompunem o schema se poate intampla ca unele dintre dependentele schemei initiale sa se piarda.

Exemplu: Fie $R = ABCD$ si $F = \{ A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A \}$.

In cazul in care descompunem R in $R_1 = AB$ si $R_2 = CD$

atunci: $F_{R_1} = \pi_{R_1}(F) = \{ A \rightarrow B, B \rightarrow A \}$ si

$$F_{R_2} = \pi_{R_2}(F) = \{ C \rightarrow D, D \rightarrow C \}$$

- A doua dependenta din fiecare multime nu este in F dar este in F^+ (obtinuta prin tranzitivitate).
- Observam insa ca dependentele $B \rightarrow C$ si $D \rightarrow A$ nu mai pot fi obtinute nici din F_{R_1} , nici din F_{R_2} si nici din reuniunea lor. Exista metode prin care se poate verifica daca prin descompunere dependentele initiale sunt pastrate sau nu.



Capitolul 5

Normalizarea schemelor de relatie



Normalizarea schemelor de relatie

- Dupa cum am aratat in capitolele anterioare, proiectarea unei baze de date se poate face in mai multe feluri si se pune problema care este cea mai buna optiune.
- Un obiectiv important in alegerea modelului de date este realizarea unei reprezentari corecte a datelor, a relatiilor dintre ele si a restrictiilor impuse asupra lor.
- Pentru realizarea acestui obiectiv se utilizeaza tehnica normalizarii, care are ca scop principal identificarea schemei de relatii care sa modeleze cat mai corect realitatea functionala.
- Proiectarea incorecta a schemelor de relatie poate duce la aparitia diferitelor anomalii care complica procesul de dezvoltare sau exploatare a unei aplicatii software.



Normalizarea schemelor de relatie

- Ca metoda de testare a corectitudinii unei scheme de relatie este verificarea dependentelor functionale atasate schemei.
- Una dintre regulile de baza in proiectare este aceea ca datele nu trebuie sa fie redundante, adica aceleasi date sa fie stocate de mai multe ori in aceeași relatie sau relatii diferite.
- O alta regula este ca datele care se pot deduce prin prelucrare din alte date nu trebuie stocate in baza de date.
- **Normalizarea** reprezinta un proces de descompunere a unei relatii in mai multe relatii cu scopul eliminarii anomaliilor de proiectare si exploatare a bazei de date. Procesul de normalizare se realizeaza cu ajutorul **formelor normale** care impun **regulile de descompunere**.
- O schema de relatie obtinuta in urma normalizarii , care indeplineste un set de standarde si cerintele specifice, se spune ca este intr-o **forma normala**.



Normalizarea schemelor de relatie

- In anul 1970 Edgar F. Codd a definit formalismul primelor forme normale: Forma Normala 1(FN1), Forma Normala 2(FN2) si Forma Normala 3(FN3).
- In anul 1974, Edgar F. Codd impreuna cu Raymond F. Boyce, au definit FNBC(Forma Normala Boyce-Codd).
- Ulterior au fost definite formele normale FN4, FN5 si FN6 care au avut aplicabilitate destul de redusa in proiectarea bazelor de date.



Forma normala 1 (FN1)

- **Definitie:** O relatie R se gaseste in **Forma Normala 1 (FN1)** daca si numai daca:
 - Pe toate attributele sale exista doar valori atomice ale datelor(nu exista attribute cu valori multiple);
 - Nu exista attribute sau grupuri de attribute care se repeta.
- Semnificatia termenului “atomic” este similara cu cea de la modelul entitate asociere: valoarea respectiva este intotdeauna folosita ca un intreg si nu se utilizeaza niciodata doar portiuni din aceasta. In prima forma normala, domeniul fiecarui atribut este construit din valori indivizibile. Cu alte cuvinte, toate attributele trebuie sa fie atomice, adica sa contina o singura informatie.

Exemplu: ABONATI (Cod_abonat, Nume, Adresa, Tip_abon).
Aceasta relatie este in FN1 daca atributul Adresa este atomic, adica niciodata nu este nevoie sa fie folosite doar anumite parti ale sale (oras, sector, strada, etc.).



Forma normala 2 (FN2)

- Fiind data o schema de relatie R si multimea de dependente functionale asociata F , putem defini inca doua concepte.

Fie A un atribut care nu face parte din cheie si X o multime de attribute din R care formeaza o cheie a relatiei. Atunci:

- **Definitie:** O dependenta functionala $Y \rightarrow A$ se numeste **dependenta partiala** daca Y este strict inclusa intr-o cheie a relatiei R .

(X este cheie, $Y \subset X$, $Y \rightarrow A$ si A nu face parte din cheie) sau
(avem dependenta $X \rightarrow Y \rightarrow A$, unde: $X = \text{cheie}$, $Y \subset X$ si $A \notin X$).

- **Definitie:** O dependenta functionala $Y \rightarrow A$ se numeste **dependenta tranzitiva** daca Y nu este inclusa in nicio cheie a relatiei R .

(X este cheie, $Y \not\subset X$, $Y \rightarrow A$ si A nu face parte din cheie) sau
(avem dependenta $X \rightarrow Y \rightarrow A$, unde: $X = \text{cheie}$, $Y \not\subset X$ si $A \notin X$).



Forma normala 2 (FN2)

- **Definitie:** Fie R o schema de relatie si F multimea de dependente functionale asociata. Relatia R este in **Forma Normala 2 (FN2)** daca si numai daca F respecta cerintele FN1 si nu contine dependente partiale (dar poate contine dependente tranzitive).

Cu alte cuvinte, o relatie se gaseste **Forma Normala 2** daca si numai daca:

- Se gaseste in FN1 si
- Orice atribut care nu face parte dintr-o cheie va fi identificat de intreaga cheie, nu doar de unele attribute care fac parte din cheie.



Forma normala 2 (FN2)

Observatie:

- Daca o entitate se gaseste in FN1 si cheia sa este formata dintr-un singur atribut, atunci se gaseste automat si in FN2.
- FN1 si FN2 nu garanteaza eliminarea anomaliilor si nu sunt recomandate pentru proiectarea schemelor de relatie ale unei baze de date.



Forma normala 2 (FN2)

Exemple:

1) Relatia PRODUSE = Cod_prod, Den_prod, Pret, Cod_furniz, Den_furniz, Adresa

cu dependentele functionale:

$F = \{ \text{Cod_prod} \rightarrow \text{Den_prod}, \text{Cod_prod} \rightarrow \text{Pret}, \text{Cod_prod} \rightarrow \text{Cod_furniz}, \text{Cod_furniz} \rightarrow \text{Den_furniz}, \text{Cod_furniz} \rightarrow \text{Adresa} \}$ si cheia unica Cod_prod.

- Relatia este in FN2 deoarece cheia are un singur atribut. Relatia nu are dependente partiale iar ultimele doua dependente sunt dependente tranzitive deoarece Cod_furniz nu apartine cheii unice Cod_prod.



Forma normala 2 (FN2)

2) Relatia

NOTE = Nr_matricol, Nume_stud, Cod_discip,
Den_discip, Nota, Data_ex

are cheia (Nr_matricol, Cod_discip) ,

considerand ca un student are o singura nota la o disciplina (nota finala).

- Relatia NOTE nu este in FN2 deoarece dependentele $\text{Nr_matricol} \rightarrow \text{Nume_stud}$ si $\text{Cod_discip} \rightarrow \text{Den_discip}$ sunt dependente partiale (Nr_matricol, Cod_discip fac parte din cheie).



Forma normala 3 (FN3)

- Pentru a defini forma FN3 este necesara definirea notiunii de **atribut prim**:
- **Definitie.** Fie R o schema de relatie si F multimea de dependente functionale asociata.

Un atribut $A \in R$ se numeste **atribut prim** daca el apartine unei chei a lui R.

Exemplu:

Fie $R = ABCDE$ avand $F = \{ A \rightarrow B, B \rightarrow A, A \rightarrow C, D \rightarrow E \}$. Cum cheile relatiei sunt AD si BD rezulta ca in R sunt trei attribute prime: A, B si D.



Forma normala 3 (FN3)

- **Definitie.** Fie R o schema de relatie si F multimea de dependente functionale asociata.

Relatia R este in **Forma Normala 3**(FN 3) daca si numai daca oricare ar fi o dependenta netriviala

$X \rightarrow A$ din F , atunci:

- X este supercheie pentru R
sau
- A este atribut prim.



Forma normala 3 (FN3)

- De remarcat ca daca in F avem dependente care contin mai multe attribute in partea dreapta putem aplica regula de descompunere pentru a obtine dependente care in partea dreapta au cate un singur atribut.
- Observatie : O relatie se gaseste in FN3 daca si numai daca se gaseste in FN2 si, in plus, niciun atribut care nu este parte a unei chei nu depinde de un alt atribut care nu face parte din cheie. Cu alte cuvinte, nu se accepta dependente tranzitive, adica un atribut sa depinda de o cheie in mod indirect.



Forma normala 3 (FN3)

Exemple:

- 1) Fie $R=NLAP$, o schema de relatie pentru furnizori cu attributele (Nume, Localitate, Articol, Pret) si dependentele $F=\{NA \rightarrow P \text{ si } N \rightarrow L\}$.
 - Relatia are cheia NA si nu respecta FN3 deoarece are dependenta partiala $N \rightarrow L$ (de asemenea nu respecta nici FN2). Intr-adevar, fie $X=NA$, $Y=N$. Atributul L (localitate) este neprim, deoarece singura cheie este NA. Atunci $X \rightarrow Y$ si $Y \rightarrow L$ sunt dependentele valabile, pe cand $Y \rightarrow X$ (adica $N \rightarrow NA$) nu este valabila. Vom observa ca in acest caz, $X \rightarrow Y$ si $Y \rightarrow L$ nu numai ca “functioneaza” in R, dar ele sunt dependente date. In general, este suficient ca $X \rightarrow Y$ si $Y \rightarrow L$ sa decurga dintr-o multime data de dependente, chiar daca sunt date ca atare.
- 1) Fie $R=OSC$, o schema de relatie pentru localitati cu attributele (Oras, Strada, Cod) si dependentele $F=\{OS \rightarrow C \text{ si } C \rightarrow O\}$. Cheile sunt OS si SC. Relatia are toate attributele prime si este in FN3.



Forma normala 3 (FN3)

- 3) Fie $R=MADS$, o relatie pentru magazine cu attributele (Magazin, Articol, Departament, Sef). Presupunem ca functioneaza urmatoarele dependente functionale:
- $MA \rightarrow D$ (fiecare articol, in fiecare magazin , este vandut de cel mult un departament/raion);
 - $MD \rightarrow S$ (fiecare departament/raion , din fiecare magazin, are un sef).
- Relatia nu este in FN3 deoarece:
 - Relatia are o singura cheie pe MA, iar MD nu este supercheie.
 - Daca notam $X=MA$ si $Y=MD$, atunci $X \rightarrow D$ si $Y \rightarrow S$ nu respecta regulile care definesc FN3 deoarece D si S nu sunt attribute prime.
 - Relatia este in FN2 pentru ca nu exista dependente parțiale (nicio submultime proprie a cheii MA nu determina functional attributele D sau S).



Forma normala 3 (FN3)

✓ Necesitatea FN3

- Asa cum am aratat, prin FN3 se evita multe dintre probleme legate de redundanta si de anomaliiile de actualizare.
- Putem presupune, astfel, ca dependentele functionale $X \rightarrow Y$ nu reprezinta numai o restrictie de integritate asupra relatiilor, ci reprezinta, in acelasi timp, o legatura (asociere) pe care baza de date “are intentia sa o memoreze”. Cu alte cuvinte, daca atributelor din X le este asignata o multime de valori, consideram important sa stim ce valoare, pentru fiecare atribut din Y, este asociata cu aceasta “asignare” de valori pentru attributele din X.



Forma normala 3 (FN3)

- Daca avem o dependenta partiala $Y \rightarrow A$, X fiind o cheie iar Y o submultime proprie a lui X , atunci in fiecare tuplu folosit pentru a asocia o valoare, din multimea asignata lui X , cu valori pentru alte attribute in afara de A si de attributele din X , trebuie sa apara aceeasi asociere intre X si A .
- Aceasta situatie este usor de evidentiata in relatia $R=NLAP$, cu cheia NA in care $N \rightarrow L$ este o dependenta partiala, iar localitatea furnizorului trebuie sa fie repetata pentru fiecare articol livrat de furnizor.
Evident ca FN3 elimina aceasta anomalie, precum si redundantele respective sau erorile de actualizare.



Forma normala 3 (FN3)

- In caz ca exista o dependenta tranzitiva $X \rightarrow Y \rightarrow A$, atunci nu putem asocia o valoare Y cu o valoare X, daca nu exista o valoare A asociata cu valoarea Y. Aceasta situatie conduce la anomalii de inserare si de stergere deoarece nu putem insera o asociere X-la-Y fara o asociere Y-la-A, iar daca stergem valoarea A asociata cu o valoare Y data vom “pierde legatura” unei asocieri X-la-Y.

De exemplu, in schema de relatie $R = \text{MADS}$, cu dependentele $F = \{MA \rightarrow D \text{ si } MD \rightarrow S\}$, nu putem inregistra un departament oarecare daca acel departament nu are sef, iar daca stergem un sef “dispare” si departamentul asociat lui.



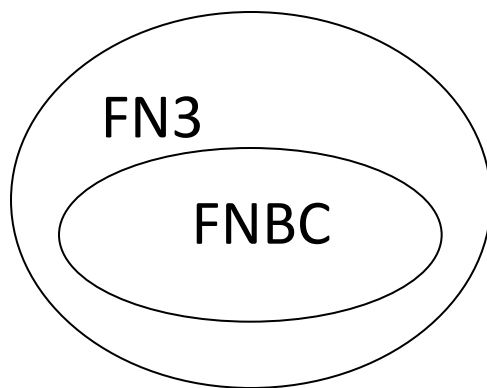
Forma normala Boyce-Codd (FNBC)

- **Definitie.** Fie R o schema de relatie si F multimea de dependente functionale asociata. Se spune ca R este in **Forma Normala Boyce-Codd (FNBC)** daca si numai daca oricare ar fi o dependenta netriviala $X \rightarrow Y$ din F atunci X este supercheie pentru R .
- Rezulta ca o relatie este in FNBC daca si numai daca fiecare dependenta din F are in partea stanga o supercheie.
- Nu este obligatoriu ca F sa fie in forma canonica, dar trebuie sa nu contina dependente triviale (obtinute din prima axioma de reflexivitate, de tipul $AB \rightarrow A$ sau $AB \rightarrow AB$).
- Dependenta triviala $R \rightarrow R$ este admisa in FNBC (o relatie nu contine linii duplicat si cheia este compusa din toate attributele).



Forma normala Boyce-Codd (FNBC)

- Observatie: Conditia de FNBC este inclusa in definitia FN3. Din acest motiv orice relatie care este in FNBC este implicit si in FN3. Reciproca nu este adevarata. Rezulta de asemenea ca daca o schema de relatie nu este in FN3 ea nu poate fi nici in FNBC.





Forma normala Boyce-Codd (FNBC)

Exemple:

- 1) Relatia $R = ABCDE$ avand $F = \{ A \rightarrow B, B \rightarrow A, A \rightarrow C, D \rightarrow E \}$ nu este in forma normala Boyce-Codd deoarece are cheile AD si BD dar nicio dependenta nu are in partea stanga o supercheie a lui R .
- 2) Relatia $R = ABCD$ avand $F = \{ AB \rightarrow C, AB \rightarrow D, D \rightarrow A \}$ are cheia unica AB .
 - Relatia este in FN3 deoarece primele doua dependente au in partea stanga o supercheie (AB) iar a treia dependenta are in partea dreapta atributul prim A .
 - Relatia nu este in FNBC deoarece a treia dependenta violeaza definitia pentru aceasta forma normala (nu are in partea stanga o supercheie).



Forma normala Boyce-Codd (FNBC)

3) Relatia $R = ABCDE$ avand $F = \{ A \rightarrow B, B \rightarrow A, A \rightarrow C, D \rightarrow E \}$ are cheile AD si BD . Rezulta ca:

- R nu este in FN3 deoarece dependentele 3 si 4 nu au nici supercheie in partea stanga si nici atribut prim in partea dreapta;
- R nu este in FNBC deoarece nu e in FN3.

4) Relatia $R = ABCD$ avand $F = \{ A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A \}$ are cheile A, B, C si D . Rezulta ca:

- R este in FNBC deoarece in partea stanga a dependentelor sunt numai superchei;
- R este in FN3 deoarece este in FNBC.



Forma normala Boyce-Codd (FNBC)

5) Relatia PRODUSE_FURNIZORI = Cod_prod, Den_prod, Pret, Cod_furniz, Den_furniz, Adresa

cu dependentele functionale:

$F = \{ \text{Cod_prod} \rightarrow \text{Den_prod}, \text{Cod_prod} \rightarrow \text{Pret}, \text{Cod_prod} \rightarrow \text{Cod_furniz}, \text{Cod_furniz} \rightarrow \text{Den_furniz}, \text{Cod_furniz} \rightarrow \text{Adresa} \}$

- Relatia nu este in FNBC deoarece cheia unica este Cod_prod dar exista dependente care nu au in partea stanga o supercheie:

$\text{Cod_furniz} \rightarrow \text{Den_furniz}, \text{Cod_furniz} \rightarrow \text{Adresa}$

- Relatia nu este nici in FN3 deoarece Den_furniz si Adresa nu sunt attribute prime, in plus aceste dependente sunt si tranzitive.



Forma normala Boyce-Codd (FNBC)

- 6) Relatia PRODUSE_FURNIZORI = Cod_prod, Den_prod, Pret, Cod_furniz, Den_furniz, Adresa cu dependentele:
- $$F = \{ \text{Cod_prod} \rightarrow \text{Den_prod}, \text{Cod_prod} \rightarrow \text{Pret}, \\ \text{Cod_prod} \rightarrow \text{Cod_furniz}, \text{Cod_furniz} \rightarrow \text{Den_furniz}, \\ \text{Cod_furniz} \rightarrow \text{Adresa} \}$$
- Consideram proiectia
PRODUSE= Cod_prod, Den_prod, Pret, Cod_furniz
 $F_{\text{PRODUSE}} = \pi_{\text{PRODUSE}}(F) = \{ \text{Cod_prod} \rightarrow \text{Den_prod}, \\ \text{Cod_prod} \rightarrow \text{Pret}, \text{Cod_prod} \rightarrow \text{Cod_furniz} \}$ care este in FN3 deoarece cheia relatiei este Cod_prod si toate dependentele au in partea stanga o supercheie (asa cum s-a mentionat, orice cheie este in acelasi timp si supercheie). Proiectia este in FN3 deoarece este in FNBC.



Anomalii in baze de date

Anomaliile care pot sa apara intr-o baza de date sunt de doua feluri:

- Anomalii de **proiectare**: redundante, tipuri gresite de attribute, constrangeri incorecte, etc.
- Anomalii de **functionare**: sunt cele care apar la operatii DML (inserare, modificare, stergere).
- Pentru a intelege mai bine anomaliile sa consideram relatia NOTE(in care un student are o singura nota la o disciplina):

NOTE = Matricol, Nume, Disciplina, Nota, Data_ex,
Cod_spec, Den_spec
cu urmatoarele date:



Anomalii in baze de date

Matricol	Nume	Disciplina	Nota	Data_ex	Cod_spec	Den_spec
1011	Ionescu Silvia	BD1	9	08.06.2024	1	CTI
1011	Ionescu Silvia	BD2	10	10.02.2025	1	CTI
1022	Popescu Daniel	PM	8	11.02.2025	2	AII
1022	Popescu Daniel	SO	7	03.06.2024	2	AII
1033	Popa Cornel	BD1	8	08.06.2024	1	CTI
1033	Popa Cornel	BD2	8	10.02.2025	1	CTI



Anomalii in baze de date

- **Redundanta** apare atunci cand datele sunt stocate de mai multe ori in baza de date(in aceeași relatie sau in relatii diferite).
In relatia NOTE se observa ca atributul Den_spec este redundant pentru ca este suficient atributul Cod_spec care specifica codul specializarii studentului si determina unic denumirea acesteia. In acest caz ar trebui sa se descompuna relatia in doua relatii , NOTE si SPECIALIZARI .



Anomalii in baze de date

Existenta redundantelor intr-o baza de date produce urmatoarele efecte negative:

- Alocare suplimentara de spatiu fizic;
- Inconsistenta datelor;
- Cresterea costurilor de acces la baza de date;
- Prelucrari eronate ale datelor;
- Creste probabilitatea erorii umane in timpul actualizarilor pe baza de date.



Anomalii in baze de date

- **Tipurile atributelor** pot genera anomalii, de exemplu daca atributul Data_ex este definit de tip Number, in loc de tip Date, atunci cand se insereaza data se va genera eroare de format. Daca este definit Text pot aparea erori la anumite functii de tip data calendaristica.
- **Constrangerile** incorecte apar in momentul definirii relatiilor dar efectul lor se vede, de regula, in timpul functionarii. De exemplu, daca in tabela NOTE se defineste cheia relatiei ca fiind Matricol, atunci nu se poate insera decat nota la o singura disciplina, a doua operatie va genera o eroare de violare de cheie.



Anomalii in baze de date

- **Inserarea** poate genera anomalii, de exemplu codurile specializarilor sa nu corespunda cu denumirile de specializare sau sa fie folosite coduri diferite pentru aceeasi specializare. Anomalia se inlatura daca se creeaza relatia SPECIALIZARI(Cod_spec, Den_spec). O alta anomalie este ca nu se stiu numarul matricol si specializarea unui student pana cand nu se insereaza cel putin o nota la o disciplina.
- **Modificarea** datelor poate duce la inconsistenta datelor, de exemplu daca un student isi schimba specializarea trebuie sa se modifice Cod_spec, Den_spec la toate inregistrarile si este posibil sa fie omise unele dintre ele.



Anomalii in baze de date

- **Stergerea** poate produce anomalii, de exemplu daca se sterg toate inregistrarile aferente unui student atunci nu se mai stie numarul matricol si nici specializarea lui. In acest caz se poate crea relatia STUDENTI(Matricol, Nume, Cod_spec) si anomalia este evitata.
- Majoritatea anomaliilor semnalate au aparut deoarece structura relatiei NOTE contine doua clase de obiecte care trebuie separate. Daca folosim modelul Entitate-Asociere diagrama corecta este cea din Figura 2.



Anomalii in baze de date

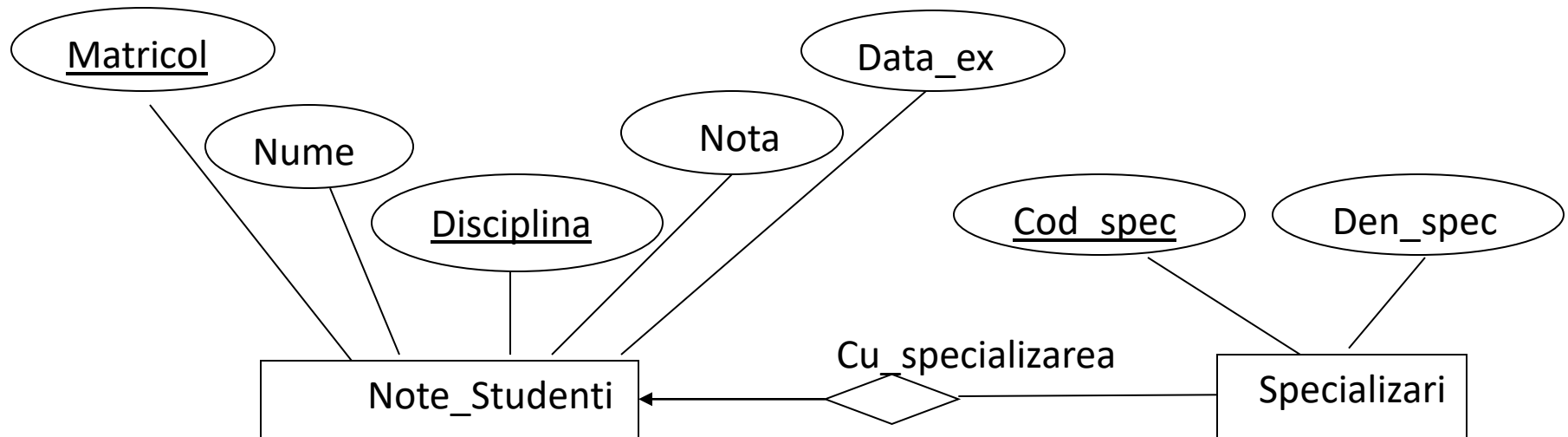


Figura 2. Diagrama entitate-asociere reproiectata



Anomalii in baze de date

- Prin transformarea diagramei initiale se obtin urmatoarele relatii:

NOTE_STUDENTI = Matricol, Nume, Disciplina, Nota, Data_ex, Cod_spec

SPECIALIZARI = Cod_spec, Den_spec

- In acest caz tabelele vor contine urmatoarele date:

SPECIALIZARI

Cod_spec	Den_spec
1	CTI
2	AII



Anomalii in baze de date

NOTE_STUDENTI

Matricol	Nume	Disciplina	Nota	Data_ex	Cod_spec
1011	Ionescu Silvia	BD1	9	08.06.2024	1
1011	Ionescu Silvia	BD2	10	10.02.2025	1
1022	Popescu Daniel	PM	8	11.02.2025	2
1022	Popescu Daniel	SO	7	03.06.2024	2
1033	Popa Cornel	BD1	8	08.06.2024	1
1033	Popa Cornel	BD2	8	10.02.2025	1

- Procesul de “spargere” a unei relatii, care are o structura incorecta, in doua sau mai multe relatii se numeste descompunerea schemei de relatie si va fi prezentat intr-un alt capitol.



Anomalii in baze de date

Existenta anomaliilor de proiectare poate crea o serie de probleme, cum ar fi:

- Cresterea costurilor prin cresterea spatiului de stocare a datelor;
- Aparitia inconsistentei datelor(valori diferite ale datelor pentru acelasi identificator);
- Nerespectarea standardelor pentru baze de date;
- Imposibilitatea aplicarii constrangerilor de integritate;
- Extragerea de informatii eronate in urma prelucrarii datelor.

Atunci cand se proiecteaza o baza de date trebuie facuta o analiza atenta pentru a vedea cum trebuie grupate attributele in relatii cu scopul de a minimiza redundanta datelor si implicit spatiul fizic alocat.



Independenta datelor

- O baza de date poate fi accesata de mai multe aplicatii, sau module ale aceleiasi aplicatii, in mod concurent. In acest caz se pune problema in ce mod sunt afectate aplicatiile de modificarile efectuate in structura bazei de date. In mod normal, modificarile majore de structura(nivel inferior) implica si modificari la nivelul aplicatiei(nivel superior).
- Acest aspect tine de independenta datelor si poate fi vazuta sub doua aspecte:
 - Independenta logica;
 - Independenta fizica.



Independenta datelor

- **Independenta logica** a datelor se refera la imunitatea schemelor externe fata de modificarile efectuate in schema conceptuala, cum ar fi:
 - Adaugarea de entitati sau attribute noi;
 - Modificarea structurii entitatilor;
 - Stergerea anumitor entitati sau attribute;
 - Adaugarea, modificarea sau stergerea de obiecte in baza de date(tabele,view-uri,proceduri,functii, etc.);
 - Activarea sau dezactivarea unor constrangeri.
- Cu alte cuvinte, independenta logica a datelor permite sa se faca modificari in structura bazei de date fara a implica rescrierea codului program in aplicatie.



Independenta datelor

- **Independenta fizica** de date se refera la imunitatea schemei conceptuale fata de modificarile efectuate in schema interna, cum ar fi:
 - Reorganizarea fisierelor de date;
 - Introducerea de noi dispozitive de stocare;
 - Exportul bazei de date de pe un server pe altul;
 - Modificari in configurarea retelei, in cazul bazelor de date distribuite;
 - Replicarea datelor din/pe mai multe servere;
 - Utilizarea serverelor redundante pentru o securitate sporita a datelor.



Independenta datelor

- Independenta datelor, de cele mai multe ori, este o problema foarte grea chiar si in cazul sistemelor de ultima generatie. Bazele de date relationale au adus imbunatatiri majore in acest sens insa complexitatea mare a aplicatiilor face ca problema independentei sa fie un concept care trebuie in continuare studiat si imbunatatit.
- Instrumentele moderne de proiectare software ofera facilitatea de modificare in cascada a obiectelor relationate, atunci cand unul dintre obiecte sufera alterari de structura.



Independenta datelor

- In cazul instrumentelor CASE este posibil, uneori, ca o simpla recompilare sau o noua generare a aplicatiei sa preia toate modificarile de structura intervenite la nivelul bazei de date. Sunt insa putine astfel de instrumente si de regula rezolva partial problema, in sensul ca trebuie facute si interventii manuale ale dezvoltatorilor. Pot fi si situatii cand o aplicatie nu este afectata de modificarea bazei de date, de exemplu daca se adauga entitati sau attribute noi, functionarea aplicatiei nu este afectata. Insa aceste modificari sunt facute, de regula, cu scopul de a imbunatati sau adauga noi functionalitati in aplicatie si in acest caz trebuie intervenit si la nivel extern.



Independenta datelor

- In cazul cererilor de interogare sunt situatii cand nu trebuie rescrise, de exemplu daca se adauga noi entitati sau se modifica relatiile dintre entitati fara afectarea datelor existente in baza de date.
- Acelasi lucru se intampla si in cazul vederilor(*view*) create pe una sau mai multe entitati, daca attributele initiale nu sunt alterate nu trebuie recreata vederea, dar si aici trebuie studiate cu atentie implicatiile care pot sa apara.



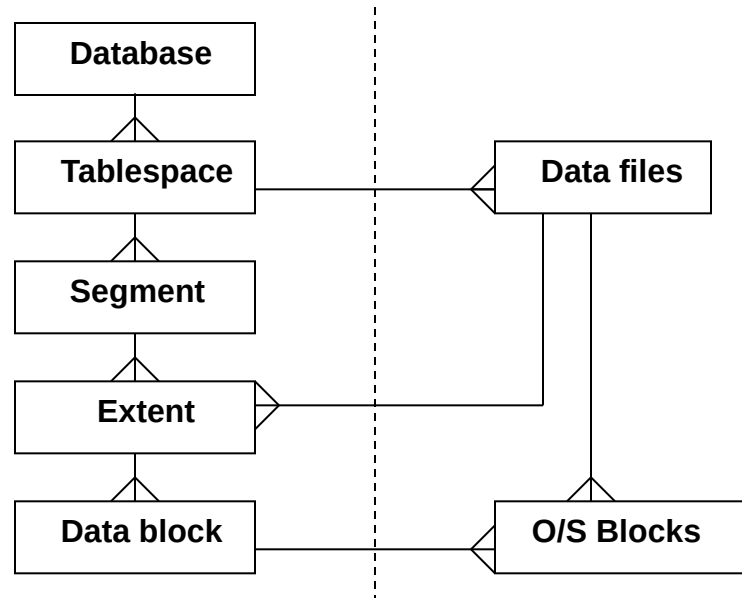
Capitolul 6

Limbajul SQL



Structura bazei de date

- Structura logica si fizica a unei **Baze de Date** (*database*) relationale:



- Tablespace - este spatiul logic in care se creeaza obiectele (tabele, view-uri, indecsi, proceduri, etc.). O baza de date poate avea mai multe tablespace-uri, iar un tablespace poate avea alocat fizic mai multe fisiere de date (*data files*) .



Structura bazei de date

- Segmentul(*segment*) – reprezinta un spatiu logic de stocare alocat unui obiect intr-un tablespace. Exista mai multe tipuri de segmente: permanente, temporare, index, rollback, etc.).
- Extensia(*extent*) - reprezinta o extensie logica a spatiului de stocare reprezentata printr-un numar contiguu de blocuri.
- Blocul de date(*data block*) - reprezinta cea mai mica unitate logica de stocare.
- Fisiere de date(*data files*) - sunt fisierele organizate fizic pe un dispozitiv de stocare. Fisierele de date stocheaza fizic datele in baza de date.
- O/S block - reprezinta cea mai mica unitate fizica de organizare a datelor intr-o baza de date.



Sistemul de Gestione a Bazei de Date

- Controlul asupra bazei de date este gestionat de catre **Sistemul de Gestione a Bazei de Date(SGBD)** si verifica respectarea unor reguli:
 - O baza de date relationala apare ca o colectie de tabele definite de catre utilizator;
 - Utilizatorul nu controleaza felul cum este organizata fizic informatia;
 - Controlul asupra fisierelor de date este gestionat exclusiv de catre sistemul de gestiune;
 - Utilizatorul poate defini anumiti parametri de sistem pentru optimizarea aplicatiilor sau pentru diferite setari;
 - Accesul la baza de date este gestionat exclusiv de catre sistem prin executarea de comenzi specifice;
 - Rularea aplicatiilor, atat pe server cat si pe masina client, este gestionata exclusiv de catre sistemul de gestiune.



Limbajul SQL

- Un sistem de gestiune a bazei de date necesita un limbaj de interogare pentru a permite utilizatorului sa acceseze datele.
- **Limbajul SQL** (*Structured Query Language*) este un limbaj de interogare structurat utilizat de majoritatea sistemelor de baze de date relationale.
- Cateva trasaturi caracteristice ale limbajului SQL:
 - Implementeaza setul standard de comenzi de manipulare a datelor(inserare, interogare, modificare, stergere);
 - Este un limbaj neprocedural care optimizeaza automat planul de executie a cererilor;
 - Cererile se executa secvential, linie cu linie, deci se prelucreaza o singura inregistrare dintr-o tabela la un moment dat .
 - Permite importul si exportul datelor;
 - Oferă suport pentru administrarea bazei de date.



Limbajul SQL

DB DB-Engines Ranking - popularit

db-engines.com/en/ranking/relational+dbms

Featured Products: [MongoDB](#) [Neo4j](#) [Bytebase](#)

Select a ranking

- Complete ranking
- Relational DBMS
- Key-value stores
- Document stores
- Time Series DBMS
- Graph DBMS
- Search engines
- Vector DBMS
- Object oriented DBMS
- RDF stores
- Wide column stores
- Multivalue DBMS
- Spatial DBMS
- Native XML DBMS
- Event Stores
- Columnar
- Content stores
- Navigational DBMS
- Time Series, Observability
- Multi-Model DBMS
- NoSQL

Special reports

- Ranking by database model

Ranking > Relational DBMS

DB-Engines Ranking of Relational DBMS

The DB-Engines Ranking ranks database management systems according to their popularity. The ranking is updated monthly.

This is a partial list of the [complete ranking](#) showing only relational DBMS.

Read more about the [method](#) of calculating the scores.

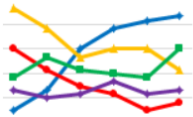
include secondary database models

166 systems in ranking, February 2026

Rank			DBMS	Database Model	Score		
Feb 2026	Jan 2026	Feb 2025			Feb 2026	Jan 2026	Feb 2025
1.	1.	1.	Oracle	Relational, Multi-model	1203.51	-33.82	-51.31
2.	2.	2.	MySQL	Relational, Multi-model	868.22	+0.70	-131.77
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model	708.14	+1.88	-78.73
4.	4.	4.	PostgreSQL	Relational, Multi-model	672.03	+5.77	+12.42
5.	5.	5.	Snowflake	Relational	208.14	+0.34	+52.56
6.	6.	9.	Databricks	Multi-model	144.51	+2.97	+54.48
7.	7.	6.	IBM Db2	Relational, Multi-model	111.22	-1.50	-14.21
8.	8.	7.	SQLite	Relational	99.19	-1.42	-14.63
9.	9.	10.	MariaDB	Relational, Multi-model	86.09	-1.64	-3.42
10.	10.	11.	Microsoft Azure SQL Database	Relational, Multi-model	73.66	-0.53	+0.90
11.	11.	12.	Apache Hive	Relational	73.00	-0.48	+10.51

RSS

RSS Feed



trend chart

Type here to search

4°C Cloudy

9:19 AM 2/8/2026

https://db-engines.com/en/ranking/relational+dbms



Limbajul SQL

- In acest capitol vom face o introducere in limbajul **Oracle SQL** utilizat pentru accesarea si administrarea unei baze de date Oracle. Comenzile si cererile SQL sunt folosite pentru :
 - Inserarea/extragerea/modificarea/stergerea randurilor intr-o tabela;
 - Crearea/modificarea/stergerea obiectelor din baza de date;
 - Controlul conexiunii si accesului la baza de date;
 - Prelucrarea datelor;
 - Relationarea datelor din mai multe tabele de date;
 - Formatarea datelor de iesire;
 - Introducerea criteriilor de cautare si sortare a datelor;
 - Refacerea starii bazei de date la un moment anterior;
 - Importul, exportul si replicarea datelor.



Limbajul SQL

- ✓ Setul de comenzi standard SQL de manipulare a datelor **DML** (*Data Manipulation Language*) :
 - SELECT – folosita pentru extragerea datelor din baza de date;
 - INSERT – folosita pentru inserarea datelor in baza de date;
 - UPDATE – folosita pentru modificarea datelor in baza de date;
 - DELETE – folosita pentru stergerea inregistrarilor.
- ✓ Setul de comenzi standard SQL pentru definirea datelor **DDL** (*Data Definition Language*):
 - CREATE – folosita pentru crearea unui obiect (tabela, view, index, etc.) in baza de date ;
 - ALTER – folosita pentru modificarea structurii unui obiect din baza de date;
 - DROP – folosita pentru stergerea unui obiect din baza de date.



Limbajul SQL

- ✓ Setul de comenzi standard SQL pentru crearea/revocarea drepturilor de acces:
- GRANT – folosita pentru a grantifica drepturile de acces la un obiect din baza de date;
- REVOKE – folosita pentru revocarea drepturilor de acces.
- Acestea sunt numai o parte a comenzilor SQL(lista completa se gaseste in *Manualul de Referinta al Limbajului SQL*).
- Cateva reguli de scriere a comenzilor SQL :
 - Comenzile se pot edita pe una sau mai multe linii;
 - Clauzele sunt uzual plasate pe linii separate, dar nu obligatoriu;
 - Cuvintele predefinite nu pot fi separate pe mai multe linii;
 - Comenzile nu sunt *case sensitive* (numai datele stocate in baza de date).



Operatori SQL

➤ Operatori de comparatie

Sunt operatori folositi pentru compararea valorilor coloanelor intre ele sau cu valori numerice si pot fi de doua feluri:

✓ Operatori logici

Operator	Semnificatie
-----	-----
=	egal cu
>	mai mare decat
>=	mai mare sau egal
<	mai mic decat
<=	mai mic sau egal



Operatori SQL

✓ Operatori SQL

Operator -----	Semnificatie -----
BETWEEN..AND...	intre doua valori(inclusiv)
IN(list)	compara cu o lista de valori
LIKE	compara cu un model de tip caracter
IS NULL	este o valoare nula



Operatori SQL

➤ Operatorii de negatie

Sunt operatori folositi pentru negarea valorilor coloanelor sau verificarea conditiilor de inegalitate si pot fi de doua feluri:

✓ Operatori logici

Operator	Semnificatie
-----	-----
!=	diferit de (VAX,UNIX,PC)
^=	diferit de (IBM)
<>	diferit de (toate OS)
NOT col_name =	diferit de
NOT col_name >	mai mic sau egal



Operatori SQL

✓ Operatori SQL

Operator	Semnificatie
-----	-----
NOT BETWEEN	nu se afla intre doua valori date
NOT IN	nu se afla intr-o lista data de valori
NOT LIKE	diferit de un sir
IS NOT NULL	nu este o valoare nula

- Folosind operatorul LIKE asociat cu simbolurile urmatoare, este posibil sa selectam randurile care se potrivesc cu un sir sau subsir de caractere :

<u>Simbol</u>	<u>Semnificatie</u>
%	orice secventa de mai multe caractere
_	un singur caracter



Operatori SQL

Observatii:

- Daca se compara o coloana sau expresie cu NULL, atunci operatorul de comparatie trebuie sa fie IS sau IS NOT. Daca se foloseste orice alt operator rezultatul va fi totdeauna FALSE (de exemplu expresia comision != NULL este intotdeauna falsa).
- Operatorii AND si OR pot fi utilizati pentru a compune expresii logice cu conditii multiple. Predicatul AND este adevarat numai daca ambele conditii sunt TRUE, iar predicatul OR este adevarat daca cel putin una dintre conditii este TRUE. Se pot combina AND sau OR in acceasi expresie logica in clauza WHERE, iar in acest caz operatorii AND sunt evaluati primii si apoi operatorii OR (deci operatorul AND are o precedenta mai mare decat OR).



Operatori SQL

- Daca operatorii au precedenta egala, atunci ei se evalueaza de la stanga la dreapta.
- Precedenta operatorilor logici este urmatoarea:
 1. Operatorii de comparatie si operatorii SQL au precedenta egala:

=, >= , <>, BETWEEN...AND, IN, LIKE, IS NULL.
 2. NOT - cand este folosit pentru a inversa rezultatul unei expresii logice (de exemplu `SELECT ....WHERE not(sal>2000) )`)
 3. AND
 4. OR.
- Pentru a fi siguri de ordinea de executie a doua operatii, se recomanda folosirea parantezelor rotunde pentru gruparea operatiilor.



Crearea unei tabele

➤ Comanda CREATE TABLE

- Este folosita pentru crearea unei tabele in baza de date. Aceasta comanda va fi prezentata in detaliu intr-un alt capitol.

Exemplu:

- Vom crea doua tabele pentru evidenta departamentelor si a angajatilor:

```
CREATE TABLE departamente
```

```
( id_dep      number(2)  not null,  
  den_dep     varchar2(30),  
  telefon     varchar2(10) );
```

```
CREATE TABLE angajati
```

```
( id_ang      number(4)  not null,  
  nume        varchar2(30),  
  functie     varchar2(20),  
  id_sef      number(4),  
  data_ang    date,  
  salariu     number(7,2),  
  comision    number(7,2),  
  id_dep      number(2) );
```



Inserarea datelor intr-o tabela

➤ Comanda INSERT

- Este folosita pentru inserarea datelor intr-o tabela si are urmatoarea sintaxa:

```
INSERT INTO [schema.] table_name[view_name][@dblink]  
            [column 1, column 2, ....]  
            VALUES (expr1, expr2. ....) subquery
```

unde :

- **schema** – este schema unde este creata tabela (specifica userul si baza de date);
- **table_name** – este numele tablei;
- **view_name** – este numele unui view creat pe o tabela;
- **column** – este numele coloanei;
- **expr** – reprezinta valoarea aferenta coloanei;
- **subquery** – este o subcerere care returneaza linii cu date din una sau mai multe tabele.



Inserarea datelor intr-o tabela

Exemple:

- Sa facem o inserare completa (toate coloanele au valori nenule) in tabelele create anterior. In acest caz nu trebuie sa specificam numele coloanelor dar valorile trebuie specificate in clauza VALUES in ordinea de creare a coloanelor in tabela.

```
SQL> INSERT INTO departamente VALUES (50, 'Proiectare Software', '0213262031');
```

```
SQL> INSERT INTO angajati VALUES (111, 'Popa Daniela', 'Inginer', 777, '1-OCT-2019', 750, 100, 10);
```

- Daca facem inserari numai in anumite coloane comanda arata astfel:

```
SQL> INSERT INTO angajati (id_ang,nume,functie) VALUES (777, 'Petrescu Florin', 'Contabil');
```

- Trebuie mentionat ca in acest caz trebuie specificate toate coloanele care fac parte dintr-o cheie primara sau sunt declarate NOT NULL la crearea tabelului, altfel se va genera un cod de eroare.



Inserarea datelor intr-o tabela

- Urmatoarea comanda va genera o eroare deoarece nu se specifica valoare pentru coloana id_ang care este declarata not null:

```
SQL> INSERT INTO angajati (nume, functie, salariu) VALUES ('Tache Marius', 'Tehnician', 1300);
```

ERROR at line 1:

ORA-01400: cannot insert NULL into ("SCOTT"."ANGAJATI"."ID_ANG")

- Nu se accepta inregistrari cu valori care depasesc dimensiunea coloanei. Urmatoarea comanda va genera o eroare deoarece se specifica o valoare prea mare pentru id_dep:

```
SQL> INSERT INTO departamente VALUES (222, 'Contabilitate', '0213262032');  
*
```

ERROR at line 1:

ORA-01438: value larger than specified precision allowed for this column



Stergerea datelor dintr-o tabela

➤ Comanda DELETE

- Este folosita pentru stergerea liniilor dintr-o tabela sau view si are urmatoarea sintaxa:

```
DELETE FROM [schema.]  
           table_name[view_name][@dblink]  
WHERE condition  
           (column1,column2, ..) IN [ NOT IN] subquery
```

unde :

- **schema** – este schema unde este creata tabela (specifica userul si baza de date);
- **table_name** – este numele tablei;
- **view_name** – este numele unui view creat pe o tabela;
- **condition** – conditia care trebuie indeplinita pentru liniile sterse;
- **column** – este numele coloanei;
- **subquery** – este o subcerere care returneaza linii cu date din una sau mai multe tabele.



Stergerea datelor dintr-o tabela

Exemple:

- Stergerea unui angajat din tabela angajati se face cu comanda:
SQL> DELETE FROM angajati WHERE nume='POPA DANIELA' ;
- Stergerea datelor tuturor angajatilor care au venit in companie inainte de anul 1981 se face cu comanda:

- **SQL> DELETE FROM angajati WHERE data_ang < '1-JAN-1981';**
- Pentru a sterge datele pentru toti angajatii care s-au angajat in luna Noiembrie, indiferent de an, folosim comanda:

SQL> DELETE FROM angajati WHERE data_ang LIKE '%NOV%';

- Stergerea datelor angajatilor care nu au sef (id_sef este null) se face astfel:

SQL> DELETE FROM angajati WHERE id_sef is null;

- Pentru a sterge toate liniile din tabela angajati folosim comanda:

SQL> DELETE FROM angajati;

- Refacerea datelor sterse accidental se face cu comanda ROLLBACK:

SQL> ROLLBACK;



Modificarea datelor dintr-o tabela

➤ Comanda UPDATE

Este folosita pentru modificarea datelor in baza de date si are urmatoarea sintaxa:

UPDATE [schema.] table_name[view_name][@dblink]

SET column=expr

column=subquery

(column1,column2,...) IN [NOT IN] subquery

WHERE condition

unde :

- **schema** – este schema unde este creata tabela (specifica userul si baza de date);
- **table_name** – este numele tablei;
- **column** – este numele coloanei;
- **condition** – conditia pentru modificarea liniilor;
- **subquery** – este o subcerere care returneaza linii cu date din una sau mai multe tabele.



Modificarea datelor dintr-o tabela

Exemple:

- Modificarea salariului si comisionului unui angajat se face astfel:

```
SQL> UPDATE angajati SET salariu=1200, comision=100 WHERE  
nume='IONESCU VICTOR';
```

```
SQL> UPDATE angajati SET salariu=1000 WHERE id_ang=7369;
```

- Modificarea tuturor salariilor prin indexare cu 10%:

```
SQL> UPDATE angajati SET salariu=salariu*1.1;
```

- Daca dorim sa crestem salariile doar pentru angajatii din departamentul 10 folosim comanda:

```
SQL> UPDATE angajati SET salariu=salariu*1.1 WHERE id_dep=10;
```

- Acordarea unui comision pentru angajatii veniti in companie in anul 1981 se face astfel:

```
SQL> UPDATE angajati SET comision=0.1*salariu WHERE  
data_ang>'1-JAN-1981' AND data_ang<'31-DEC-1981';
```

- In absenta clauzei WHERE toate liniile vor fi actualizate.



Modificarea datelor dintr-o tabela

- Cand se face actualizarea datelor intr-o tabela se verifica automat si constrangerile de integritate definite pe tabela respectiva, altfel comanda genereaza un cod de eroare si tranzactia esueaza.
- Situatiile in care pot sa apara erori sunt:
 - Noile valori fac duplicare de cheie primara sau unica;
 - Actualizarea valorii cu o valoare nula cand coloana este NOT NULL;
 - Valorile noi nu respecta o constrangere CHECK;
 - Valorile noi nu respecta o constrangere FOREIGN KEY;
 - Valorile vechi erau referite de alte tabele printr-o constrangere FOREIGN KEY;



Vizualizarea datelor dintr-o tabela

➤ Comanda **SELECT**

- Este folosita pentru vizualizarea datelor dintr-o tabela. Aceasta comanda va fi prezentata in detaliu intr-un alt capitol.

Exemple:

```
SQL> SELECT * FROM angajati;
```

```
SQL> SELECT nume, functie, salariu FROM angajati;
```

```
SQL> SELECT nume, functie, salariu, comision FROM angajati WHERE  
id_dep=10;
```

```
SQL> SELECT * FROM angajati WHERE functie='DIRECTOR';
```

- Datele din baza de date sunt case sensitive. Urmatoarea comanda nu va returna niciun rand, cu toate ca exista angajati cu functia DIRECTOR:

```
SQL> SELECT * FROM angajati WHERE functie='Director';
```

no rows selected



Capitolul 7

Cereri de interogare SQL



Cereri de interogare SQL*Plus

- Cererile de interogare SQL folosesc in exclusivitate comanda **SELECT**, fiind utilizate atat pentru interogarea obiectelor create de utilizatori cat si a celor sistem. Sintaxa comenzii este urmatoarea :

```
SELECT [DISTINCT,ALL] [schema.table.]expresion expr_alias  
FROM [schema.table@dblink] table_alias  
[WHERE condition]  
[START WITH condition][CONNECT BY condition]  
[UNION,UNION ALL,INTERSECT,MINUS][SELECT command]  
[GROUP BY expresion][ HAVING condition]  
[ORDER BY expresion(position)] [ASC,DESC]  
[FOR UPDATE OF schema.table.column] [NOWAIT]
```



Cereri de interogare SQL*Plus

- Parametrii comenzii au urmatoarea semnificatie(cei din paranteze sunt optionali):
- ***DISTINCT*** - returneaza o singura linie in cazul in care cererea returneaza linii duplicate;
- ***ALL*** – returneaza toate liniile simple si duplicate;
- ***schema.table*** – reprezinta shema de identificare a tablei(sau view-lui) specificata prin *user.table_name*;
- ***expresion*** – reprezinta un nume de coloana sau o expresie care poate folosi functii sistem (* selecteaza toate coloanele tabelor din clauza FROM);
- ***expr_alias*** – este un nume alocat unei expresii care va fi folosit in formatarea coloanei (apare in antetul listei);
- ***dblink*** – reprezinta numele complet sau partial de identificare a unei baze de date (database.domain@connection_qualifier)



Cereri de interogare SQL*Plus

- ***table_alias*** – este un nume alocat unei tabele(view) care va fi folosit in cereri corelate;
- ***WHERE condition*** – reprezinta o clauza (inlantuire de conditii) care trebuie sa fie indeplinita in criteriul de selectie a liniilor;
- ***START WITH condition*** – stabileste criteriul de selectie pentru prima linie;
- ***CONNECT BY condition*** – stabileste o ierarhie de selectie a liniilor;
- ***GROUP BY expresion*** – stabileste criteriile de grupare a liniilor(numele coloanelor folosite in criteriul de grupare);
- ***HAVING condition*** – restrictioneaza liniile din grup la anumite conditii;



Cereri de interogare SQL*Plus

- ***UNION, UNION ALL, INTERSECT, MINUS*** – face operatii pe multimi de linii selectate de mai multe comenzi *SELECT* prin aplicarea anumitor restrictii;
- ***ORDER BY expresion(position)*** – ordoneaza liniile selectate dupa coloanele din expresie sau in ordinea coloanelor specificate prin pozitie;
- ***FOR UPDATE OF*** – face o blocare (*lock*) a liniilor in vederea modificarii anumitor coloane;
- ***NOWAIT*** – returneaza controlul userului daca comanda asteapta eliberarea unei linii blocata de un alt user.



Cereri simple de interogare

1. Cereri simple

- Cererea urmatoare returneaza toate coloanele si toate inregistrarile continute de o tabela:

```
SQL> SELECT * FROM angajati;
```

- Interogari pe anumite coloane ale unei tabele:

```
SQL> SELECT id_dep, den_dep  
FROM departamente;
```

- Interogari care returneaza calcule pe anumite coloane si asigneaza un alias:

```
SQL> SELECT id_ang ecuson, nume,  
salariu*12+nvl(comision,0) venit_anual  
FROM angajati;
```



Cereri simple de interogare

- Interogari care concateneaza anumite coloane:

```
SQL> SELECT id_ang || '-' || nume angajat , functie, data_ang  
FROM angajati;
```

- Interogari care adauga coloane noi in lista:

```
SQL> SELECT id_ang || '-' || nume angajat , functie,  
salariu*12+nvl(comision,0) venit_lunar, ' ' semnatura  
FROM angajati;
```



Cereri cu clauza WHERE

2. Cereri cu clauza WHERE

- Clauza WHERE poate compara valori de coloana ,valori literale, expresii aritmetice (sau functii) si poate avea patru tipuri de parametri:
 - nume de coloana;
 - operator de comparatie;
 - operator de negatie;
 - lista de valori.



Cereri cu clauza WHERE

- Lista persoanelor angajate in anul 1980:

```
SQL> SELECT id_ang ecuson, nume, functie, data_ang  
        FROM angajati  
        WHERE data_ang LIKE '%80';
```

- Lista persoanelor al caror nume incepe cu litera **F** si au numele functiei pe 7 caractere :

```
SQL> SELECT id_ang ecuson, nume, functie, data_ang  
        FROM angajati  
        WHERE nume LIKE 'F%' and functie LIKE '_____';
```



Cereri cu clauza WHERE

- Lista angajatilor din departamentul 20 care nu au primit comision:

```
SQL> SELECT id_ang ecuson, nume, functie, salariu FROM angajati  
WHERE (comision=0 OR comision IS NULL) AND id_dep=20  
ORDER BY nume;
```

- Lista angajatilor care au primit comision si nu sunt directori:

```
SQL> SELECT id_ang ecuson, nume, functie, salariu,comision  
FROM angajati  
WHERE (comision IS NOT NULL and comision<>0)  
and functie NOT LIKE 'DIRECTOR'  
ORDER BY nume;
```



Cereri cu variabile substituite

3. Cereri cu variabile substituite

- Cererile SQL pot fi executate folosind anumiti parametri, care se mai numesc si variabile substituite(sau de substitutie).

➤ Variabile ampersand (&)

O astfel de variabila se defineste sub forma **&nume_variabila** si este un parametru care se va introduce de la tastatura in timpul executiei comenzii in care este utilizata. Parametrul cu un singur ampersand trebuie introdus de fiecare data, chiar daca este folosit de mai ori in aceeasi comanda SQL.

Exemplu:

```
SQL> SELECT id_ang,nume,functie,salariu FROM angajati  
      WHERE id_sef=&ecuson_sef;
```

Enter value for ecuson_sef: 7698

- old 1: SELECT id_ang,nume,functie,salariu FROM angajati WHERE id_sef=&ecuson_sef
- new 1: SELECT id_ang,nume,functie, salariu FROM angajati WHERE id_sef=7698;



Cereri cu variabile substituite

➤ Variabile dublu ampersand (&&)

- Spre deosebire de variabila cu un singur ampersand, o variabila cu dublu ampersand va fi stocata si va putea fi apelata pe toata sesiunea de lucru.
- Definirea se face similar **&&nume_variabila** si va fi ceruta o singura data, folosirea ei de mai multe ori in cadrul comenzii se face apeland-o cu **&nume_variabila**.

Exemplu:

```
SQL> SELECT nume,functie,salariu,comision, &&venit venit_lunar  
      FROM angajati WHERE &venit>&val_venit;
```

Enter value for venit: salariu+nvl(comision,0)

- Pentru a reseta o variabila cu dublu ampersand se utilizeaza comanda UNDEFINE :

```
SQL> UNDEFINE venit
```



Cereri cu variabile substituite

➤ Variabile de sistem (&n)

- Sunt variabile definite numeric (1-9) care sunt predefinite de catre sistem si care functioneaza similar cu variabilele cu dublu ampersand. Avantajul folosirii acestor variabile este ca pot fi apelate direct dintr-un fisier de comenzi indirecte, fara a fi definite in prealabil. Suporta valori numerice, alfanumerice si data calendaristica.

Exemplu:

```
SQL> SELECT id_ang,nume,functie,data_ang FROM angajati  
        WHERE functie='&1' and data_ang>'&2'  
        ORDER BY data_ang;
```

```
SQL> SAVE angajari.sql
```

Created file angajari.sql

```
SQL> START angajari PROGRAMATOR 15-AUG-1981
```

```
SQL> UNDEFINE 1
```



Cereri definite cu ACCEPT

➤ Variabile definite cu ACCEPT

- Cand definim o variabila cu ampersand, totdeauna promptul va fi numele variabilei.
- Folosind comanda ACCEPT, se poate redefini promptul si chiar se pot ascunde caracterele introduse de la tastatura(facilitate utila in cazul unei parole).

Exemplu:

```
Se vor edita urmatoarele comenzi in fisierul c:\temp\functia.sql :  
ACCEPT functie_sef CHAR PROMPT 'Introduceti functia sefului:' ;  
SELECT nume,functie, salariu,comision FROM angajati  
WHERE functie='&functie_sef';  
UNDEFINE functie_sef
```

Fisierul se va rula in SQL*Plus si se va introduce functia sefului:

```
SQL> @c:\temp\functia.sql
```

Introduceti functia sefului: DIRECTOR



Cereri definite cu DEFINE

➤ Variabile definite cu DEFINE si resetate cu UNDEFINE

- Variabilele definite cu DEFINE nu vor mai afisa promptul atunci cand sunt setate si raman setate pana cand vor fi resetate cu comanda UNDEFINE.

Exemple:

```
SQL> DEFINE procent_prima= 1.15
```

```
SQL> SELECT nume, salariu, salariu*&procent_prima prima  
FROM angajati WHERE id_dep=20;
```

```
SQL> DEFINE venit= salariu+nvl(comision,0)
```

```
SQL> SELECT nume, data_ang, &venit venit_lunar FROM angajati  
WHERE functie='DIRECTOR';
```

```
SQL> UNDEFINE procent_prima
```

```
SQL> UNDEFINE venit
```



Cereri definite cu DEFINE

➤ Variabile folosite in operatii DDL si DML

- Variabilele substituite pot fi folosite si in alte operatii DDL sau DML.

Exemple:

```
SQL> CREATE TABLE angajati_10 as SELECT nume, functie, data_ang,  
salariu FROM angajati WHERE id_dep=&depart and functie='&job';
```

```
SQL> INSERT INTO angajati_10 SELECT nume, functie, data_ang, salariu  
FROM angajati WHERE functie='&job';
```

```
SQL> UPDATE angajati_10 SET salariu= &salariu_nou WHERE  
functie='&job';
```

```
SQL> DELETE FROM angajati_10 where functie='&job' and  
data_ang='&data_angajare';
```




Cereri de interogare SQL*Plus

Exercitii:

1. Sa se faca o lista cu toti angajatii care s-au angajat inainte de anul 1982 si nu au primit comision.

a)

```
SQL> SELECT * FROM angajati  
        WHERE data_ang<'1-JAN-1982' and  
              (comision is null or comision =0);
```

b)

```
SQL> SELECT * FROM angajati  
        WHERE data_ang<'1-JAN-1982' and  nvl(comision,0)=0;
```



Cereri de interogare SQL*Plus

2. Sa se faca o lista cu toti angajatii care au salariul peste 3000 \$ si nu au sefi, ordonati dupa departamente si nume.

```
SQL> SELECT * FROM angajati  
      WHERE salariu>3000 and id_sef is null  
      ORDER BY id_dep, nume;
```

3. Sa se faca o lista cu numele, functia si venitul anual al angajatilor dintr-un departament introdus de la tastatura si care nu sunt directori.
a)

```
SQL> SELECT nume, functie, salariu*12+nvl(comision,0) venit_anual  
      FROM angajati  
      WHERE functie not like 'DIRECTOR' and id_dep=&nr_depart;
```



Cereri de interogare SQL*Plus

b)

```
SQL> DEFINE venit_an='salariu*12+nvl(comision,0)';
```

```
SQL> SELECT nume,functie,&venit_an venit_anual FROM angajati  
WHERE functie not like 'DIRECTOR' and id_dep=&nr_depart;
```

```
SQL> UNDEFINE venit_an ;
```

4. Sa se faca o lista cu departamentul, numele, data angajarii si salariul tuturor persoanelor angajate in anul 1981, din doua departamente pentru care id_dep se introduce de la tastatura .

a)

```
SQL> SELECT id_dep,nume,data_ang,salariu FROM angajati  
WHERE data_ang like '%81' and  
(id_dep=&nr_depart1 or id_dep=&nr_depart2)  
ORDER BY id_dep;
```



Cereri de interogare SQL*Plus

b)

```
SQL> SELECT id_dep, nume, data_ang, salariu FROM angajati  
WHERE data_ang like '%81' and  
      (id_dep=&&nr_depart1 or id_dep=&&nr_depart2)  
ORDER BY id_dep;  
SQL> UNDEFINE nr_depart1 ;  
SQL> UNDEFINE nr_depart2 ;
```



Cereri de interogare SQL*Plus

5. Sa se scrie o cerere SQL care face o lista cu toti angajatii dintr-un departament (introdus ca parametru de la tastura) care au venitul anual peste un venit mediu anual (introdus tot de la tastatura). Lista este ordonata dupa ID_ang si are antetul:

ID_dep	ID_ang	Nume_Functie	Salariu	Comision	Venit_anual
--------	--------	--------------	---------	----------	-------------

Sa se rezolve prin mai multe metode.

a)

```
SQL> SELECT id_dep, id_ang, nume || '-' || functie
          nume_functie, salariu, comision,
          salariu*12+nvl(comision,0) venit_anual
FROM angajati
WHERE id_dep=&departament and
      (salariu*12+nvl(comision,0))> &venit_mediu
ORDER BY id_ang;
```



Cereri de interogare SQL*Plus

b)

```
SQL> DEFINE venit_an='(salariu*12+nvl(comision,0))';  
SQL> SELECT id_dep, id_ang, nume || '-' || functie nume_functie,  
           salariu, comision,  
           salariu*12+nvl(comision,0) venit_anual  
FROM angajati  
WHERE id_dep=&departament and  
      &venit_an > &venit_mediu  
ORDER BY id_ang;  
SQL> UNDEFINE venit_an;
```



Cereri de interogare SQL*Plus

c)

```
SQL> DEFINE venit_an='&formula_venit'
```

```
SQL> DEFINE venit_mediu = 10000
```

```
SQL> SELECT &id_dep, id_ang, nume || '-' || functie nume_functie,  
           salariu, comision,  
           &venit_an venit_anual
```

```
FROM angajati
```

```
WHERE id_dep=&id_dep and &venit_an > &venit_mediu
```

```
ORDER BY id_ang;
```

```
SQL> UNDEFINE venit_an ;
```

```
SQL> UNDEFINE venit_mediu ;
```

```
SQL> UNDEFINE id_dep ;
```



Cereri de interogare SQL*Plus

d)

```
SQL> DEFINE venit_an= '(salariu*12+nvl(comision,0))';
SQL> ACCEPT depart number PROMPT 'Departament: ';
SQL> ACCEPT val_venit number PROMPT 'Venit anual: ';
SQL> SELECT id_dep, id_ang, nume || '-' || functie nume_functie,
           salariu, comision,
           salariu*12+nvl(comision,0) venit_anual
FROM angajati
WHERE id_dep=&depart and &venit_an > &venit_mediu;
SQL> UNDEFINE venit_an ;
SQL> UNDEFINE val_venit ;
SQL> UNDEFINE depart ;
```




Cereri de interogare SQL*Plus

e) Se editeaza un fisier si se salveaza cu numele venit_anual.txt:

```
DEFINE venit_an= '(salariu*12+nvl(comision,0))';  
ACCEPT depart number PROMPT 'ID_departament:';  
ACCEPT val_venit number PROMPT 'Venit anual:';  
SELECT id_dep, id_ang, nume || '-' || functie nume_functie,  
        salariu, comision, &venit_an venit_anual  
FROM angajati  
WHERE id_dep=&depart and &venit_an> &val_venit;  
ORDER BY id_dep;  
UNDEFINE venit_an ;  
UNDEFINE val_venit ;  
UNDEFINE depart ;
```

Se executa scriptul: **SQL>@c:\Lab_BD\venit_anual.txt**



Capitolul 8

Metode de JOIN



Metode de JOIN

- Pentru extragerea datelor din mai multe tabele din baza de date, comanda SELECT foloseste una sau mai multe metode de JOIN. Sintaxa pentru un join simplu este urmatoarea:

```
SELECT [DISTINCT,ALL] [table].expresion expr_alias  
FROM [schema.table1] table1_alias,  
      [schema.table2] table2_alias,  
WHERE table1_alias.column=table2_alias.column  
ORDER BY expresion(position)] [ASC,DESC]
```



Metode de JOIN

- Parametrii comenzii au urmatoarea semnificatie(cei din paranteze sunt optionalii):
 - ***DISTINCT*** - returneaza o linie in cazul in care cererea returneaza linii duplicate;
 - ***ALL*** – returneaza toate liniile simple si duplicate;
 - ***schema.table*** – reprezinta shema de identificare a tablei(sau view-lui) specificata prin *user.table_name*;
 - ***expresion*** – reprezinta un nume de coloana sau o expresie care poate folosi functii sistem (* selecteaza toate coloanele tabelului din clauza FROM);
 - ***expr_alias*** – este un nume alocat unei expresii care va fi folosit in formatarea coloanei (apare in antetul listei);



Metode de JOIN

- ***table_alias*** – este un nume alocat unei tabele(sau view) care va fi folosit in cereri corelate;
- ***WHERE condition*** – reprezinta o clauza (inlantuire de conditii) care trebuie sa fie indeplinita in criteriul de selectie a liniilor;
- ***ORDER BY expresion(position)*** – ordoneaza liniile selectate dupa coloanele din expresie sau in ordinea coloanelor specificate prin pozitie.



Equi JOIN

➤ Equi-join

- Daca in conditia de join apar numai egalitati, avem de-a face cu un *equi-join*. Pentru a putea sa realizam un join pe mai multe tabele, este obligatoriu ca ele sa contina coloane de acelasi tip, cu date comune sau corelate.

Exemplu:

- Pentru a lista angajatii dintr-un departament folosim tabela *angajati*, insa in aceasta tabela gasim asociat fiecarui angajat un `id_dep`, iar denumirea departamentului respectiv o gasim in tabela *departamente*.
- Se impune un join intre cele doua tabele, pentru ca in lista sa apara si denumirea departamentului.



Equi JOIN

```
SQL> SELECT a.id_dep ,b.den_dep depart,  
a.nume, a.functie  
FROM angajati a, departamente b  
WHERE a.id_dep=b.id_dep and  
a.id_dep=10;
```

- Se observa ca au fost folosite aliasuri pentru tabele pentru a nu crea ambiguitate cand referim coloane cu aceeasi denumire.



Non Equi JOIN

➤ Non Equi-join

- Se foloseste cand in conditia de join avem alti operatori in afara de operatorul de egalitate sau cand doua sau mai multe tabele nu au coloane comune, dar trebuie totusi relateate.

Exemple:

- Relatia dintre tabelele *angajati* si *grila_salariu* este un non-equi-join, in care nicio coloana din prima tabela nu corespunde direct cu o coloana din cealalta.
- Relatia se obtine folosind tot un operator, altul decat = , de exemplu *between*.
- Pentru a evalua gradul de salarizare al unui angajat, trebuie sa consultam grila de salarizare pentru a identifica in ce plaja de salariu se incadreaza salariul:



Non Equi JOIN

```
SQL> SELECT a.numa, a.salariu, b.grad  
FROM angajati a, grila_salariu b  
WHERE a.salariu BETWEEN b.nivel_inf  
AND b.nivel_sup AND a.id_dep=20;
```

- Sunt situatii cand trebuie sa folosim si equi-join si non equi-join intr-o cerere . In exemplul anterior, daca dorim sa listam si departamentul, va trebui sa facem join intre trei tabele:

```
SQL> SELECT c.den_dep,a.numa, a.salariu, b.grad  
FROM angajati a, grila_salariu b, departamente c  
WHERE a.salariu BETWEEN b.nivel_inf AND  
b.nivel_sup AND a.id_dep=c.id_dep AND  
a.id_dep=20;
```



Self JOIN

➤ Joinul unei tabele cu ea insasi (*self join*)

- Sunt situatii cand avem nevoie sa extragem date corelate din aceeasi tabela. De exemplu, daca dorim sa afisam care sunt sefii angajatilor trebuie sa extragem din tabela *angajati* si numele sefului (*id_sef*).

```
SQL> SELECT a.numa nume_ang, a.functie functie_ang,  
          b.numa nume_sef, b.functie functie_sef  
FROM angajati a, angajati b  
WHERE a.id_sef=b.id_ang AND a.id_dep=10;
```



Cross JOIN

➤ **Produs cartezian**

- Produsul cartezian a doua tabele se obtine prin concatenarea fiecărei linii dintr-o tabela cu fiecare linie din cealalta, rezultand un numar de linii egal cu produsul numarului de linii din fiecare tabela. Aceasta situatie este mai putin practica si se intalneste, de regula, cand sunt puse gresit conditii in clauza WHERE.

Exemplu:

```
SQL> SELECT nume ,functie ,den_dep  
        FROM angajati , departamente  
        WHERE functie='DIRECTOR';
```



Outer JOIN

➤ Join extern (*outer join*)

- Folosind equi-join putem selecta toate liniile care indeplinesc conditiile din clauza WHERE. Apar situatii cand cererea trebuie sa selecteze si linii care nu indeplinesc toate conditiile din clauza.

Exemple:

- Sa construim structura organizatorica a firmei selectand toate departamentele si angajatii care fac parte din fiecare departament. Exista, in tabela departamente, departamentul 40 care nu are niciun angajat si folosind equi-join acesta nu apare in lista. Pentru a depasi situatia se foloseste un *join extern* (+) asa cum se vede mai jos:



Outer JOIN

```
SQL> SELECT a.id_dep, a.den_dep, b.nume, b.functie  
      FROM departamente a, angajati b  
      WHERE a.id_dep=b.id_dep(+);
```

- In lista va apare si departamentul 40 care nu are niciun angajat. Totdeauna semnul (+) se pune in dreptul tabelii deficitare ca informatii.
- Putem sa folosim si operatorul *BETWEEN* intr-un join extern. Daca vrem sa aflam care angajati raman in grila de salarizare prin triplarea salariilor, executam urmatoarea cerere:



Outer JOIN

```
SQL> SELECT c.den_dep, a.nume, a.salariu*3 salariu_nou,  
a.comision, b.grad  
FROM angajati a, grila_salariu b, departamente c  
WHERE a.salariu*3+ nvl(a.comision,0)  
BETWEEN b.nivel_inf(+) AND b.nivel_sup(+) AND  
a.id_dep=c.id_dep ;
```

- Daca dorim sa selectam toate departamentele care au primul caracter din id_dep din tabela departamente folosit printre id_dep din tabela angajati, se poate folosi si operatorul *LIKE* :

```
SQL> SELECT a.id_dep,a.den_dep,b.nume,b.functie, b.id_dep  
FROM departamente a, angajati b  
WHERE b.id_dep(+) LIKE substr(a.id_dep,1,1)||'%';
```



Vertical JOIN

➤ Join vertical (*vertical join*)

- Join-ul vertical este folosit pentru prelucrarea liniilor returnate de mai multe cereri *SELECT* si foloseste operatorii ***UNION*** (reuniune), ***INTERSECT*** (intersectie), ***MINUS*** (diferenta). In acest caz, join-ul se face dupa coloane de acelasi tip, nu dupa randuri, de aceea se mai numeste si join vertical.

Exemple:

- Daca dorim lista angajatilor din departamentele 10 si 30, se poate utiliza cererea urmatoare:



Vertical JOIN

```
SQL> SELECT id_dep,nume,functie,salariu  
      FROM angajati WHERE id_dep=10  
  
UNION  
  
      SELECT id_dep,nume,functie,salariu  
      FROM angajati WHERE id_dep=30;
```

- Trebuie retinut ca reuniunea se poate face pe coloane declarate de acelasi tip (number, varchar, date), chiar daca au semnificatii diferite. Sa construim o cerere care reuneste pe aceeasi coloana salariile angajatilor din departamentul 10 si cu comisioanele celor din departamentul 30.



Vertical JOIN

```
SQL> SELECT id_dep,nume,functie,'are salariu'  
       salariu_com, salariu sal_com  
       FROM angajati WHERE id_dep=10  
  
UNION  
  
SELECT id_dep,nume,functie,'are comision ',  
       comision FROM angajati WHERE id_dep=30;
```

Obs. Primul SELECT impune aliasurile de coloane.

- Folosind operatorul *UNION ALL* se selecteaza si liniile duplicate:

```
SQL> SELECT functie FROM angajati WHERE id_dep=10  
  
UNION ALL  
  
SELECT functie FROM angajati WHERE id_dep=20;
```



Vertical JOIN

- Operatorul *INTERSECT* este folosit pentru a selecta liniile comune. Daca dorim sa aflam care sunt functiile care nu au primit comision si care se regasesc in toate departamentele, scriem urmatoarea cerere:

```
SQL> SELECT functie, comision FROM angajati  
      WHERE id_dep=10 and nvl(comision,0)=0  
      INTERSECT  
      SELECT functie,comision FROM angajati  
      WHERE id_dep=20 and nvl(comision,0)=0  
      INTERSECT  
      SELECT functie,comision FROM angajati  
      WHERE id_dep=30 and nvl(comision,0)=0;
```



Vertical JOIN

- Pentru a afla care sunt functiile din departamentul 10 care nu se regasesc in departamentul 30, folosim operatorul *MINUS* :

```
SQL> SELECT functie FROM angajati WHERE id_dep = 10  
MINUS  
SELECT functie FROM angajati WHERE id_dep = 30;
```



Reguli de JOIN

- Observatii:
 - Atunci cand conditia de join lipseste, fiecare linie a unei tabele din lista FROM este asociata cu fiecare linie a celorlalte tabele, obtinandu-se de fapt **produsul cartezian** al acestora.
 - Daca in conditia de join apar numai egalitati operatia este numita si **equi-join**. In celelalte cazuri avem un **non-equi-join**.
 - In lista de tabele care participa la join o tabela poate sa apara repetat. O astfel de operatie este numita si **joinul unei tabele cu ea insasi (self-join)**.
 - In cazul in care o linie a unei tabele nu se coreleaza prin conditia de join cu nicio linie din celelalte tabele ea nu va participa la formarea rezultatului. Se poate insa obtine ca aceasta sa fie luata in considerare pentru rezultat, folosind un **join extern (outer join)**.



Reguli de JOIN

- In cazul general al unui **join pe N tabele, conditia de join este compusa din N - 1 subconditii** conectate prin AND care relationeaza intreg ansamblul de tabele. Altfel spus, daca se construiesc un graf al conditiei in care nodurile sunt tabele si arcele subconditii de join care leaga doua tabele, atunci acest graf trebuie sa fie conex.
- Tipul de **join extern se poate folosi si atunci cand conditia de join este compusa**, cu exceptia cazului in care se foloseste OR sau operatorul de incluziune IN urmat de o lista care contine mai mult de o valoare.
- Joinul extern **se poate folosi si in conjunctie cu operatorii specifici SQL.**



Metode de JOIN in SQL-3

- Pana la versiunea Oracle 9i sintaxa joinului in Oracle era diferita de standardul ANSI (*American National Standards Institute*). Incepand cu aceasta versiune au fost introduse in limbaj si tipurile de join din standardul SQL-3(anul 1999) printre care ***cross-join, natural join*** si mai multe variante de ***outer join***:
 - **CROSS JOIN** – join pe produs cartezian
 - **[INNER] JOIN ... USING** – join pe coloane comune
 - **[INNER] JOIN ... ON** – join general
 - **NATURAL JOIN** – join natural
 - **OUTER JOIN ... ON** – join extern
- In clauza FROM avem perechi de tabele care participa la join.



Cross JOIN in SQL-3

➤ CROSS JOIN

- Este folosit pentru obtinerea produsul cartezian si are urmatoarea sintaxa:

```
SELECT [DISTINCT | ALL]  
[[table | table_alias].]{column | expression}  
[column_alias]  
FROM  
[schema.]table1 [table1_alias] CROSS JOIN  
[schema.]table2 [table2_alias]  
[other clauses]
```



Cross JOIN in SQL-3

Exemple:

```
SQL> SELECT * FROM angajati a  
      CROSS JOIN departamente b;
```

```
SQL> SELECT a.nume, b.den_dep FROM angajati a  
      CROSS JOIN departamente b;
```

- Pentru a face JOIN si CROSS JOIN adaugam conditia de join in clauza WHERE:

```
SQL> SELECT a.nume, a.data_ang, b.den_dep, b.telefon  
      FROM angajati a  
      CROSS JOIN departamente b  
      WHERE a.id_dep=b.id_dep;
```

- Observatie: Conditia CROSS JOIN poate sa lipseasca in acest caz, rezultatul va fi acelasi.



Inner JOIN in SQL-3

➤ JOIN ... USING

- Este un equi-join dupa coloane cu acelasi nume specificate in USING (dar nu toate). Sintaxa este urmatoarea:

```
SELECT [DISTINCT|ALL]  
{table|table_alias}. {column|expression}  
[column_alias]  
FROM  
[schema.]table1 [table1_alias]  
[INNER] JOIN [schema.]table2 [table2_alias]  
USING (column 1, column 2...)  
[other clauses]
```



Inner JOIN in SQL-3

Exemple:

- Deoarece in ANGAJATI si DEPARTAMENTE avem o coloana cu acelasi nume (ID_DEP)) putem face un equi-join astfel:
**SQL> SELECT id_dep, den_dep, nume, data_ang, telefon
FROM angajati
INNER JOIN departamente USING (id_dep);**
- Dupa cum se observa in lista USING numele coloanelor dupa care se efectueaza joinul nu trebuie prefixat cu numele sau aliasul vreuneia dintre tabele (coloanele comune se afiseaza o singura data).
- Daca se doreste selectia doar dintr-un departament se va adauga conditia suplimentara in WHERE:

```
SQL> SELECT den_dep, nume, functie, data_ang, telefon  
FROM angajati  
JOIN departamente USING (id_dep)  
WHERE den_dep='CONTABILITATE';
```



Inner JOIN in SQL-3

➤ JOIN .. ON

- Prin aceasta clauza se implementeaza un join general. Conditia de join (si eventual si conditiile suplimentare) se pun in clauza ON. Sintaxa este urmatoarea:

```
SELECT [DISTINCT|ALL]  
{{table|table_alias}.}{column|expression} [column_alias]  
FROM  
[schema.]table1 [table1_alias]  
[INNER] JOIN [schema.]table2 [table2_alias]  
ON  {{table1|table1_alias}.}column_fromTable1 =  
    {{table2|table2_alias}.}column_fromTable2  
[other clauses]
```



Inner JOIN in SQL-3

Exemple:

- Cererea urmatoare efectueaza joinul dupa coloana ID_DEP si contine si o conditie suplimentara(filtru) dupa functia DIRECTOR:

```
SQL> SELECT a.nume, a.data_ang, b.den_dep  
        FROM angajati a JOIN departamente b  
        ON (a.id_dep=b.id_dep AND a.functie='DIRECTOR');
```

Conditia suplimentara se putea pune si in clauza WHERE.

- Intr-un JOIN ON se poate folosi si non-equi-join. Daca se doreste o lista a angajatilor cu salariul intre 1000 si 3000 se poate introduce urmatorul filtru:

```
SQL> SELECT a.nume, a.data_ang, a.salariu, b.den_dep  
        FROM angajati a JOIN departamente b  
        ON ( a.id_dep=b.id_dep AND  
            a.salariu BETWEEN 1000 AND 3000);
```



Inner JOIN in SQL-3

- Observatie: Un join general se poate aplica pe mai multe tabele.

Exemplu:

- Cererea urmatoare efectueaza joinul tabelei de angajati cu tabelele de departamente si grila de salarii:

```
SQL> SELECT b.den_dep , a.numa, a.salariu, c.grad  
        FROM angajati a  
        JOIN departamente b  
          ON (a.id_dep=b.id_dep  
             AND salariu+nvl(comision,0) >1000 )  
        JOIN grila_salariu c  
          ON (a.salariu>=c.nivel_inf AND  
             a.salariu<=c.nivel_sup)  
        ORDER BY 1;
```



Natural JOIN in SQL-3

➤ NATURAL JOIN

- Este un equi-join dupa coloane cu acelasi nume si are sintaxa urmatoare:

```
SELECT [DISTINCT | ALL]  
[[table | table_alias].]{column | expression}  
[column_alias]  
FROM [schema.]table1 [table1_alias]  
NATURAL JOIN [schema.]table2 [table2_alias]  
[other clauses]
```



Inner JOIN in SQL-3

Exemplu:

- Tabele DEPARTAMENTE si ANGAJATI au o coloana comuna(ID_DEP) dupa care se poate face un join natural:
**SQL> SELECT id_dep, nume, data_ang, salariu, den_dep
FROM angajati
NATURAL JOIN departamente ;**
- Observatii:
 - In cazul folosirii clauzei NATURAL JOIN cele doua coloane trebuie sa aiba acelasi nume.
 - Nu se accepta aliasuri pentru coloanele comune.
 - Daca coloanele au acelasi nume, nu se tine cont de tipul si semnificatia coloanelor si rezultatele pot fi eronate.



Outer JOIN in SQL-3

➤ OUTER JOIN ... ON

- Join-ul extern se foloseste cand se doreste ca in rezultat sa apara si liniile cu valori nule pe coloanele corespondente din tabelele relationate. Sintaxa este urmatoarea:

```
SELECT [DISTINCT] lista_de_expresii  
FROM tabela1  
LEFT \  
RIGHT | OUTER JOIN tabela2  
FULL /  
ON (tabela1.nume_coloana1 =  
    tabela2.numecoloana2)
```




Left Outer JOIN in SQL-3

➤ LEFT OUTER JOIN

- In cazul join-ului extern stanga valorile nule provin din tabela2(cea deficitara ca date).

```
SELECT [DISTINCT | ALL]  
{{table | table_alias}.}{column | expression}  
[column_alias]  
FROM  
[schema.]table1 [table1_alias]  
LEFT [OUTER ] JOIN [schema.]table2 [table2_alias]  
ON  {{table1 | table1_alias}.}column_fromTable1 =  
    {{table2 | table2_alias}.}column_fromTable2  
[other clauses]
```



Left Outer JOIN in SQL-3

Exemplu:

- Cererea de mai jos face o lista cu toate departamente si angajatii lor, dar afiseaza si departamentele care nu au niciun angajat:

```
SQL> SELECT b.den_dep departament, a.numa nume_ang,  
a.data_ang, a.salariu FROM departamente b  
LEFT OUTER JOIN angajati a ON(a.id_dep=b.id_dep)  
ORDER by 1;
```

- Urmatoarea cerere este echivalenta cu prima:

```
SQL> SELECT b.den_dep departament, a.numa nume_ang,  
a.data_ang, a.salariu FROM angajati a , departamente b  
WHERE a.id_dep(+)=b.id_dep;
```

- Notatia pentru join extern stanga este: $R \lt \circ \triangleright_l S$. In rezultat apar toate liniile tablei din stanga operatorului (R), inclusiv cele care nu au corespondent in tabela din dreapta(S).



Right Outer JOIN in SQL-3

➤ RIGHT OUTER JOIN

- In cazul join-ului extern dreapta valorile nule provin din tabela1(cea deficitara ca date). Sintaxa este similara cu join extern stanga.

Exemplu:

- Cererea urmatoare face o lista cu sefi de departament si subalternii lor, inclusiv angajatii care nu au sefi:

```
SQL> SELECT b.id_ang sef, b.numename_sef, a.numename_ang,  
           a.data_ang, a.salariu  
FROM angajati b  
RIGHT OUTER JOIN angajati a ON(a.id_sef= b.id_ang);
```

- Urmatoarea cerere este echivalenta cu prima:

```
SQL> SELECT b.id_ang sef, b.numename_sef, a.numename_ang,  
           a.data_ang, a.salariu  
FROM angajati b, angajati a  
WHERE a.id_sef= b.id_ang(+);
```

- Notatia pentru join extern dreapta este: $R \bowtie_R S$. In rezultat apar toate liniile tabelului din dreapta operatorului (S).



Outer JOIN in SQL-3

- Sa analizam urmatoarele cereri:

```
SQL> SELECT b.id_ang sef, b.numa numa_sef, a.numa numa_ang,  
        a.data_ang, a.salariu  
        FROM angajati a  
        LEFT OUTER JOIN angajati b ON (a.id_sef = b.id_ang);
```

```
SQL> SELECT b.id_ang sef, b.numa numa_sef, a.numa numa_ang,  
        a.data_ang, a.salariu  
        FROM angajati b  
        RIGHT OUTER JOIN angajati a ON (a.id_sef = b.id_ang);
```

Observatie:

- Daca executam cele doua cereri vom obtine acelasi rezultat.
- Practic, daca inversam tipul de join si aliasurile de coloana vom obtine doua cereri identice.



Full Outer JOIN in SQL-3

➤ FULL OUTER JOIN

- In cazul unui join extern complet rezultatul contine toate liniile din rezultatul joinului general si joinul extern LEFT/RIGTH, obtinut cu aceeasi conditie . Sintaxa este urmatoarea:

```
SELECT [DISTINCT|ALL]  
{{table|table_alias}.}{column|expression} [column_alias]  
FROM  
[schema.]table1 [table1_alias]  
FULL [OUTER] JOIN [schema.]table2 [table2_alias]  
ON  {{table1|table1_alias}.}{column_fromTable1 =  
    {{table2|table2_alias}.}{column_fromTable2  
[other clauses]
```



Full Outer JOIN in SQL-3

Exemplu:

- Cererea urmatoare returneaza o lista cu toti angajatii si toate departamentele:

```
SQL> SELECT a.nume, a.data_ang, b.den_dep, b. telefon  
      FROM angajati a  
      FULL OUTER JOIN departamente b ON (a.id_dep=b.id_dep);
```

- Observatie: Urmatoarea cerere **NU ESTE** echivalenta cu prima:

```
SQL> SELECT a.nume, a.data_ang, b.den_dep, b.telefon  
      FROM angajati a, departamente b  
      WHERE a.id_dep(+) = b.id_dep(+);
```

ERROR at line 3: ORA-01468: a predicate may reference only one outer-joined table

- Notatia pentru join extern complet este: $R \bowtie S$. In rezultat apar toate liniile tabelului din stanga si dreapta operatorului.



Full Outer JOIN in SQL-3

- Observatie: Un join extern se poate face pe mai multe tabele.
Exemplu:
- Cererea urmatoare returneaza o lista cu toate departamentele (inclusiv cele fara angajati), toti angajatii si toate gradele de salarizare(inclusiv cele care nu au corespondent in salariile angajatilor):

```
SQL> SELECT b.den_dep, a.nume, a.data_ang, c.grad  
        FROM angajati a  
        FULL OUTER JOIN departamente b  
                ON (a.id_dep=b.id_dep)  
        FULL OUTER JOIN grila_salariu c  
                ON a.salariu between c.nivel_inf and c.nivel_sup  
        ORDER BY b.den_dep;
```



Capitolul 9

Functii SQL



Functii SQL

- Functia poate fi vazuta ca un operator de manipulare a datelor care accepta unul sau mai multe argumente (constanta , variabila , referire de coloana, etc.) si returneaza un rezultat.

Sintaxa de apelare este urmatoarea:

function_name(arg1, arg2,...)

- Rolul lor este de a face mai puternice cererile Oracle SQL*Plus si se pot folosi pentru:
 - Efectuarea calculelor numerice;
 - Prelucrare de siruri de caractere;
 - Prelucrare de date calendaristice;
 - Schimbarea formatului pentru datele de afisare;
 - Conversia tipurilor de date.



Functii SQL

- Dupa specific si tipuri de argumente, functiile se pot imparti in urmatoarele categorii:
 - **Functii numerice**
 - **Functii pentru siruri**
 - **Functii pentru data calendaristica**
 - **Functii de conversie**
 - **Functii diverse** (accepta diverse tipuri de argumente)
 - **Functii de grup**



Funcții numerice

➤ Funcții numerice

- Aceste funcții au ca parametri valori numerice și returnează tot o valoare numerică.

Exemple:

- **ABS (n)** – returnează valoarea absolută a lui **n**

SQL> SELECT abs(-12) FROM dual; -- returnează valoarea 12

- **CEIL (n)** – returnează cel mai mic întreg $\geq n$

SQL> SELECT ceil(14.7) FROM dual; -- returnează valoarea 15

- **COS (n)** – returnează **cosinus (n)** unde **n** este în radiani

SQL> SELECT cos(180 * 3.14/180) FROM dual; -- returnează -1

- **COSH (n)** – returnează cosinus hiperbolic

SQL> SELECT cosh(0) FROM dual; -- returnează valoarea 1



Functii numerice

- **EXP (n)** – returneaza **e** la puterea **n** (**e**=2.7182..)

SQL> SELECT exp(4) FROM dual; -- returneaza valoarea 54.5981

- **FLOOR (n)** – returneaza cel mai mare intreg $\leq n$

SQL> SELECT floor(11.6) FROM dual; -- returneaza valoarea 11

- **LN (n)** – returneaza logaritmul natural al lui **n** ($n > 0$)

SQL> SELECT ln(95) FROM dual; -- returneaza valoarea 4.5538

- **LOG (m,n)** – returneaza logaritmul in baza **m** al lui **n**

SQL> SELECT log(10,100) FROM dual; -- returneaza valoarea 2

- **MOD (m,n)** – returneaza restul impartirii lui **m** la **n**

SQL> SELECT mod(14,5) FROM dual; -- returneaza valoarea 4

- **POWER (m,n)** – returneaza **m** la puterea **n**

SQL> SELECT power(3,2) FROM dual; -- returneaza valoarea 9



Functii numerice

- **ROUND (n[,m])** – returneaza **n** rotunjit la :

m zecimale daca $m > 0$;

0 zecimale daca **m** este omis;

m cifre inainte de virgula daca $m < 0$;

SQL> SELECT round(15.193, 1) FROM dual; -- returneaza 15.2

SQL> SELECT round(15.193) FROM dual; -- returneaza 15

SQL> SELECT round(15.193, -1) FROM dual; -- returneaza 20

- **SIGN (n)** – returneaza -1 daca $n < 0$, 0 daca $n = 0$ si 1 daca $n > 0$

SQL> SELECT sign(-17.5) FROM dual; -- returneaza valoarea -1

- **SIN (n)** – returneaza **sinus (n)** unde **n** este in radiani

SQL> SELECT sin(30 * 3.14/180) FROM dual; -- returneaza 0.5

- **SINH (n)** – returneaza cosinus hiperbolic

SQL> SELECT sinh(1) FROM dual; -- returneaza valoarea 1.1752



Functii numerice

- **SQRT (n)** – returneaza radacina patrata a lui **n**, unde $n > 0$
SQL> SELECT sqrt(26) FROM dual; -- returneaza valoarea 5.099
- **TAN (n)** – returneaza tangenta lui **n**, unde **n** este in radiani
SQL> SELECT tan(135 * 3.14/180) FROM dual; -- returneaza -1
- **TANH (n)** – returneaza tangenta hiberbolica a lui **n**, unde **n** este in radiani
SQL> SELECT tanh(0.5) FROM dual; -- returneaza 0.4621
- **TRUNC (n[,m])** – returneaza **n** trunchiat la :
 - m** zecimale daca $m > 0$;
 - 0** zecimale daca **m** este omis;
 - m** cifre inainte de virgula daca $m < 0$.**SQL> SELECT trunc(15.193, 1) FROM dual; -- returneaza 15.1**
SQL> SELECT trunc(15.193) FROM dual; -- returneaza 15
SQL> SELECT trunc(15.193, -1) FROM dual; -- returneaza 10



Functii alfanumerice

➤ Functii pentru siruri

- Aceste functii accepta la intrare valori alfanumerice si returneaza o valoare numerica sau tot o valoare alfanumerica. Cele care returneaza valori de tip VARCHAR2 pot avea lungimea maxima 4000 caractere iar cele de tip CHAR pot avea lungimea maxima 2000 caractere.

Exemple:

- **CHR (n)** -returneaza caracterul ASCII care are reprezentarea **n**

SQL> SELECT chr(75) FROM dual; -- returneaza valoarea K

- **CONCAT (char1,char2)** – returneaza concatenarea lui **char1** cu **char2**

**SQL> SELECT concat(concat(ume,' este '),functie) ume_functie
FROM angajati WHERE id_ang=7839;**



Functii alfanumerice

- **INITCAP (char)** – returneaza **char** cu majuscule

**SQL> SELECT initcap('popescu mihai') FROM dual; -- returneaza
Popescu Mihai**

- **REPLACE (string,string1,[string2])** – inlocuieste in **string** caracterele din **string1** cu caracterele din **string2**

**SQL> SELECT replace('JACK si JUE','J','BL') FROM dual;
-- returneaza valoarea BLACK si BLUE**

- **TRANSLATE (col/string,string1,string2)** – translateaza in **col/string** caracterele specificate in **string1** in caracterele specificate in **string2**

**SQL> SELECT nume,translate(nume,'C','P') FROM angajati
WHERE id_dep=10; -- translateaza in nume litera C in P**

- **RPAD/LPAD (char1,n,[char2])** – adauga la dreapta/stanga lui **char1** caracterele **char2** pana la lungimea **n**

**SQL> SELECT rpad(nume,20,'*') completare_nume FROM
angajati WHERE id_dep=10;**



Functii alfanumerice

- **RTRIM (char[,set])** sterge din **char** ultimele caractere specificate in **set**

SQL> SELECT rtrim('Popescu','scu') FROM dual;--returneaza Pope

- **SUBSTR (char,m[,n])** – returneaza **n** caractere din **char** incepand cu pozitia **m**

SQL> SELECT substr ('Popescu ',2,3) FROM dual; -- returneaza ope

- **INSTR (char1,char2[,n[,m]])** – returneaza pozitia lui **char2** incepand cu pozitia **n** la a **m**-a aparitie

SQL> SELECT instr('Protopopescu','op',3,2) FROM dual; --
returneaza **7**

- **LENGTH (char)** – returneaza lungimea lui **char** ca numar de caractere

SQL> SELECT length('analyst') FROM dual; --returneaza 7

- **SIGN(col/expr/value)**-returneaza **-1** daca col/exp/valoarea este un numar<0 , **0** daca este zero si **+1** daca este numar>0

SQL> SELECT nume, comision, sign(comision) from angajati;



Functii alfanumerice

Exemplu:

- Sa se scrie o cerere care returneaza numarul de aparitii ale literei A in sirul obtinut prin concatenarea numelui angajatului si a functiei.

```
SQL> SELECT nume,functie,  
       nume || functie "NUME+FUNCTIE",  
       length(concat(nume,functie)) lung_init,  
       length(TRANSLATE(concat(nume,functie),'XA','X')) lung_fin,  
       length(concat(nume,functie))-  
       length(TRANSLATE(concat(nume,functie),'XA','X')) nr_apar  
FROM angajati;
```



Funcții pentru data calendaristică

➤ Funcții pentru data calendaristică

- O bază de date ORACLE stochează datele calendaristice în următorul format intern:

Secol / An / Luna / Zi / Ora / Minut / Secunda

- Formatul implicit de afișare sau intrare pentru o dată calendaristică este *DD-MON-YY*.
- Plaja datei calendaristice este între '1-JAN-4712' i.e.n și '31-DEC-4712' e.n.
- Toate funcțiile de tip dată calendaristică întorc o valoare de tip *DATE* cu excepția lui *MONTHS_BETWEEN* care întoarce o valoare numerică.



Functii pentru data calendaristica

Exemple:

- **ADD_MONTHS (date,n)** – returneaza o data prin adaugarea a **n** luni la **date**

```
SQL> SELECT nume, data_ang, add_months(data_ang,3)  
      data_mod FROM angajati WHERE id_dep=10;
```

- **LAST_DAY (date)** – returneaza data ultimei zile din luna cuprinsa in **date**

```
SQL> SELECT nume, data_ang, last_day(data_ang) ultima_zi  
      FROM angajati WHERE id_dep=10;
```

- **MONTHS_BETWEEN(date1,date2)** – returneaza numarul de luni (si fractiuni de luna) cuprinse intre **date1** si **date2**.

Daca **date1>date2** rezultatul va fi pozitiv altfel va fi negativ.

```
SQL> SELECT nume, data_ang, sysdate,  
      months_between(sysdate,data_ang) luni_vechime  
      FROM angajati WHERE id_dep=10;
```



Functii pentru data calendaristica

- **NEXT_DAY(date,char)** – returneaza data urmatoarei zile **char** dupa **date**

```
SQL> SELECT next_day('17-MAR-2025', 'TUESDAY')  
       marti_urmatoare FROM dual;
```

- **ROUND(date,fmt)** – returneaza data prin rotunjirea lui **date** la formatul **fmt**

```
SQL> SELECT round(to_date('17-NOV-2025'), 'YEAR') rot_an  
       FROM dual;
```

```
SQL> SELECT round(to_date('17-MAR-2025'), 'MONTH')  
       rot_luna FROM dual;
```

- **TRUNC(date,fmt)** – returneaza data prin trunchierea lui **date** la formatul **fmt**

```
SQL> SELECT trunc(to_date('17-MAR-2025'), 'YEAR') trunc_an  
       FROM dual;
```



Functii pentru data calendaristica

- Folosind operatorii aritmetici + si – se pot face diferite operatii cu date calendaristice:
 - **data + numar** - aduna un numar de zile la data, returnand tot o data calendaristica;
 - **data - numar** - scade un numar de zile din data, returnand tot o data calendaristica;
 - **data1 – data2** - scade data2 din data 1, obtinand un nr. de zile;
 - **data + numar/24** - aduna la data un numar de ore pentru a obtine tot o data calendaristica.

Exemplu:

```
SQL> SELECT data_ang,data_ang+7,data_ang-7,  
           sysdate-data_ang FROM angajati  
WHERE data_ang LIKE '%JUN%';
```



Funcții de conversie

➤ Funcții de conversie

- Aceste funcții fac conversia unui tip de data în alt tip de data.
- **TO_CHAR(date[,fmt[, 'nlsparams']])** – face conversia unei date de tip DATE în format VARCHAR2

```
SQL> SELECT nume, to_char(data_ang, 'Month DD, YYYY')  
       data_ang FROM angajati  
       WHERE to_char(data_ang, 'YYYY') LIKE '1987';
```

- **TO_DATE(char[,fmt[, 'nlsparams']])** – face conversia unei variabile de tip CHAR sau VARCHAR2 în format DATE

```
SQL> SELECT to_date('15032025', 'DD-MM-YYYY') data  
       FROM dual;
```



Funcții de conversie

- **TO_NUMBER(char[,fmt[, 'nlsparams']])** – face conversia unei variabile de tip CHAR sau VARCHAR2 în format NUMBER

Exemplu:

- Pentru calculul primei în funcție de anul angajării, folosim cererea:

```
SQL> SELECT nume, functie, salariu,  
        salariu+to_number(to_char(data_ang,'yyyy'))/10 prima  
FROM angajati  
WHERE to_number(to_char(data_ang,'YYYY'))=1987;
```




Functii de conversie

- Exemple de format pentru date numerice:

Format	Semnificatie	Exemple	
9	numere(nr.de 9 determina lungimea de afisare)	999999	1234
0	afiseaza zerourile de la inceput	099999	001234
\$	simbolul dolar	\$999999	\$1234
.	punct zecimal	999999.99	1234.00
,	virgula	999,999	1,234
MI	semnele minus la dreapta(valori negative)	999999MI	1234-
PR	paranteze pentru numere negative	999999PR	<112>
EEEE	notatie stiintifica	99.999EEEE	1.234E+03
V	inmultire cu 10**n (n=numar de 9 dupa V)	9999V99	123400
B	afiseaza valori zero ca blancuri(nu 0)	B9999.99	1234.00

- Formatul de afisare a datelor se poate seta cu comanda COLUMN:
SQL> COLUMN nume FORMAT a30 -- format alfanumeric max 30 car
SQL> COL salariu FOR 99999.99 -- format numeric cu 2 zecimale¹⁸



Funcții de conversie

- **EXTRACT ([YEAR], [MONTH], [DAY], [HOUR], [MINUTE], [SECONDE] from datetime)** – extrage anul, luna, ziua, minutul, secunda din data calendaristica **datetime**.

```
SQL> SELECT  nume, data_ang,  
             extract (year from data_ang) ANUL,  
             extract(month from data_ang) LUNA,  
             extract (day from data_ang) ZIUA  
FROM angajati  
WHERE functie ='ANALIST' ;
```



Functii diverse

➤ Functii diverse

- Aceste functii accepta orice tip de argumente.
- **GREATEST(expr1 [,expr2]...)** – returneaza cea mai mare valoare din lista de argumente numerice, sau expresia care are prima litera pozitionata ultima in alfabet , pentru date alfanumerice.

SQL> SELECT greatest(12,34,77,89) from dual;

- **LEAST(expr1 [,expr2]...)** – similar cu GREATEST dar returneaza cea mai mica valoare din lista de argumente.

SQL> SELECT nume, functie, least(nume,functie) from angajati;

- **DECODE(expr,search1,result1,...default)** - **expr** este comparata cu fiecare valoare **search k** si returneaza **rezult k** daca **expr** este egala cu valoarea **search k**, iar daca nu gaseste nicio egalitate returneaza valoarea **default**.
- Functia DECODE are efectul unui **CASE** sau a unei constructii **IF-THEN-ELSE**.



Functii diverse

- Tipurile de parametri pot fi :
 - **expression** poate fi orice tip de data;
 - **search** este de acelasi tip ca *expression*;
 - **result** este valoarea intoarsa si poate fi orice tip de data;
 - **default** este de acelasi tip ca *result*.

Exemplu:

- In exemplul urmator se calculeaza prima in raport de functia angajatului in companie:

```
SQL> SELECT nume,functie,salariu,  
        decode(functie,'DIRECTOR', salariu*0.35, 'ANALIST',  
        salariu*0.25, salariu/5) prima  
        FROM angajati WHERE id_dep=20  
        ORDER BY functie;
```



Functii diverse

- **CASE expr WHEN value1/expr1 THEN statements_1;
WHEN value2/expr2 THEN statements_2;
...
[ELSE statements_k;]
END**
- CASE functioneaza similar cu functia DECODE. Parametrii sunt:
 - **expr** – este expresia care se va evalua
 - **statements_1** – este valoarea returnata cand **expr = value1/expr1**;
 - **statements_2** – este valoarea returnata cand **expr = value2/expr2**;
 - **statements_k** – este valoarea implicită returnata cand
expr <> value1, value2, ...

Observatii:

- Expresia **expr_i** poate fi diferita pentru fiecare ramura WHEN;
- Expresia **expr_i** poate contine mai multi operatori de comparative;
- Functia CASE poate fi folosita in cererea SELECT sau clauza WHERE si are mai multe tipuri de sintaxa.



Funcții diverse

Exemple:

```
SQL> SELECT nume,data_ang, salariu,
```

```
  CASE
```

```
  WHEN salariu <= 1000 THEN salariu*0.1
```

```
  WHEN salariu > 1000 THEN salariu*0.2
```

```
  END prima
```

```
  FROM angajati;
```

```
SQL> SELECT id_ang, nume,functie, data_ang, salariu
```

```
  FROM angajati
```

```
  WHERE functie= (CASE id_ang
```

```
    WHEN 7839 then 'PRESEDINTE'
```

```
    WHEN 7698 then 'DIRECTOR'
```

```
  END);
```



Functii diverse

- **NVL(expr1 ,expr2)** – returneaza **expr2** daca **expr1** este null

```
SQL> SELECT nume,data_ang,comision,nvl(comision,0) nvl_com  
      FROM angajati WHERE id_dep=30  
      ORDER BY nume;
```

- **COALESCE(expr1 ,expr2, ...)** – returneaza prima expresie **not null** din lista de argumente

```
SQL> SELECT nume,data_ang, comision, coalesce(comision,0)  
      FROM angajati WHERE id_dep=30  
      ORDER BY nume;
```



Functii diverse

- De retinut ca valoarea null trebuie obligatoriu convertita la zero atunci cand se fac operatii aritmetice, altfel rezultatul va fi null.
- Functiile pot fi imbricate pana la orice nivel, ordinea de executie fiind din interior spre exterior.

USER – returneaza userul curent Oracle



Funcții de grup

➤ Funcții de grup

- Sintaxa pentru funcțiile de grup este următoarea:

SELECT [column,] group_function(column)...

FROM table

[WHERE condition]

[GROUP BY [CUBE] [ROLLUP] group_expression]

[HAVING having_expression]

[ORDER BY column(position)] [ASC,DESC];



Functii de grup

- Parametrii comenzii au urmatoarea semnificatie(cei din paranteze sunt optionalii):
- **group_function** – specifica numele functiei/functiilor de grup;
- **WHERE condition** – reprezinta o inlantuire de conditii care trebuie sa fie indeplinita in criteriul de selectie a liniilor;
- **GROUP BY group_expresion** - specifica coloanele dupa care se face gruparea rezultatelor;
- **HAVING**– este folosita pentru a specifica conditiile de grupare a rezultatelor;
- **having_expresion** – reprezinta un nume de coloana sau o expresie care poate folosi functii si coloane;
- **ORDER BY**– specifica coloanele(sau ordinea coloanelor specificate prin pozitie) dupa care se face ordonarea rezultatelor;



Functii de grup

- **CUBE** – este un operator folosit impreuna cu o funcție de grup pentru a genera randuri suplimentare in rezultate (produce subtotaluri pentru toate combinatiile posibile ale gruparii specificate in clauza GROUP BY);
- **ROLLUP** – este un operator folosit pentru a obtine un set de rezultate care contin subtotaluri, pe langa valorile pentru randurile grupate in mod obisnuit.



Functii de grup

- Aceste functii returneaza rezultate bazate pe grupuri de inregistrari, nu pe o singura inregistrare ca cele prezentate la punctele anterioare.
- Gruparea se face folosind clauza GROUP BY intr-o cerere SELECT si in acest caz toate elementele listei trebuie cuprinse in clauza de grupare.
- Functiile de grup pot fi apelate si in clauza HAVING, dar nu pot fi apelate in clauza WHERE in mod explicit.
- Se poate folosi operatorul DISTINCT pentru a sorta numai elementele distincte din lista, dar se poate folosi si operatorul ALL pentru a considera si inregistrarile duplicate.



Functii de grup

- **AVG([DISTINCT/ALL] expr)** – returneaza valoarea medie a lui **expr**, ignorand valorile nule.

Exemplu: Sa calculam valoarea medie a salariului si comisionului pe toate departamentele:

```
SQL> SELECT avg(salariu), avg(comision) FROM  
angajati;
```

Exemplu: Daca dorim sa aflam valorile medii pe fiecare departament, trebuie sa folosim obligatoriu clauza GROUP BY:

```
SQL> SELECT id_dep, avg(salariu), avg(comision)  
FROM angajati  
GROUP BY id_dep;
```

- Prin folosirea operatorilor DISTINCT si ALL rezultatul interogarii poate fi diferit:

```
SQL> SELECT id_dep, avg(salariu), avg(all salariu),  
avg(distinct salariu)  
FROM angajati GROUP BY id_dep;
```



Functii de grup

- **COUNT(* | [DISTINCT/ALL] expr)** – returneaza numarul de linii specificate in **expr**. Daca se foloseste ***** se numara toate liniile, inclusiv cele care contin valori nule pe anumite coloane, iar daca se foloseste **expr** se numara numai valorile **not null** ale lui **expr**.

Exemplu: Pentru a afla numarul angajatilor care au primit comision pe fiecare departament, folosim urmatoarea cerere:

```
SQL> SELECT id_dep, count(*), count(comision),  
           count(all comision), count(distinct comision)  
FROM angajati GROUP BY id_dep;
```

- **MAX([DISTINCT/ALL] expr)** – returneaza valoarea maxima pentru **expr**.

Exemplu: Sa calculam salariul maxim pe fiecare departament:

```
SQL> SELECT a.id_dep, b.den_dep, max(salariu)  
FROM angajati a, departamente b  
WHERE a.id_dep=b.id_dep  
GROUP BY a.id_dep, b.den_dep;
```



Functii de grup

- **MIN([DISTINCT/ALL] expr)** – returneaza valoarea minima pentru expr.

Exemplu: Sa calculam venitul minim pe fiecare departament:

```
SQL> SELECT id_dep,  
           min(salariu + nvl(comision,0)) venit_minim  
FROM angajati GROUP BY id_dep;
```

- Trebuie mentionat ca operatorii DISTINCT si ALL nu au vreun efect pentru functiile MIN si MAX.
- **SUM([DISTINCT/ALL] expr)** – returneaza suma valorilor pentru expr.

Exemplu: Suma salariilor si comisioanelor pe fiecare departament:

```
SQL> SELECT id_dep, sum(salariu),  
           sum(distinct salariu), sum(comision)  
FROM angajati GROUP BY id_dep;
```



Functii de grup

- **Functii de grup cu clauza HAVING**

Exemplu: Pentru a afla numarul departamentelor care au cel putin doua functii distincte pentru angajati, folosim cererea:

```
SQL> SELECT id_dep,count(funcctie),count(distinct functie)  
FROM angajati  
HAVING count(distinct functie)>=2  
GROUP BY id_dep  
ORDER by 1;
```

- Trebuie mentionat ca functiile de grup se pot folosi direct numai in clauza HAVING, nu si in clauza WHERE:

```
SQL> SELECT id_dep, sum(salariu) FROM angajati  
WHERE sum(salariu)>1000  
GROUP BY id_dep;
```

ERROR at line 3: ORA-00934: group function is not allowed here



Functii de grup

- **CUBE**– Genereaza un superset de grupuri prin referiri incrucisate pe coloanele incluse in clauza GROUP BY.

Exemplu: Cererea urmatoare calculeaza salariul total pe toate departamentele, pe fiecare tip de functie, pe fiecare departament si pe fiecare tip de functie din cadrul departamentelor 10 si 20:

```
SQL> SELECT id_dep, functie, SUM(salariu) salariu_total  
FROM angajati  
WHERE id_dep < 30  
GROUP BY CUBE (id_dep, functie);
```

ID_DEP	FUNCTIE	SALARIU_TOTAL
		19625
	ANALIST	6000
	DIRECTOR	5425
	TEHNICIAN	3200
	PRESEDINTE	5000
10		8750
10	DIRECTOR	2450
10	TEHNICIAN	1300
10	PRESEDINTE	5000
20		10875
20	ANALIST	6000
20	DIRECTOR	2975
20	TEHNICIAN	1900



Functii de grup

- **ROLLUP** – Genereaza un superset de grupuri pe coloanele incluse in clauza GROUP BY.

Exemplu: Cererea urmatoare calculeaza salariul total pe fiecare tip de functie din cadrul departamentelor, pe fiecare departament si pe toate departamentele care indeplinesc restrictia de id_dep(10 si 20):

```
SQL> SELECT id_dep, functie, SUM(salariu) salariu_total  
FROM angajati  
WHERE id_dep < 30  
GROUP BY ROLLUP (id_dep, functie);
```

ID_DEP	FUNCTIE	SALARIU_TOTAL
10	DIRECTOR	2450
10	TEHNICIAN	1300
10	PRESEDINTE	5000
10		8750
20	ANALIST	6000
20	DIRECTOR	2975
20	TEHNICIAN	1900
20		10875
		19625



Functii de grup

- **CAST ([*Expression* | NULL | ?] AS *Datatype*)** - Face conversia datelor de la un tip de data la alt tip de data, sau la valoarea nula.

```
SQL> SELECT id_dep,  
           AVG(salariu), CAST(avg(salariu) AS DECIMAL(10,2)) cast_sal,  
           AVG(comision), CAST(avg(comision) AS CHAR(10)) cast_com  
FROM angajati  
GROUP BY id_dep;
```

ID_DEP	AVG(SALARIU)	CAST_SAL	AVG(COMISION)	CAST_COM
20	2175	2175		
30	1541.66667	1541.67	550	550
10	2916.66667	2916.67		



Capitolul 10

Subcereri SQL



Subcereri SQL

- In Oracle SQL*Plus subcererile sunt cereri SQL incluse in clauzele SELECT, FROM, WHERE, HAVING sau ORDER BY ale altei cereri numita si cerere principala.
- Rezultatele returnate de o subcerere sunt folosite de o alta subcerere sau de cererea principala in situatii cum ar fi:
 - Crearea de tabele sau view-uri;
 - Inserarea, modificarea si stergerea inregistrarilor din tabele;
 - In returnarea datelor pentru conditii puse in comenzile SELECT, UPDATE, DELETE, INSERT si CREATE TABLE.



Subcereri SQL

- Din punct de vedere al rolului pe care îl au într-o comandă SQL și a modului de a face construcția comenzii, subcererile pot fi:
 - **Subcereri ascunse**
 - **Subcereri corelate**
 - **Subcereri pe tabelă temporară**
 - **Subcereri pe clauză HAVING**
 - **Subcereri pe clauză SELECT**
 - **Subcereri pe clauză ORDER BY**



Subcereri ascunse

➤ Subcereri ascunse

- Subcererile ascunse pot fi impartite in mai multe categorii, in functie de numarul de coloane sau linii pe care le returneaza:
 - **Subcereri care returneaza o valoare**
 - **Subcereri care returneaza o coloana**
 - **Subcereri care returneaza o linie**
 - **Subcereri care returneaza mai multe linii**

✓ Subcereri care returneaza o valoare

- Aceste subcereri se folosesc intr-o alta cerere si au urmatoarea constructie:

SELECT column 1, column 2, ...

FROM table 1

WHERE column =

(SELECT column FROM table 2

WHERE condition);



Subcereri ascunse

- Subcererea se executa prima pentru a verifica conditia din clauza WHERE si poate intoarce o singura valoare dintr-o tabela, prin conditii care depind de datele din tabela folosita in cererea principala. Daca conditia din clauza WHERE a subcererii nu returneaza o singura valoare, atunci se va genera o eroare.

Exemplu:

- Pentru a afla care sunt angajatii care au cel mai mare salariu in companie, putem sa folosim urmatoarea cerere:

```
SQL> SELECT id_dep, nume, functie, salariu  
FROM angajati  
WHERE salariu=( SELECT max(salariu)  
FROM angajati );
```




Subcereri care returneaza o coloana

✓ Subcereri care returneaza o coloana

- Sunt subcereri care returneaza mai multe valori si folosesc operatorii **IN** si **NOT IN** intr-o constructie ca mai jos:

SELECT column 1, column 2, ...

FROM table 1

WHERE column IN [NOT IN]

(SELECT column

FROM table 2

WHERE condition);

- In cazul in care am folosi operatorul **=** in locul operatorului **IN** s-ar genera o eroare pentru ca subcererea returneaza mai mult de o valoare (ORA-01427). Trebuie specificat ca operatorul **IN** se poate folosi in locul operatorului **=** , dar invers este gresit.



Subcereri care returneaza o coloana

Exemplu:

- Daca vrem sa aflam salariile angajatilor care au functii similare functiilor aferente departamentului 20, folosim urmatoarea cerere :

```
SQL> SELECT nume, functie, salariu  
      FROM angajati  
      WHERE functie IN  
            ( SELECT DISTINCT functie  
              FROM angajati  
              WHERE id_dep=20);
```



Subcereri care returneaza o linie

✓ Subcereri care returneaza o linie

- Sunt subcereri care returneaza mai multe coloane dar o singura linie si folosesc operatorii **IN** si **NOT IN**, intr-o constructie ca mai jos:

```
SELECT column 1, column 2, ...
```

```
FROM table 1
```

```
WHERE (column 1,column 2...) = [<>][IN] [NOT IN]
```

```
( SELECT column 1,column 2..
```

```
FROM table 2
```

```
WHERE condition );
```

- Numarul, ordinea si tipul coloanelor din clauza WHERE trebuie sa coincida cu cele din subcerere.



Subcereri care returneaza o linie

Exemplu:

- Daca dorim o lista cu angajatii din departamentul lui Savescu Ovidiu, care au aceeaasi functie ca el, folosim urmatoarea cerere:

```
SQL> SELECT id_dep, nume, functie, data_ang  
        FROM angajati  
        WHERE (id_dep, functie) =  
              (SELECT id_dep, functie FROM angajati  
               WHERE nume='SAVESCU OVIDIU');
```

- In acest caz subcererea returneaza o singura linie, deoarece Savescu are o singura functie si este angajat la un singur departament. In eventualitatea ca pot exista mai multi angajati cu acelasi nume, se va folosi clauza DISTINCT in subcerere(daca au acelasi departament si job).



Subcereri care returneaza mai multe linii

✓ Subcereri care returneaza mai multe linii

- Aceste subcereri au o constructie asemanatoare ca la punctul precedent, dar intorc mai multe linii cu mai multe coloane, drept pentru care se mai numesc si subcereri care intorc o tabela.

SELECT expr 1,... expr n

FROM table 1

WHERE (expr 1,...expr k) IN [NOT IN]

(SELECT expr 1, ...expr k

FROM table 2

WHERE condition);

- Trebuie specificat ca in locul coloanelor se pot folosi si expresii. Numarul de expresii si ordinea lor specificate in clauza WHERE trebuie sa coincida cu numarul de expresii din subcerere si sa fie de acelasi tip.



Subcereri care returneaza mai multe linii

Exemplu:

- Pentru a afla angajatii care au venit maxim pe fiecare departament, folosim urmatoarea cerere:

```
SQL> SELECT id_dep, nume, functie, salariu+nvl(comision,0) venit  
FROM angajati  
WHERE (id_dep, salariu+nvl(comision,0)) IN  
      (SELECT id_dep,max(salariu+nvl(comision,0)) venit_max  
FROM angajati GROUP BY id_dep);
```

- In cazul unei subcereri ascunse, cererea SELECT din subcerere ruleaza prima si se executa o singura data, intorcand valori ce vor fi folosite de cererea principala.



Subcereri corelate

➤ Subcereri corelate

- Subcererile corelate se executa o data pentru fiecare linie candidat prelucrata de cererea principala si la executie folosesc cel putin o valoare dintr-o coloana din cererea principala.
- O subcerere corelata se relationeaza cu cererea exterioara prin folosirea unor coloane ale cererii exterioare in clauza predicatului cererii interioare.

Constructia unei subcereri corelate este urmatoarea:

```
SELECT expr 1,... expr n
```

```
FROM table 1
```

```
WHERE (expr 1,...expr k) IN [NOT IN]
```

```
  (SELECT expr 1, ...expr k
```

```
    FROM table 2
```

```
    WHERE table 2.expr x = table 1.expr y [AND,OR] .. );
```



Subcereri corelate

- Pentru o cerere corelata, subcererea se executa de mai multe ori, cate o data pentru fiecare linie prelucrata de cererea principala, deci cererea interioara este condusa de cererea exterioara.
- Pasii de executie ai unei subcereri corelate sunt:
 - Se obtine linia candidat prin procesarea cererii exterioare;
 - Se executa cererea interioara corelata cu valoarea liniei candidat;
 - Se folosesc valorile rezultate din cererea interioara pentru a prelucra linia candidat;
 - Se repeta pana nu mai ramane nicio linie candidat.
- Trebuie specificat ca desi subcererea corelata se executa repetat, aceasta nu inseamna ca subcererile corelate sunt mai putin eficiente decat subcererile ascunse.



Subcereri corelate

Exemplu:

- Daca vrem sa aflam angajatii care au salariul peste valoarea medie a salariului din departamentul din care fac parte, folosim cererea urmatoare:

```
SQL> SELECT a.id_dep, a.numa, a.functie, a.salariu  
FROM angajati a WHERE a.salariu >  
      (SELECT avg(salariu) salariu_mediu FROM angajati b  
      WHERE b.id_dep=a.id_dep )  
ORDER By id_dep;
```

- Cererea principala proceseaza fiecare linie din tabela angajati si pastreaza toti angajatii care au salariul peste salariul mediu pe departamentul respectiv returnat de subcerere, care se coreleaza cu cererea principala prin conditia din clauza WHERE. In acest exemplu se foloseste o corelare de date pe aceeasi tabela.



Subcereri imbricate

- Subcererile pot fi imbricate. De exemplu, pentru a afla care angajati din companie au salariul mai mare decat salariul maxim din departamentul de proiectare, folosim cererea urmatoare:

```
SQL> SELECT nume, functie, data_ang, salariu
      FROM angajati
      WHERE salariu > ( SELECT max(salariu)
                        FROM angajati
                        WHERE id_dep= (SELECT id_dep
                                      FROM departamente
                                      WHERE
                                      den_dep= 'PROIECTARE'));
```



Subcereri pe tabela temporara

➤ Subcereri pe tabela temporara

- Aceste subcereri se intalnesc in cazul in care se foloseste o subcerere la nivelul clauzei FROM din cererea principala. Constructia este urmatoarea:

```
SELECT t1.column1,t1.column2,.. t2.expr 1,... t2.expr k  
FROM table t1,  
        ( SELECT expr 1, ...expr n  
          FROM table  
          WHERE conditions ) t2  
WHERE conditions  
ORDER BY expr;
```



Subcereri pe tabela temporara

Exemplu:

- Pentru a afla salariul maxim pe fiecare departament, se poate face o constructie ca mai jos, unde subcererea intoarce o tabela temporara cu salariul maxim pe fiecare departament:

```
SQL> SELECT b.id_dep, a.den_dep, b.sal_max_dep  
        FROM departamente a, (SELECT id_dep, max(salariu)  
                               sal_max_dep FROM angajati GROUP BY id_dep) b  
        WHERE a.id_dep = b.id_dep  
        ORDER BY b.id_dep;
```



Subcereri pe clauza HAVING

➤ Subcereri pe clauza HAVING

- Aceste subcereri sunt specificate in clauza HAVING intr-o constructie ca cea de mai jos :

SELECT expr 1,... expr n

FROM table 1

WHERE conditions

HAVING expr (operator)

(SELECT expr 1, ...expr k

FROM table 2

WHERE conditions)

GROUP BY expresion;

- Trebuie retinut ca o clauza HAVING se foloseste intotdeauna impreuna cu clauza GROUP BY si intr-o clauza HAVING se pot folosi functii de grup in mod explicit.



Subcereri pe clauza HAVING

Exemplu:

- Pentru a afla ce departamente au cei mai multi angajati pe aceeasi functie, folosim urmatoarea cerere:

```
SQL> SELECT d.den_dep, a.functie, count(a.id_ang) nr_ang  
      FROM angajati a, departamente d  
      WHERE a.id_dep = d.id_dep  
      HAVING count(a.id_ang) = (SELECT max(count(id_ang))  
                                FROM angajati GROUP BY id_dep, functie)  
      GROUP BY d.den_dep, a.functie;
```

- Clauza HAVING poate fi folosita si intr-o subcerere pe tabela temporara.



Subcereri pe clauza SELECT

➤ Subcereri pe clauza SELECT

- Se poate construi o subcerere si pe clauza SELECT, avand urmatoarea constructie :

```
SELECT expr 1,... expr n,  
      (SELECT expr FROM table 2  
      WHERE table 2.expr x = table 1.expr y [AND,OR] .. ) alias  
      FROM table 1  
WHERE conditions  
ORDER BY expr 1,...expr k
```

- Aceste subcereri pot fi corelate sau ascunse dar trebuie sa returneze intotdeauna o singura valoare.



Subcereri pe clauza SELECT

Exemplu:

- Daca vrem sa aflam care sunt sefii directi ai angajatilor din departamentul 20, folosim cererea urmatoare:

```
SQL> SELECT nume nume_ang,  
          (SELECT nume FROM angajati b  
           WHERE b.id_ang=a.id_sef) nume_sef  
FROM angajati a  
WHERE id_dep=20  
ORDER BY nume ;
```




Subcereri pe clauza ORDER BY

➤ Subcereri pe clauza ORDER BY

- O subcerere pe clauza ORDER BY se foloseste numai pentru cereri corelate si trebuie sa returneze intotdeauna o singura valoare, avand urmatoarea constructie :

SELECT expr 1,... expr n,

FROM table 1

WHERE conditions

ORDER BY

(SELECT expr FROM table 2

WHERE table 2.expr x = table 1.expr y [AND,OR] ..)

ASC/DESC



Subcereri pe clauza ORDER BY

Exemplu:

- Daca vrem sa facem o lista cu angajatii din departamentele 10 si 20, ordonati descrescator tinand cont de numarul lor pe fiecare departament, folosim cererea urmatoare:

```
SQL> SELECT id_dep, nume, functie FROM angajati a  
WHERE id_dep in (10,20)  
ORDER BY  
      (SELECT count(*)  
FROM angajati b  
WHERE a.id_dep=b.id_dep)  
DESC;
```



Operatori in subcereri

➤ Operatori in subcereri

- Operatorii prezentati pentru cereri sunt valabili si pentru subcereri, dar mai sunt si alti operatori specifici.
- ✓ **Operatorii SOME/ANY si ALL**
 - Acesti operatori sunt folositi in subcereri care intorc mai multe linii si sunt folositi impreuna cu operatorii logici in clauzele WHERE si HAVING.
 - Operatorul SOME (sau sinonimul lui ANY) compara o expresie cu fiecare valoare returnata de o subcerere si pastreaza liniile unde expresia indeplineste conditia impusa de operatorul logic.
 - Daca este folosit impreuna cu operatorul logic > , atunci are semnificatia de *mai mare decat minim*.



Operatori in subcereri

Exemplu:

- Pentru a afla care sunt angajatii care au salariul mai mare decat cel mai mic salariu pentru functia de programator, folosim cererea urmatoare:

```
SQL> SELECT id_dep, nume, functie, salariu  
FROM angajati  
WHERE salariu > SOME (SELECT DISTINCT salariu  
FROM angajati  
WHERE functie='PROGRAMATOR')  
ORDER BY id_dep, nume;
```



Operatori in subcereri

- Operatorul ALL lucreaza similar cu operatorii SOME/ANY, iar daca este folosit impreuna cu operatorul logic > , atunci are semnificatia de *mai mare decat maxim*.

```
SQL> SELECT id_dep, nume, functie, salariu  
      FROM angajati  
      WHERE salariu > ALL ( SELECT DISTINCT salariu  
                           FROM angajati  
                           WHERE functie='PROGRAMATOR')  
      ORDER BY id_dep, nume;
```



Operatori in subcereri

✓ Operatorii EXISTS si NOT EXISTS

- Acesti operatori sunt folositi adesea in subcereri corelate si testeaza daca subcererea returneaza cel putin o valoare pentru EXISTS, sau niciuna in cazul lui NOT EXISTS, returnad TRUE sau FALSE.

Exemplu:

- Pentru a afla care departamente au cel putin un angajat, folosim cererea urmatoare:

```
SQL> SELECT d.id_dep, d.den_dep  
      FROM departamente d  
      WHERE EXISTS ( SELECT nume  
                     FROM angajati  
                     WHERE id_dep=d.id_dep)  
      ORDER BY id_dep;
```



Operatori in subcereri

- Trebuie specificat ca o constructie cu EXISTS este mult mai eficienta decat o constructie cu IN, SOME/ANY sau ALL, deoarece in cazul in care folosim tabele temporare acestea nu sunt indexate, ducand la scaderea considerabila a performantelor de interogare.
- Performanta depinde de folosirea indecsilor, de dimensiunea tabelelor din baza de date, de numarul de linii returnate de subcerere si daca sunt necesare tabele temporare pentru a evalua rezultatele returnate.
- Desi o subcerere cu o constructie pe operatorul NOT IN poate fi la fel de eficienta ca si in cazul unei constructii pe NOT EXISTS, cea din urma este totusi mult mai sigura, daca subcererea intoarce si valori NULL.



Operatori in subcereri

- In cazul operatorului NOT IN, conditia se evalueaza la FALSE cand in lista de comparatii sunt incluse valori NULL.

Exemplu:

- Daca vrem sa facem o lista cu angajatii care nu sunt sefi si folosim o constructie cu NOT IN, cererea nu va intoarce nicio inregistrare, ceea ce este fals:

```
SQL> SELECT id_dep, id_ang, nume, functie, id_sef  
      FROM angajati a  
      WHERE id_ang NOT IN ( SELECT DISTINCT id_sef  
                           FROM angajati)  
      ORDER BY id_dep;
```

- **no rows selected**



Actualizarea datelor prin subcereri

Exemple:

- Urmatoarea comanda actualizeaza comisionul angajatilor la 10% din salariul minim din departamentul din care fac parte, dar numai pentru angajatii care nu au primit comision:

```
SQL> UPDATE angajati a
```

```
SET a.comision=(SELECT min(salariu)*0.1 FROM angajati b  
WHERE a.id_dep=b.id_dep)
```

```
WHERE nvl(comision,0)=0 ;
```

- Pentru a crea o tabela cu angajatii care au ecusonul mai mic decat cel mai mic ecuson de sef pe fiecare departament folosim comanda:

```
SQL> CREATE TABLE ecusoane_old (id_dep,id_sub,nume_sub, id_sef) as  
SELECT id_dep, id_ang, nume,id_sef FROM angajati a  
WHERE a.id_ang <(SELECT min(b.id_sef)  
FROM angajati b WHERE b.id_dep = a.id_dep);
```

- Pentru a face o copie a tablei angajati executam comanda:

```
SQL> CREATE TABLE angajati_copy AS SELECT * FROM angajati;
```



Actualizarea datelor prin subcereri

- Daca vrem sa stergem angajatii care nu sunt sefi din tabela copie, executam urmatoarea comanda:

```
SQL> DELETE FROM angajati_copy  
      WHERE id_ang NOT IN (SELECT DISTINCT id_sef FROM angajati  
                          WHERE id_sef IS not null);
```

8 rows deleted.

Observatie:

- Daca nu se pune conditia in subcerere ca ecusonul sefului sa nu fie null, rezultatul va fi eronat:

```
SQL> DELETE FROM angajati_copy  
      WHERE id_ang NOT IN (SELECT DISTINCT id_sef FROM angajati);
```

0 rows deleted.

- Rezultatul eronat este din cauza ca exista un angajat care nu are sef, adica are valoarea NULL pentru ID_SEF (presedintele companiei).



Reguli in subcereri

- Cand folosim subcereri, trebuie sa respectam cateva reguli:
 - Cererea interioara trebuie sa fie inclusa intre paranteze si trebuie sa fie in partea dreapta a conditiei;
 - Expresiile din clauza SELECT a subcererii trebuie sa fie in aceeasi ordine ca cele din lista de expresii din clauza WHERE a cererii principale(avand acelasi tip si numar de expresii);
 - Subcererile nu pot fi ordonate, deci nu contin clauza ORDER BY;
 - Clauza ORDER BY poate sa fie pusa la sfarsitul cererii principale;
 - Subcererile sunt executate de la cea mai adanca imbricare pana la nivelul principal de imbricare(cu exceptia cererilor corelate);
 - Subcererile pot folosi functii de grup si clauza GROUP BY ;
 - Subcererile pot fi inlantuite cu predicate multiple AND sau OR in aceeasi cerere externa;
 - In subcereri se pot folosi operatori de multimi;
 - Subcererile pot fi imbricate pana la nivelul 255.



Capitolul 11

Structuri de stocare a datelor



Structuri de stocare a datelor

- Comenzile de definire a datelor sunt oferite de catre limbajul **DDL** (*Data Definition Language*).
Acele comenzi acorda facilitati pentru:
 - Crearea, modificarea si stergerea tabelelor in baza de date;
 - Crearea constrangerilor de integritate si a relatiilor dintre obiectele bazei de date;
 - Definirea vederilor pe tabelele bazei de date;
 - Crearea si administrarea indecsilor;



Crearea si definirea stucturilor tabelare

- **Crearea si definirea stucturilor tabelare**
 - Comenzile pentru crearea si definirea de structuri tabelare sunt comenzi de tip DDL si permit crearea, definirea de constrangeri si relationarea lor intr-o baza de date.
 - Structura unei tabele este data de urmatoarele specificatii de definire:
 - definirea coloanelor;
 - definirea constrangerilor de integritate;
 - definirea tablespace-ului unde se creeaza;
 - definirea parametrilor de stocare;



Crearea si definirea stucturilor tabelare

- Sintaxa comenzii de creare a unei tabele este urmatoarea(parametrii dintre paranteze drepte sunt optionalii):

```
CREATE TABLE [schema.] table_name  
column_name  
column datatype [ DEFAULT expr ]  
[column_constraints]  
[ table_constraints ][ TABLESPACE tablespace]  
[ storage parameters ]  
[ ENABLE/DISABLE clause] [ AS subquery ]
```



Crearea si definirea stucturilor tabelare

unde :

- **schema** – este schema unde se creeaza tabela (specifica userul si baza de date);
- **table_name** – este numele tablei;
- **column** – este numele coloanei;
- **datatype** – reprezinta tipul coloanei;
- **DEFAULT expr** – specifica valoarea implicita a coloanei;
- **column_constraints** – defineste constrangerile de integritate pentru coloana;
- **table_constraints** - defineste constrangerile de integritate la nivel de tabela;
- **tablespace** – specifica in ce tablespace al bazei de date se creeaza tabela.



Crearea si definirea stucturilor tabelare

- **storage parameters** – defineste parametrii de creare si pot fi:
 - PCTFREE –procentaj de spatiu rezervat pentru update;
 - PCTUSED – procent minim folosit pentru un bloc de date;
 - INITTRANS – numarul initial de tranzactii pentru fiecare bloc(1-255);
 - MAXTRANS- numarul maxim de tranzactii concurente (1-255);
 - CLUSTER – specifica daca tabela face parte dintr-un cluster.
- **ENABLE/DISABLE clause** – activare/dezactivare de constrangeri;
- **AS subquery** – inserare de date dintr-o alta tabela obtinute printr-o interogare.



Crearea si definirea stucturilor tabelare

- Tipurile de date care pot fi asociate coloanelor unei tabele pot fi :
 - numerice;
 - alfanumerice;
 - data calendaristica si timp;
 - compuse (matrice sau tabela) .
- Cateva dintre cele mai uzuale tipuri sunt:
- **NUMBER** – numar real de dimensiune variabila (maxim 38 cifre);
- **NUMBER(n)** – numar intreg de **n** cifre;
- **NUMBER(n,m)** – numar real de **n** cifre din care **m** zecimale;
- **CHAR(n)** – sir de caractere de lungime fixa **n** (1- 2000);



Crearea si definirea stucturilor tabelare

- **NCHAR(n)** – analog cu CHAR dar poate stoca siruri de caractere nationale;
- **VARCHAR2(n)** – sir de caractere de lungime variabila **n** (1-4000);
- **NVARCHAR2(n)** – analog cu VARCHAR dar poate stoca siruri de caractere;
- **LONG** – sir de caractere de maxim 2 la puterea 31 octeti;
- **LONG RAW** – similar cu LONG dar contine date binare ;
- **ROWID** – poate stoca identificatorul unei linii din tabela;
- **DATE** – data calendaristica;
- **TIMESTAMP(n)** – extensie pentru tipul DATE si contine si fractiuni de secunda pe **n** zecimale .



Crearea si definirea stucturilor tabelare

- Cand se creeaza o tabela trebuie respectate anumite reguli cum ar fi :
 - Userul trebuie sa aiba drept de creare de tabele (privilegiul CREATE TABLE);
 - Numele tablei trebuie sa fie unic in contul in care se creeaza si nu este case sensitive;
 - Numele trebuie sa aiba maxim 30 caractere continue, sa inceapa cu o litera si sa nu fie cuvant rezervat Oracle;
 - O tabela poate fi creata oricand si nu poate fi alterata cand este accesata de un alt user;
 - La creare nu trebuie specificata dimensiunea tablei dar trebuie estimat ce spatiu va ocupa in tablespace;
 - Structurile tabelare pot fi modificate si ulterior(adaugare sau stergere de coloane, constrangeri, indecsi, etc).



Crearea si definirea stucturilor tabelare

- Daca dimensiunea initiala este insuficienta i se alocă automat mai mult spatiu in limita tablespace-lui (care la randul lui poate fi extins cu un nou data_file);
- Clauza DEFAULT poate contine constante numerice, sir de caractere , functii (inclusiv *sysdate* si *user*) dar nu poate contine numele unei alte coloane sau pseudocoloane);
- Valoarea DEFAULT este luata in considerare in cazul inserarii unei linii in care nu se specifica nimic in legatura cu coloana respectiva.



Crearea si definirea stucturilor tabelare

Exemplu:

```
SQL> CREATE TABLE studenti  
( facultate char(30) DEFAULT 'Automatica si Calculatoare',  
departament char(30) DEFAULT 'Calculatoare',  
matricol number(5),  
nume varchar2(30),  
CNP number(13),  
data_nastere date,  
an_univ varchar2(9) DEFAULT 2021-2022,  
media_admitere number(5,2) ,  
adresa varchar2(50),  
operator varchar2(20) DEFAULT user,  
data_operare date DEFAULT sysdate );
```

- In cazul inserarii unei linii, valorile campurilor operator si data operare nu trebuie specificate deoarece sunt preluate prin functiile sistem *user* si *sysdate*.



Crearea si definirea stucturilor tabelare

✓ Crearea unei tabele printr-o cerere SELECT

- In acest caz tabela va fi creata in urma unei cereri SELECT si va contine si liniile returnate de cerere. Sunt doua posibilitati de a crea o tabela folosind o astfel de constructie :
- Comanda CREATE TABLE nu contine descrierea structurii tablei si in acest caz :
 - Se creeaza o tabela cu o structura identica cu campurile specificate in cererea SELECT;
 - Tipurile coloanelor se pastreaza;
 - Daca cererea contine o expresie sau functie trebuie sa i atribuie un alias valid;
 - Noua tabela nu mosteneste nicio constrangere de integritate de la vechile tabele(cu exceptia celei not null).



Crearea si definirea stucturilor tabelare

Exemplu:

- Sa cream o noua tabela care va contine veniturile angajatilor din departamentul 20, folosind tabela angajati:

```
SQL> CREATE TABLE depart_20 AS  
      SELECT id_ang, nume, data_ang,  
             salariu+nvl(comision,0) venit  
      FROM angajati WHERE id_dep=20;
```

- Noua tabela DEPART_20 va avea urmatoarea structura :

<u>Name</u>	<u>Null?</u>	<u>Type</u>
ID_ANG	NOT NULL	NUMBER(4)
NUME		VARCHAR2(30)
DATA_ANG		DATE
VENIT		NUMBER

- Cu toate ca **ID_ANG** este creata cu optiunea **NOT NULL** coloana nu este definita ca o cheie primara sau unica.



Crearea si definirea stucturilor tabelare

Exemplu:

- Sa cream o tabela cu primele angajatilor din departamentul 30 daca prima reprezinta 15% din venitul lunar:

```
SQL> CREATE TABLE depart_30
      (depart  DEFAULT 30,
       nume    NOT NULL,
       data_ang,
       prima  )
      AS
      SELECT id_dep, nume, data_ang,
             round(0.15*(salariu + nvl(comision,0)))
      FROM angajati
      WHERE id_dep=30;
```



Constrangeri de integritate

➤ Constrangeri de integritate

- Constrangerile de integritate sunt restrictii care trebuie respectate la nivel de tabela sau in relatiile cu alte tabele ;
- Aceste reguli sunt verificate automat in cazul operatiilor de inserare, stergere, modificare si, in cazul in care nu se valideaza, se genereaza o eroare si tranzactia este anulata.
- Constrangerile de integritate pot fi:
 - **NOT NULL** - inregistrarile nu pot contine valori nule;
 - **UNIQUE** - defineste o valoare unica pe una sau mai multe coloane (nu pot fi mai multe inregistrari cu aceleasi valori pe coloanele respective, dar accepta valori nule);
 - **PRIMARY KEY** - defineste o cheie primara pe o coloana sau coloane multiple (nu pot fi mai multe inregistrari cu aceeasi cheie primara);
 - **FOREIGN KEY** - defineste o cheie externa (tabela se relationeaza cu alta tabela pe o cheie primara, cheie unica sau coloane cu unicitate);
 - **CHECK** - forteaza o conditie pe coloana, conditia putand sa contina si functii (cu unele exceptii, de exemplu sysdate si user).



Constrangeri de integritate

- Caracteristici ale constrangerilor de integritate:
 - Fiecare constrangere va avea un nume dat de user sau generat automat de catre sistem ;
 - Constrangerile pot fi activate sau dezactivate cu comanda ALTER TABLE ;
 - Constrangerile pot fi adaugate sau sterse si dupa ce o tabela a fost creata ;
 - Informatiile legate de constrangeri se pastreaza in dictionarul de date;
 - Cand se creeaza copii de tabele sau tabele create cu subcereri nu se preiau automat si constrangerile de integritate create pe tabelele de referinta(cu exceptia NOT NULL).



Constrangeri de integritate

- ✓ Constrangerea **NOT NULL**:
 - Se aplica la nivel de coloane si verifica daca inregistrarile au valoarea nula pe coloanele respective, fortand un cod de eroare care anuleaza tranzactia ;
 - Cand se creeaza constrangeri pe o cheie primara se creeaza automat si o constrangere NOT NULL pe coloanele respective (o cheie primara nu trebuie sa contina valori nule pe coloanele care o definesc).

Exemplu:

```
SQL> CREATE TABLE functii  
    ( cod_functie number(2)  CONSTRAINT NL NOT NULL,  
      den_functie varchar2(20) NOT NULL,  
      data_vigoare date );
```



Constrangeri de integritate

✓ Constrangerea **UNIQUE**:

- Verifica ca o coloana, sau perechi de coloane, sa nu contina valori duplicate ;
- Verificarea se face numai pentru inregistrarile cu valori nenule deoarece constrangerea permite inserarea de valori nule in coloanele respective. Daca se asociaza cu constrangerea NOT NULL se creeaza cheie unica;
- In mod automat se creeaza si un index pe coloanele definite chei unice, ceea ce duce la marirea vitezei de interogare pe tabela ;
- Daca constrangerea se creeaza pe mai multe coloane atunci setul de date trebuie sa fie unic.
- Sintaxa pentru definirea la nivel de coloana este:

column_name datatype [CONSTRAINT constraint_name] UNIQUE

- Sintaxa pentru definirea la nivel de tabela(ultima linie) este:
[, CONSTRAINT constraint_name] UNIQUE (col1[, col2, [...]])



Constrangeri de integritate

Exemple:

- Sa cream tabela JOBURI definind unicitate pe anumite coloane:

```
SQL> CREATE TABLE joburi
```

```
( cod_job number(2)    CONSTRAINT UK_FUN UNIQUE,  
  den_job varchar2(20)  UNIQUE,  
  data_vigoare date ) ;
```

- Putem sa facem si urmatoarea constructie la nivel de tabela:

```
SQL> CREATE TABLE joburi
```

```
( cod_job number(2),  
  den_job varchar2(20),  
  data_vigoare date,  
  CONSTRAINT UK_FUN UNIQUE (cod_functie,den_functie) );
```

- In prima constructie putem insera oricate inregistrari cu valori nule pe coloanele definite unice, dar in constructia a doua nu putem sa inseram inregistrari cu una din coloane nula si cealalta sa nu respecte unicitatea.



Constrangeri de integritate

- ✓ Constrangerea **PRIMARY KEY**:
 - Se foloseste pentru definirea cheii primare pe una sau mai multe coloane;
 - O tabela poate avea o singura constrangere de tip PRIMARY KEY si nu accepta valori nule pe coloana sau coloanele care o definesc ;
 - Cand se creeaza o cheie primara se creeaza in mod automat si o constangere de timp NOT NULL si UNIQUE ;
 - Cand se creeaza o cheie primara se creeaza in mod automat si un index pe coloanele care o definesc(care imbunatateste timpul de interogare).
 - Sintaxa pentru definirea la nivel de coloana este:
column_name datatype [CONSTRAINT constraint_name] PRIMARY KEY
 - Sintaxa pentru definirea la nivel de tabela(ultima linie) este:
[, CONSTRAINT constraint_name] PRIMARY KEY(col1[, col2, [...]])



Constrangeri de integritate

Exemple:

- Daca vrem sa creem un nomenclator de functii in care fiecare functie sa aiba cod unic folosim urmatoarea constructie :

```
SQL> CREATE TABLE nom_functii
```

```
( cod_functie number(2) CONSTRAINT PK PRIMARY KEY,  
  den_functie varchar2(20),  
  data_vigoare date );
```

- Daca dorim ca un cod de functie sa poata fi utilizat de mai multe ori trebuie sa definim cheia pe mai multe coloane, de exemplu pe codul functiei si data la care intra in vigoare codificarea.

```
SQL>CREATE TABLE nom_functii_multiple
```

```
( cod_functie number(2) ,  
  den_functie varchar2(20),  
  data_vigoare date,  
  CONSTRAINT PK_FUN PRIMARY KEY (cod_functie,data_vigoare) )
```




Constrangeri de integritate

✓ Constrangerea **FOREIGN KEY**:

- Se foloseste pentru a relationa o tabela cu una sau mai multe tabele, verificand daca valorile continute in coloanele definite de FOREIGN KEY (numita cheie straina sau cheie externa) sunt cuprinse in valorile coloanelor altei tabele care trebuie sa fie definite ca UNIQUE sau PRIMARY KEY.
- Sintaxa pentru definirea la nivel de coloana este:

**column_name datatype [CONSTRAINT constraint_name]
REFERENCES table(column)
[on DELETE CASCADE | DELETE SET NULL]**

- Sintaxa pentru definirea la nivel de tabela(ultima linie) este:

**[, CONSTRAINT constraint_name] FOREIGN KEY(col1[, col2, [...]])
REFERENCES table(column)
[on DELETE CASCADE | DELETE SET NULL]**



Constrangeri de integritate

- Reguli pentru constrangerea FOREIGN KEY:
 - Inserarea unei linii intr-o tabela relationata (pe care am definit FOREIGN KEY) se poate face numai daca exista o singura linie in tabela de referinta (in care am definit PRIMARY KEY sau UNIQUE) corespunzator coloanelor de relationare ;
 - Stergerea unei linii din tabela de referinta nu se poate face atata timp cat exista linii relationate pe linia respectiva in tabela relationata ;
 - Regulile de mai sunt valabile si in cazul relationarii pe coloane;
 - Pentru a putea sterge in tabela de referinta linii referite in alte tabele se foloseste optiunea ON DELETE CASCADE. In acest caz, cand se sterge o linie din tabela de referinta se vor sterge toate liniile din tabelele relationate care sunt in relatie cu linia respectiva ;
 - In cazul unei tabele relationata cu ea insasi, stergerea unei linii care este referita duce la aparitia de valori nule pe coloanele din liniile relationate(de exemplu, in tabela angajati, cand se sterge linia aferenta unui sef, toti angajatii care au seful respectiv vor avea valoare null pe coloana id_sef) ;
 - Folosind optiunea ON DELETE SET NULL, coloanele de relatie din tabela relationata devin nule si nu sunt sterse liniile relationate atunci cand se sterge o linie din tabela de referinta.



Constrangeri de integritate

Exemplu:

- O constructie de cheie externa la nivel coloana si tabela arata astfel:

```
SQL> CREATE TABLE angajati_dep
```

```
( id_ang      number(4)    PRIMARY KEY,  
  nume        varchar2(30),  
  functie     varchar2(20) REFERENCES joburi(den_job),  
  id_sef      number(4)    REFERENCES angajati_dep(id_ang),  
  data_ang    date,  
  salariu     number(7,2),  
  comision    number(7,2),  
  id_dep      number(2) ,  
  CONSTRAINT FK_ANG FOREIGN KEY (id_dep)  
  REFERENCES departamente(id_dep) );
```

- Tabela DEPARTAMENTE trebuie sa aiba definita cheie primara sau unicitate pe id_dep:

```
SQL> ALTER TABLE departamente ADD CONSTRAINT nr_dep PRIMARY KEY(id_dep);
```



Trunchierea datelor dintr-o tabela

➤ Comanda **TRUNCATE**

Este folosita pentru trunchierea (golirea de date) a unei tabele si are sintaxa:

```
TRUNCATE TABLE [schema.] table_name  
[DROP STORAGE] [REUSE STORAGE]
```

unde :

- **schema** – este schema unde este creata tabela (specifica userul si baza de date);
- **table_name** – este numele tabelei;
- **DROP STORAGE** – elibereaza spatiul rezultat din stergerea liniilor;
- **REUSE STORAGE** – pastreaza spatiul rezultat din stergerea liniilor alocat tabelei.



Trunchierea datelor dintr-o tabela

Exemplu:

```
SQL> TRUNCATE TABLE angajati_copy;
```

Observatii:

- Trunchierea unei tabele cu optiunea REUSE STORAGE este utila atunci cand se folosesc in mod uzual tabele temporare in diferite prelucrari si exista riscul ca spatiul pe disc sa fie prea mic la o noua rulare. Acest risc apare si atunci cand se sterge o tabela temporara cu comanda DROP si se recreeaza, daca intre timp spatiul pe disc nu mai are dimensiunea necesara numarului de linii rezultate din prelucrari.
- Cand se foloseste comanda TRUNCATE datele nu mai pot fi recuperate cu comanda ROLLBACK, aceasta facilitate este oferita numai in cazul in care se foloseste comanda DELETE pentru stergerea datelor.
- Daca dupa comanda DELETE se da comanda COMMIT datele nu mai pot fi refacute cu comanda ROLLBACK, deoarece tranzactiile au fost executate pe baza de date.



Crearea si utilizarea vederilor

➤ Crearea si utilizarea vederilor

- Vederea (*view*) este un tip de obiect care se poate crea prelucrând date stocate în tabelele bazei de date.
- O vedere se creează prin asocierea unei cereri SELECT cu un nume de obiect stocat în baza de date.
- O vedere se comportă ca o tabelă în cazul execuției de cereri SELECT și poate fi folosită uneori pentru efectuarea de actualizări ale bazei de date.
- Există însă și diferențe între o vedere și o tabelă, mai ales din punct de vedere al implementării:



Crearea si utilizarea vederilor

- In cazul unei vederi, baza de date stocheaza definitia acesteia (cererea SQL ca sir de caractere) si nu datele regasite prin aceasta(este o tabela logica, nu fizica, cu exceptia unui view materializat).
- Ori de cate ori se executa o cerere avand la baza o vedere, ea este recalculata. Astfel, orice modificare efectuata in tabelele pe baza carora e definita vederea se reflecta automat in aceasta.
- Structura unei vederi nu se poate modifica prin cereri de tip ALTER ci prin recrearea vederii cu specificarea unei alte cereri SQL.
- Constrangerile de integritate nu se pot defini la nivel de vedere.
- O vedere mosteneste implicit toate constrangerile definite pe coloanele tabelelor pe baza carora este definita prin cererea asociata ei.



Crearea si utilizarea vederilor

- Vederile permit implementarea de scheme externe prin care diverse categorii de utilizatori acceseaza doar acele portiuni din baza de date pe care le folosesc in mod obisnuit, asigurandu-se astfel:
 - Confidentialitatea datelor - utilizatorii care acceseaza baza de date doar prin intermediul unor vederi pot fi restrictionati in ceea ce priveste datele pe care le pot utiliza prin insasi definitia vederilor respective (ele nu contin coloanele/liniile/datele la care nu se doreste accesul acelei categorii de utilizatori).
 - Corectitudinea datelor - prin faptul ca un utilizator nu are acces la date pe care uzual nu le foloseste, impiedicand coruperea accidentala a celorlalte date din cauza necunoasterii semnificatiei lor.



Crearea vederilor

- Sintaxa comenzii de creare a unei vederi este urmatoarea(parametrii dintre parantezele drepte sunt optionalii):

CREATE [OR REPLACE]

VIEW [schema.] view_name [(column_list)]

AS query_statement;

unde:

- **view_name** - specifica numele asociat vederii respective. Respecta aceleasi conditii ca in cazul numelor de tabele.
- **query_statement** - cererea SQL de tip SELECT asociata vederii.



Crearea vederilor

- **schema** – este numele schemei(userului) asociata vederii.
- **column_list** - specifica numele coloanelor din vedere. Acestea trebuie sa respecte restrictiile uzuale pentru nume de coloane descrise in capitolele precedente (30 de caractere, etc.). In lipsa acestei optiuni coloanele vederii au aceleasi nume cu ale rezultatului cererii SQL asociate. In cazul in care rezultatul cererii asociate vederii are nume de coloane care nu respecta restrictiile privind astfel de nume, este obligatorie, fie folosirea listei_de_coloane, fie modificarea cererii prin adaugarea unor aliasuri de coloana corespunzatoare.



Crearea vederilor

- **OR REPLACE** - in cazul in care exista deja o vedere cu acel nume, se face suprascriere. Este modul uzual prin care se modifica structura si continutul unei vederi. In lipsa acestei optiuni sistemul semnaleaza in astfel de situatii o eroare.
- **READ ONLY** – specifica ca datele pot fi citite dar nu se permite adaugarea sau modificarea de date.



Crearea vederilor

Exemple:

- Sa cream un view care contine date despre angajatii din departamentul 10:

```
SQL> CREATE OR REPLACE VIEW angajati_10 AS  
      SELECT * FROM angajati WHERE id_dep=10;
```

View created.

- Urmatoarea comanda creeaza un view care contine numai anumite date despre angajati:

```
SQL> CREATE OR REPLACE VIEW date_angajare AS  
      SELECT id_ang, nume, data_ang, functie FROM angajati;
```

View created.



Crearea vederilor

- Un view poate fi creat si prelucrand date din alt view:

```
SQL> CREATE OR REPLACE VIEW joburi_view AS  
      SELECT id_ang ecuson, nume nume_ang, functie job  
      FROM date_angajare;
```

View created.

Observatii:

- O vedere se poate folosi pentru definirea altor vederi (JOBURI_VIEW este definita folosind vederea DATE_ANGAJARE).
- Numele coloanelor din vederea JOBURI_VIEW sunt altele decat cele din tabela ANGAJATI si vederea DATE_ANGAJARE, ca urmare a folosirii unor aliasuri in cererea de creare.



Crearea vederilor

- Se pot defini vederi avand asociate cereri SELECT pe mai multe tabele:

```
SQL> CREATE OR REPLACE VIEW angajati_depart AS  
      SELECT a.den_dep, b.numa, b.data_ang, b.functie job  
      FROM departamente a, angajati b  
      WHERE a.id_dep=b.id_dep;
```

Observatii:

- In acest caz trebuie folosite metode de JOIN sau subcereri;
- Astfel de vederi sunt folosite pentru a regasi date si nu pentru modificarea acestora. Ele pot usura dezvoltarea de aplicatii prin utilizarea cererilor care folosesc vederi.



Modificarea datelor in vederi

- ✓ **Modificarea datelor prin intermediul vederilor**
- Deoarece o vedere nu are date proprii si se calculeaza de fiecare data pe baza cererii SQL asociata, orice modificare este efectuata direct in tabelele bazei de date care sunt specificate in definitia sa. Din aceasta cauza nu orice vedere poate fi folosita pentru modificarea datelor, ci doar cele pentru care sistemul poate detecta exact care linii ale acestora sunt afectate de operatia respectiva.
- In cazul modificarii datelor prin vederi, mecanismele de tranzactie descrise anterior se comporta exact ca in cazul modificarii tabelelor.



Modificarea datelor in vederi

- Comenzile DML trebuie sa respecte un set de restrictii pe care o vedere trebuie sa le indeplineasca pentru a putea fi folosita in operatiile de inserare, modificare si stergere. Acestea sunt diferite dupa tipul operatiei:

- ✓ **Stergere si actualizare**

- Cererile DELETE si UPDATE se pot aplica pe o vedere daca aceasta nu contine:
 - Functii de grup (SUM, MIN, MAX, COUNT, etc.);
 - DISTINCT;
 - GROUP BY;
 - HAVING;
 - UNION or UNION ALL;



Modificarea datelor in vederi

Exemple:

- Urmatoarele vederi vor da erori la creare si nu pot fi folosite pentru stergerea/actualizarea datelor deoarece incalca una sau mai multe dintre restrictiile de mai sus:

a) Vedere care contine DISTINCT, grupare si functii de grup:

```
SQL> CREATE OR REPLACE VIEW salarii_max AS  
      SELECT DISTINCT id_dep, MAX(salariu)  
      FROM angajati  
      GROUP BY id_dep;
```

ERROR at line 2:

ORA-00998: must name this expression with a column alias

- Pentru salariul maxim este obligatoriu un alias de coloana deoarece MAX(salariu) nu respecta conventia pentru nume de coloane (contine paranteze).



Modificarea datelor in vederi

b) Vedere care contine literali si join:

```
SQL> CREATE OR REPLACE VIEW functii_ang AS  
      SELECT a.id_dep depart, a.den_dep depart,  
             b.numa numa_ang, b.functie job  
      FROM departamente a, angajati b  
      WHERE a.id_dep=b.id_dep;
```

ERROR at line 2:

ORA-00957: duplicate column name

- Eroarea apare deoarece coloana DEPART ar duce la crearea unei vederi avand doua coloane cu aceeasi denumire.



Modificarea datelor in vederi

- Prin intermediul vederii SALARII_INDEXATE definita in continuare se pot actualiza toate coloanele vederii care provin din coloane ale tablei ANGAJATI, dar nu se poate specifica modificarea coloanei suplimentare SALARIU_NOU definita de expresia aritmetica $SALARIU * 1.1$:

```
SQL> CREATE OR REPLACE VIEW salarii_indexate AS  
      SELECT id_dep, nume, data_ang, functie,  
             salariu*1.1 salariu_nou  
      FROM angajati;
```

View created.



Modificarea datelor in vederi

- Urmatoarea comanda va modifica functia atat in view-ul SALARII_INDEXATE cat si in tabela ANGAJATI:

```
SQL> UPDATE salarii_indexate  
      SET functie='INGINER'  
      WHERE nume='TACHE IONUT';
```

1 row updated.

- Daca se face insa o modificare de salariu se va genera o eroare deoarece se incearca actualizarea unei coloane din view care a fost definita printr-o expresie(coloana virtuala):

```
SQL> UPDATE salarii_indexate  
      SET salariu_nou=2500  
      WHERE nume='TACHE IONUT'
```

ERROR at line 2:

ORA-01733: virtual column not allowed here



Inserarea datelor in vederi

➤ Inserarea datelor in vederi

Pentru a se putea insera noi linii prin intermediul unei vederi, aceasta trebuie sa satisfaca urmatoarele restrictii:

- Vederea trebuie sa contina cheia tabeli de baza.
- Inserarea de date nu se face in coloane virtuale care provin din expresii sau literal (fiecare coloana provine dintr-o coloana a tabeli de baza).
- Inserarea datelor intr-o tabela prin intermediul unei vederi se poate face numai prin vederile create pe o singura tabela, respectand toate constrangerile de integritate ale acesteia.



Inserarea datelor in vederi

Exemple:

- Daca se insereaza o linie noua in view-ul ANGAJATI_10 se va insera automat linia si in tabela ANGAJATI:

```
SQL> INSERT INTO angajati_10(id_ang, nume, data_ang, functie)  
VALUES (333,'MITRACHE SILVIU', '15-JAN-2021', 'ECONOMIST');
```

- Daca se face o inserare in view-ul SALARII_INDEXATE se va genera o eroare deoarece coloana SALARIU_NOU a fost definite printr-o expresie:

```
SQL> INSERT INTO salarii_indexate(id_dep, nume, data_ang,  
funcție, salariu_nou)  
VALUES (10,'STROE VICTOR', '1-JUL-2021', 'INGINER', 2300);
```

ERROR at line 2:

ORA-01733: virtual column not allowed here



Stergerea vederilor din dictionar

- Stergerea unei vederi din dictionarul bazei de date se face cu comanda DROP VIEW:

```
SQL> DROP VIEW angajati_dep;
```

- Daca se sterge un view din care s-au create alte view-uri, datele se pierd in cascada:

```
SQL> DROP VIEW date_angajare;
```

View dropped.

```
SQL> SELECT * FROM joburi_view;
```

ERROR at line 1:

ORA-04063: view "SCOTT.JOBURI_VIEW" has errors;

- Fiind o comanda DDL, stergerea unei vederi din dictionar nu poate fi anulata cu comanda ROLLBACK.



Crearea si administrarea indecsilor

➤ Crearea si administrarea indecsilor

- Un index este un obiect creat pe baza de date pentru cresterea performantelor de interogare pe baza de date. Sintaxa comenzii de creare a unui index este urmatoarea:

CREATE

[ONLINE|OFFLINE]

[UNIQUE|FULLTEXT|SPATIAL|BITMAP] INDEX index_name

ON table_name(column_name, ..)

index_options

unde:

- **UNIQUE|FULLTEXT|SPATIAL|BITMAP** – specifica tipul indexului;
- **table_name** – reprezinta numele tablei pe care se creeaza indexul;
- **column_name** – reprezinta numele coloanelor care compun indexul;
- **index_options** – optiuni care se pot specifica la crearea indexului.



Crearea si administrarea indecsilor

Exemple:

- Crearea unui index de tip arbore(B-Tree) cu unicitate pe id_ang:

```
SQL> CREATE INDEX angajati_idx ON angajati(id_ang) PCTFREE 30  
STORAGE(INITIAL 200K NEXT 200K PCTINCREASE 0 MAXEXTENTS  
50) TABLESPACE bd_data;
```

- Crearea unui index de tip BITMAP pe jobul angajatilor :

```
SQL> CREATE BITMAP INDEX functii_idx ON angajati(funcție)  
PCTFREE 30 STORAGE(INITIAL 200K NEXT 200K PCTINCREASE 0  
MAXEXTENTS 50) TABLESPACE bd_data;
```

- Tipul indexului este implicit B-Tree si pentru a modifica parametrii unui index se foloseste comanda ALTER INDEX.
- Stergerea unui index din dictionar se face cu comanda DROP INDEX.
- Indecsii se creeaza automat in momentul in care se creeaza o cheie primara sau unicitate pe coloane.
- Unui index i se asociaza automat o tabela index.